

Verification Continuum™

VC Apps Toolbox User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Audience	7
Customer Support	7
Synopsys Statement on Inclusivity and Diversity	8
run.f	8

1. Introduction	9
Overview	9
VC Apps Overview	9
Design Exploration	10
Design Validation	10
FSDB Investigation	10
Utility	10
Advanced VC Apps	10
Design Manipulation	11
Design Exploration	11
FSDB Investigation	11

2. VC Apps Toolbox Usage	12
Overview	12
Usage	12
Invoke the VC Apps Toolbox	13
Manage Applications	14
Execute Applications	15
Find Applications	17
Configure Applications	18
Access Application Source Files	22
Add Applications and Folders	23
Get Help for Applications	25
Update Applications	25
Practice	26
Appendix A: Built-in Apps in VC Apps Toolbox	30

3. Design Exploration	33
Get Constant Assign	34
Get Equivalent Signal	37
Get FanIn Register	45
Get FanOut Register	52
Get Force Statement	58
Get Generate Scope	60
Get Hierarchy CSV	63
Get Module Hierarchy	66
Get Module IO	69
Get Module Parameter	72
Get Module Signal	75
Get Netlist Overview	80
Get PG Pin Connection	85
Get Port Connection	91
List Include Files	96
List Module Define	99
List Module Files	102
List Module Include Files	105
List Registers	108
List Scope Parameter	112
Report Hierarchical Reference	118
Signal Search	121
List All Files in Design	124
Report VHDL Component	126
Find Instance with Module Def Name	129
Report Exception Paths	132

4. Design Validation	137
Detect Dangling Port	137
Detect Assign Bit Width Mismatch	142
Detect Constant Port	144

Detect Open Port	148
Detect Tri-State	150
Check Module Registered Outputs	153
Check Module Registered Inputs	157
Check Module Port Declaration Using Modport	160
Check Module Port Connections Using Modport	163
Signal Existence Check in Design	166
Module Array Instantiation Check	169
5. FSDB Investigation	172
Detect PortZ	172
Get FSDB Scope Hierarchy	176
Get Register Values	178
Serial Signal to Parallel Value	181
Signal Existence Check in FSDB	185
FSDB Signal Query	188
6. Utility	193
Smart Log	193
Get Instance List	195
Get Signal List	198
Analyze Force Debug Report	201
Macro Utility	204
Report Conditional Macro	204
Detect FSM Macro Define	210
Report Macro Define	213
Analyze Time Text Log	216
Open Time Text Log / Synchronize Waveform Cursor to Log	216
View FSDB Frequency	219
Batch Mode Usage:	219
Arguments:	219
7. Advanced VC Apps	221

Contents

Design Manipulation	221
ChipInt	221
HierMan	224
PortTieConst	236
Ungroup	238
Rename Instance	241
Delete Pass Through	244
Disconnect Port	245
Fast ECO Mode	246
X Rectify	247
Design Exploration	249
Trace Instance Connection	249
Trace Pipeline	254
FSDB Investigation	258
Report Moving Average	258

Preface

This book is a reference manual for the VC Apps Help and provides detailed descriptions and application example for each App. For advanced use of the Native Programming Interface (NPI), refer to *VC Apps - Verdi Native Programming Interface (NPI)* manual and Verdi User Guide and Tutorial document - [Using Verdi Executable to Import Design from UFE](#).

Audience

The audience for this manual includes engineers who are familiar with languages and tools used in design and verification such as Verilog, VHDL, and SystemVerilog.

This document assumes that you have a basic knowledge of the Verdi debug platform and also the platform on which your version of the Verdi platform runs: UNIX or Linux and that you are knowledgeable in C/C++ and Tcl.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

run.f

`-f run.f` stands for all Verdi import design options.

One of them is

```
%cat run.f
-dbdir simv.daidir
```

Or

```
-simBin simv
```

Using Verdi Executable to Import Design from UFE

The Unified Compiler Flow (UFE) uses VCS with the `-kdb` option and the generated `simv.daidir` includes the KDB information. Verdi supports the following use models to import design from `simv.daidir` related files:

- `verdi -dbdir simv.daidir`
- `verdi -simBin simv`
- `verdi -ssf novas.fsdb`

1

Introduction

This section describes about the following topics:

- [Overview](#)
- [VC Apps Overview](#)

Overview

Commonly used VC Apps are pre-packaged in the installation package of the Verdi platform. User can access these VC Apps from VC Apps Toolbox area from the GUI of the Verdi Automated Debug Platform. The VC Apps in VC Apps Toolbox are developed and tested by Synopsys. User can also access the original source code of some Apps in the Toolbox and customize to fit different requirements. VC Apps toolbox also provides flexibility that allow user to add in-house tools into VC Apps Toolbox and share within design team by leveraging the manage.rc mechanism.

VC Apps Overview

VC Apps includes the following categories:

- Design Exploration
- Design Validation
- FSDB Investigation
- Utility
- Advanced VC Apps

The following sections provide the overview of these categories.

- [Design Exploration](#)
- [Design Validation](#)
- [FSDB Investigation](#)

- [Utility](#)
 - [Advanced VC Apps](#)
-

Design Exploration

VC Apps in this category help user to traverse the design information from the Verdi Knowledge Database (KDB) and output the traversed information into a specific format for other purposes.

Refer to the [Design Exploration](#) chapter for details.

Design Validation

VC Apps in this category help user to traverse the design information from the Verdi Knowledge Database (KDB) and check whether the traversed information meets certain design rules.

Refer to the chapter for details [Design Validation](#).

FSDB Investigation

VC Apps in this category help user to traverse information from Verdi Fast Signal Database (FSDB) and output the result into specific format.

Refer to the chapter for details [FSDB Investigation](#).

Utility

Apps in this category are Synopsys tools provided to help user to leverage the information in Verdi for various purposes. The utilities come in binary format, user will not be able to access the original source code of the utilities.

Refer to the chapter for details [Utility](#).

Advanced VC Apps

Advanced VC Apps includes the advanced apps that includes both Design Manipulation, Design Exploration, and FSDB Investigation.

The advanced VC Apps are available in Verdi Apex mode.

Design Manipulation

VC Apps in this category are the advanced VC Apps that helps the user to modify the design content and output the new design file for future uses.

Refer to the [Design Manipulation](#) chapter for details.

Design Exploration

VC Apps in this category are the advanced VC Apps that helps the user to traverse the design information from the Verdi Knowledge Database (KDB) and output the traversed information into a specific format for other purposes.

Refer to the [Design Exploration](#) chapter for details.

FSDB Investigation

VC Apps in this category are the advanced VC Apps that helps users to modify the design content and output the new design file for future use.

Refer to the [FSDB Investigation](#) chapter for details.

2

VC Apps Toolbox Usage

This section describes about the following topics:

- [Overview](#)
- [Usage](#)
- [Practice](#)
- [Appendix A: Built-in Apps in VC Apps Toolbox](#)

Overview

The VC Apps Toolbox is an interface to execute VC applications (VC Apps) from the GUI of the Verdi® Automated Debug Platform. VC Apps are provided by Synopsys personnel and are based on commonly used Verdi NPI (Native Programming Interface) scripts and Verdi Tcl commands. The VC Apps Toolbox further enriches the Verdi feature set with automated and customized toolkits, such as traversing the design or investigating an FSDb file in a fast and efficient way. Users can additionally use the toolkits to create customized applications on demand.

The VC Apps Toolbox mechanism includes the following features:

- Gives a platform to execute and organize VC Apps.
- Provides a general GUI Input Form to input the value of arguments.
- Supports specification of the output file and rule file to be opened by the Smart Log after execution.
- Creates customized applications.
- Updates the latest VC Apps without re-downloading.

Usage

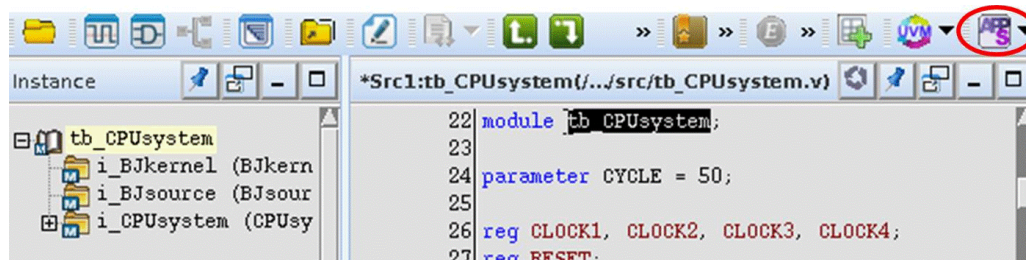
This VC Apps Toolbox includes many NPI scripts. The user interface that is provided to manage these scripts can be invoked from the Verdi platform's main window. The following

sections describe how to invoke the VC Apps Toolbox and how to use the features in the *VC Apps Toolbox* form.

Invoke the VC Apps Toolbox

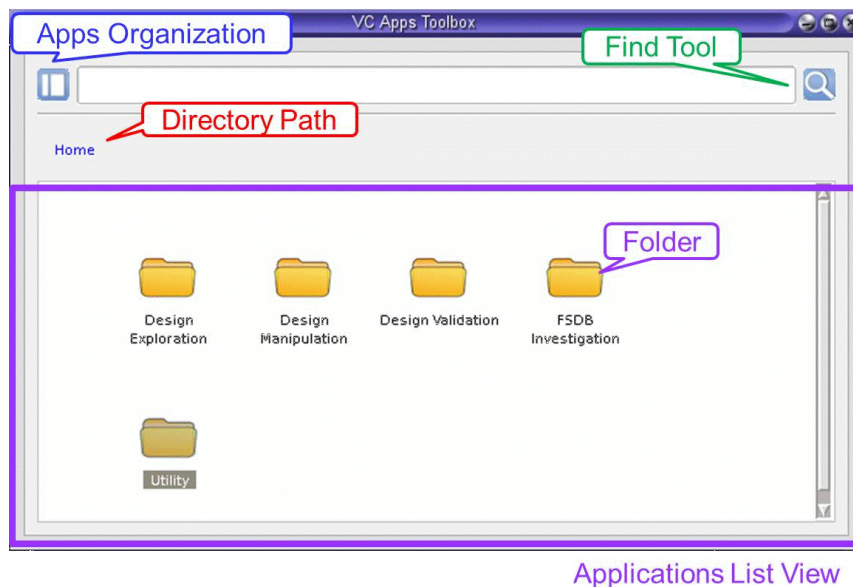
After invoking the Verdi platform, the **VC Apps Toolbox** icon will appear in the upper right corner of the main window.

Figure 1 VC Apps Toolbox Icon



Clicking the **VC Apps Toolbox** icon will open the VC Apps Toolbox form. This form can be used to manage and execute all VC Apps. As shown below, the VC Apps Toolbox form is composed of the *Applications List View* pane, **Directory Path** field, **Find Tool** field and the **Apps Organization** icon by default.

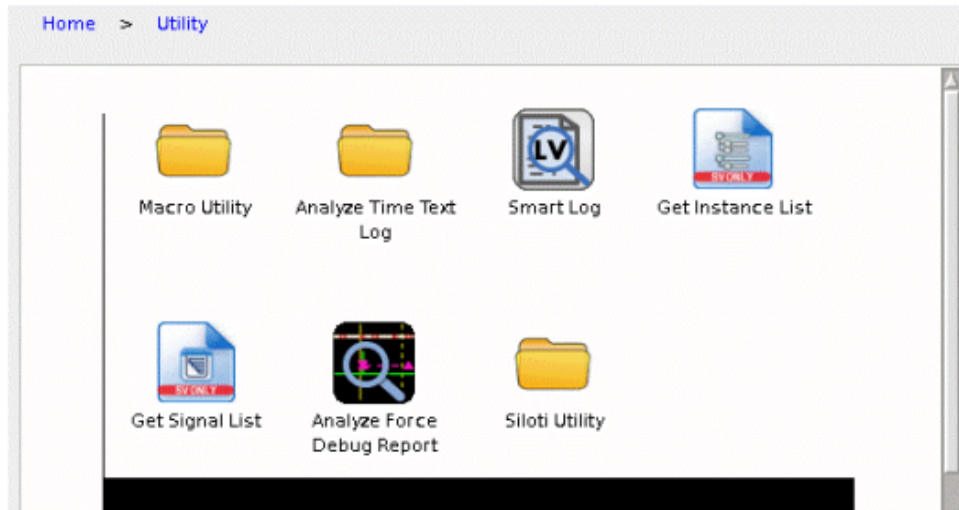
Figure 2 VC Apps Toolbox Form



You can use the **Find Tool** field to locate VC Apps. The **Directory Path** field shows the current directory path. The Applications List View pane displays all the folders and Applications belonging to the current directory. Each VC App has its own unique icon.

In the *Utility* directory, there is a Smart Log application that can show the results of the VC Apps execution with different rules.

Figure 3 *Utility Directory*



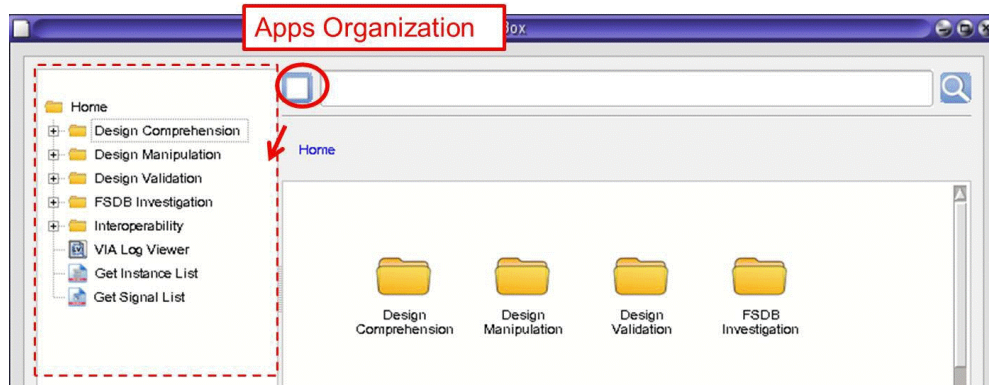
Note:

The details of the *Smart Log* frame is out of the scope of this document.

Manage Applications

Currently, VC Apps are categorized in the following folders: **Design Exploration**, **Design Manipulation**, **Design Validation**, **FSDb Investigation**, and **Utility**. Double-clicking the folder in the VC Apps Toolbox form will push the view into the folder where all VC Apps belonging to this folder are displayed. Clicking the **Apps Organization** icon in the upper left will open the hierarchy browser to view the organization of all VC Apps.

Figure 4 Apps Organization Icon



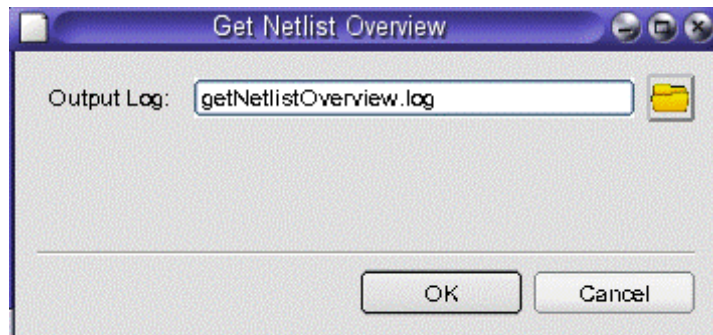
You can change the content of the folder by dragging and dropping the script to another desired location. For example, you can select the **Get Signal List** icon and while holding the left mouse button, drag to your desired location (say under the **Design Manipulation** folder), and then drop the object by releasing the mouse button until the preview border becomes a line under the desired location. You can drag and drop objects from the main window to the hierarchy browser, or move the objects from a location in the hierarchy browser to another location in the same hierarchy browser.

You can delete folders and applications using the **Delete** command in the right-click command menu.

Execute Applications

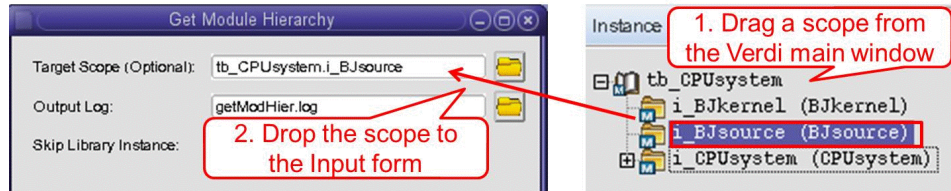
Double-clicking a VC App icon in the VC *Apps Toolbox* form will execute the application. The generic Input form may be opened for users to input additional information, for example, the log file name.

Figure 5 Input GUI with Output Log File Field



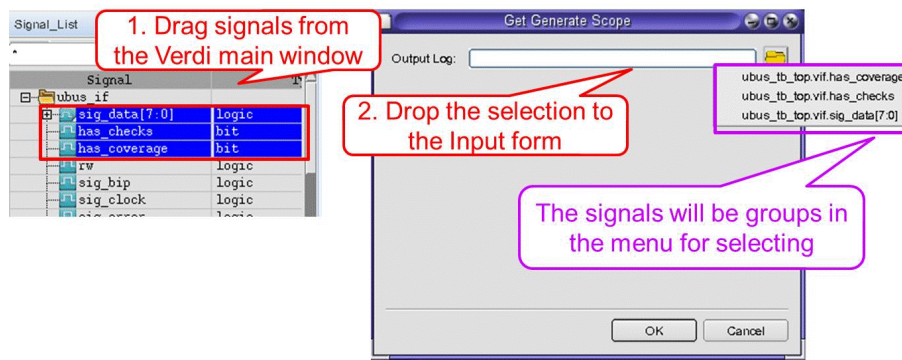
Interaction between the *Input* form and the Verdi main windows is supported. You can directly drag a scope or scopes from a Verdi main window and drop the items to a field in the *Input* form.

Figure 6 Interaction between Input Form and Verdi Main Window



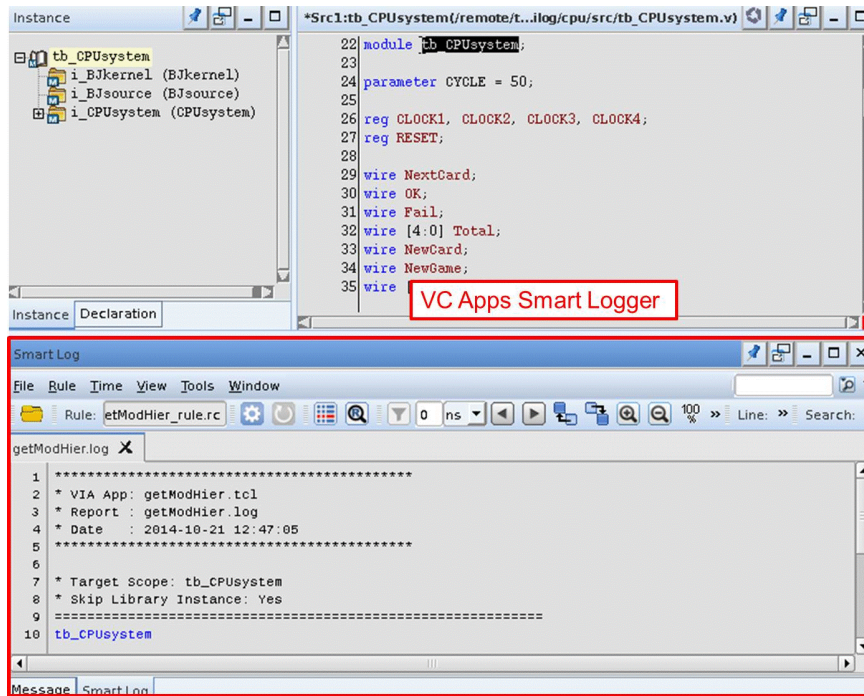
You can also drag and drop signals to a field in the *Input* form. Then, a selection menu (history) will display with each drag and drop of a new signal. You can select one of the signals as the value of the field.

Figure 7 History Selection Menu



After specifying the arguments for the App in the *Input* form, click the **OK** button to execute this App. If a log file is generated after executing the script, the *Smart Log* frame displaying the log results will be opened in the Verdi main window. The following figure shows an example of the log file in the Verdi platform.

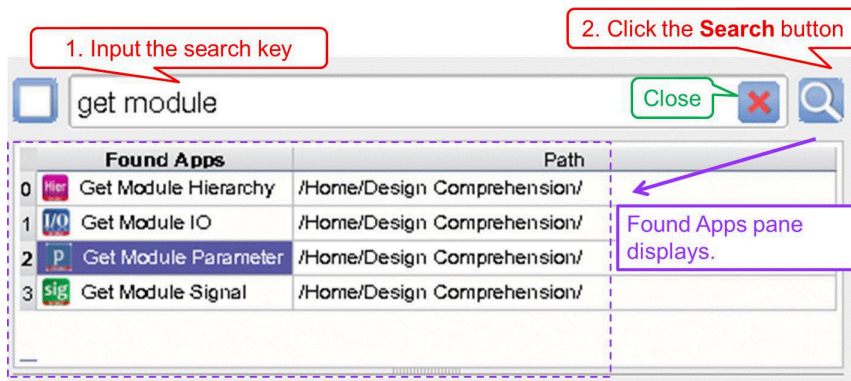
Figure 8 Log Results



Find Applications

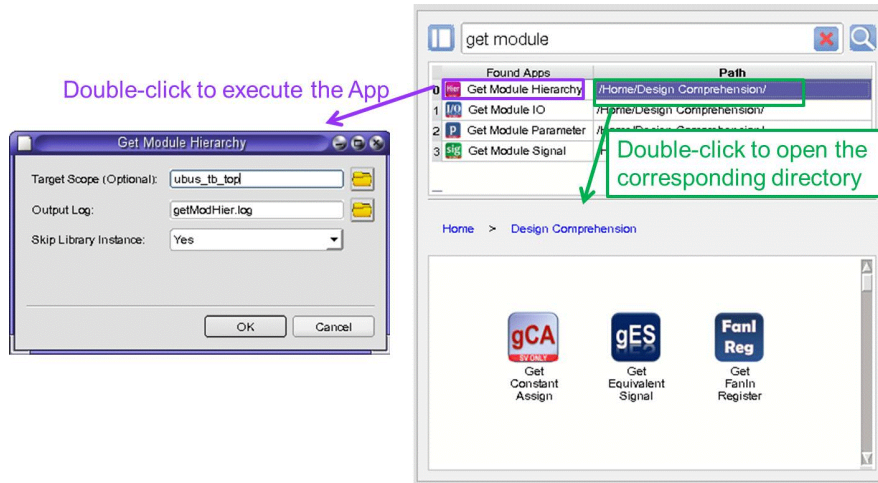
The **Find Tool** field provides searching features to locate specific VC Apps. The search text field supports wildcard and regular expression searching. Input the search string in the blank field and click the  button. Then, the searching result is listed in the *Found Apps* pane. The searching result includes all Apps whose names include the search string.

Figure 9 Locate VC Apps



You can directly double-click a matched App to execute it or double-click the path to open the corresponding directory path in the *Applications List View* pane.

Figure 10 Operations in Found Apps Pane



In the **Directory Path** field, each directory is active to let you change to the associated path with a click. For example, click the *"Home"* path. Then, the current directory changes to the *"Home"* directory from the *"Design Comprehension"* directory.

Configure Applications

Most VC Apps in the VC Apps Toolbox are open sourced, so the configuration of each application can be modified. In the *Applications List View* pane of the *VC Apps Toolbox* form, select a VC App icon and use the **Config** command in the right-click command menu to open the *Config App* form. The following figure shows an example of the *Config App* form from the **Get Module Hierarchy** App.

Figure 11 Config App Form

Locations for the Files

Arguments Table

Argument Prompt	Default Value	Browser Type	Display Log	Log Rule
1 Target Scope (Optio...	[srcGetScope]	Instance	☐	
2 Output Log	getModHier.log	File	☒	getModHier_...
3 Skip Library Instance	{Yes} [No]	None	☐	

In this form, you can change the display name, file location, and icon in the following fields:

- **Display Name:** Specify the display name to be shown in the *VC Apps Toolbox* form and as the naming for the *Input* form. In a directory, applications or folders with the same display names are not allowed.
- **Path:** Specify the reference path to the location of the files. The recommendation is to put all relative files into one directory.
- **File Name:** Specify the Tcl file for execution. All Tcl commands should be wrapped in "proc".
- **Help File:** Specify the HTML file as the manual.
- **Icon:** Specify the icons to be shown in the *VC Apps Toolbox* form. The supported types are *.bmp, *.gif, *.ico, *.jpeg, *.jpg, *.mng, *.pbm, *.pgm, *.png, *.ppm, *.svg, *.tif, *.tiff, *.xbm, and *.xpm. The recommendation is to use the square icon with pixel 50x50 and bigger.

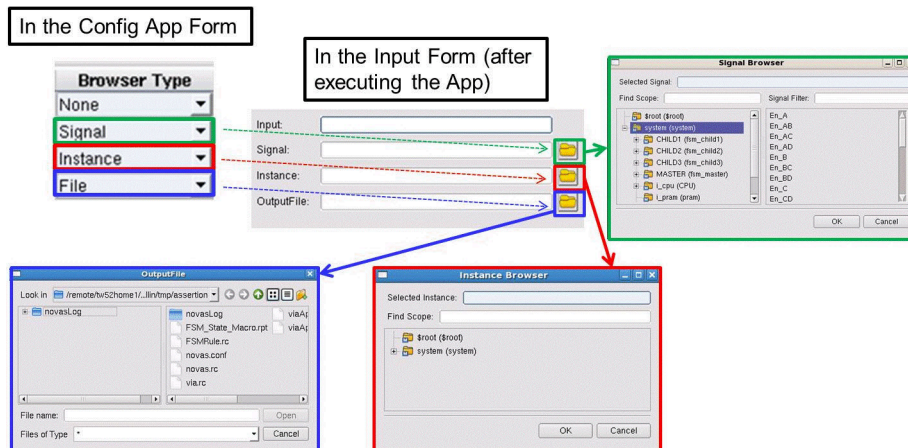
Click the **OK** button to apply the updated configuration.

If you are familiar with Tcl, you can even modify the Tcl script and then modify the argument settings for the main procedure in the **Arguments** table. If the **Arguments**

table is empty, the execution application will not have an *Input* form. The following fields compose the procedure:

- **Argument Prompt:** Specify the argument prompt in the *Input* form.
- **Default Value:** Specify the default value of the argument after displaying the *Input* form. Refer to the [Set Default Value for the Argument](#) section for details.
- **Browser Type:** Specify the type of browser to invoke after clicking the  button in the *Input* form. Specify each browser type in each **Browser Type** field. After executing the App, the argument will have a  button to select the content if the **Browser Type** field is not **None**. You can see the corresponding browser after clicking each button.

Figure 12 Browser Types



- **Display Log:** Specify the argument as the output file that will be opened in the *Smart Log* frame after the App is executed.
- **Log Rule:** Specify the log rule file that is used by the *Smart Log* frame.

Set Default Value for the Argument

In the **Arguments** table of the *Config App* form, you can specify the default value with various formats for an argument. You can directly input the value after selecting a **Default Value** field:

- **Variable default value:** Specify a Tcl command and wrap the command in the "[]" bracket characters. The command will be executed by the Tcl interpreter and the return value will be obtained after invoking the *Input* form.
- **Fixed default value:** Specify a fixed value without the "[]" bracket characters.
- **Combo box menu:** You can also specify multiple default values for an argument selection in the *Input* form. In a **Default Value** field, separate each value with a blank

character to identify each value as an item of the combo box menu. If the value contains a blank, wrap the value in the "{}" brace characters. For example, if *{Hello World} Test Demo* value is specified as the default value, then after executing the App, the default value will be shown in a combo box menu with the **Hello World, Test** and **Demo** values.

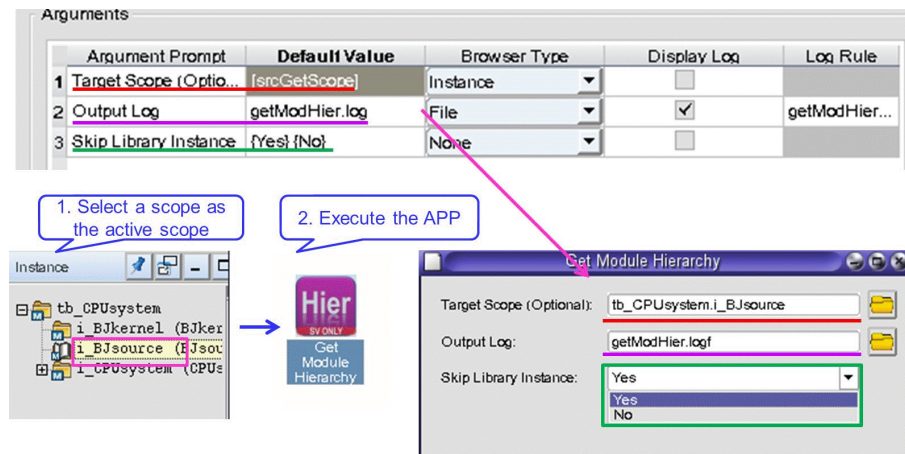
For example, if the arguments and default values are specified as follows:

- The *Target Scope* argument with default value `[srcGetScope]`
- The *Output Log* argument with default value `getModHier.log`
- The *Skip Library Instance* argument with default value `{Yes}{No}`

Then, as shown in the following figure, after selecting the *i_BJsource* scope in the *nTrace* main window and executing the **Get Module Hierarchy** application, the *Input* form of this application displays with the following corresponding default values:

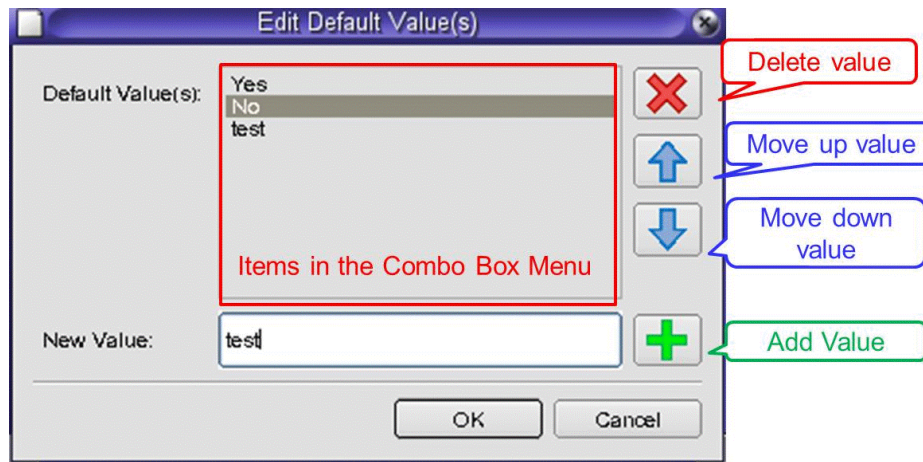
- The *Target Scope* argument with default value `tb_CPUsystem.i_BJsource` which is the
- The *Output Log* argument with default value `getModHier.log`.
- A menu for the selection of the *Skip Library Instance* argument with default values Yes and No.

Figure 13 Set Default Value(s)



You can also click the  button after selecting a default value to configure the combo box menu in the *Edit Default Value(s)* form. Each added value will be composed as a selection in the combo box menu.

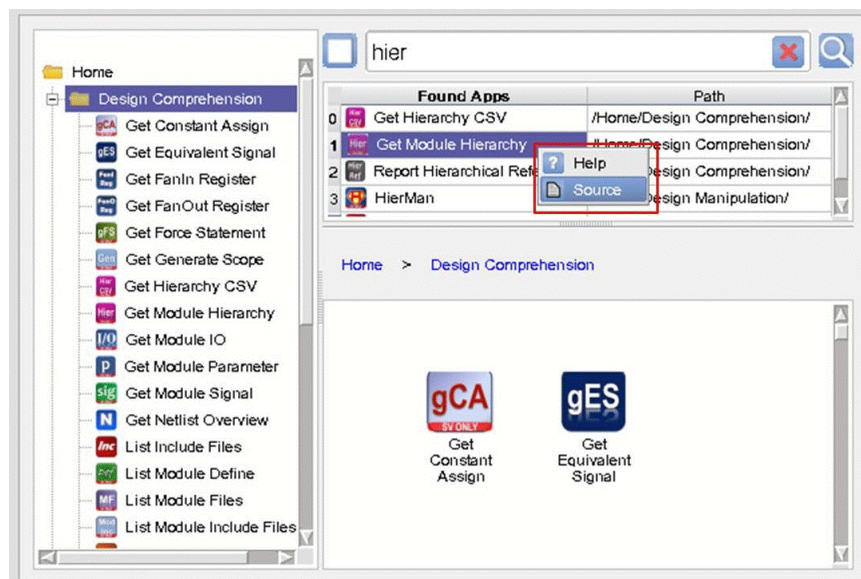
Figure 14 Edit Default Value(s)



Access Application Source Files

You can access the source file of a VC App using the **Source** command in the right-click command menu. This command can be invoked in the hierarchy browser, *Found Apps* and *Applications List View* panes of the *VC Apps Toolbox* form.

Figure 15 Invoke the Source Command



The source Tcl file will be display in the GVIM text editor by default. You can review the file or edit it on demand. The `VBOX_OPEN_SOURCE` environment variable is provided to

specify another editor for accessing the source file. For example, you can specify Gedit as the text editor:

```
> setenv VBOX_OPEN_SOURCE gedit
```

Add Applications and Folders

You can include your own Applications (written as Tcl scripts) and create an entry in the *VC Apps Toolbox* form to execute it. Change to your desired hierarchy in the *VC Apps Toolbox* form, invoke the right-click command menu in a blank space and select the **New** command to open a blank *Config App* form.

Figure 16 Blank Config App Form

The following fields highlighted in red in this form must be set for your own applications:

- **Display Name:** The application name you want to see in the *VC Apps Toolbox* form.
- **File Name:** The Tcl script file name for your application.
- **Name:** The main procedure name for your Tcl script. After the script file is specified in the **File Name** field, the procedures in this file will be automatically listed in the selection menu. When double-clicking an App in the *VC Apps Toolbox* form, the Verdi platform will source the Tcl script file and then call the specified procedure name to execute your scripts.

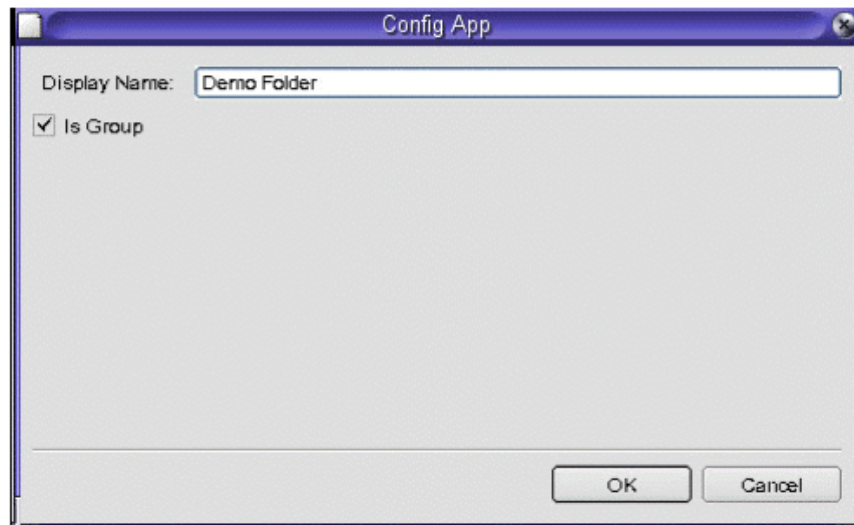
The following fields are optional for your own applications:

- **Path:** You can specify an absolute path or a relative path for your Tcl script. The VC Apps Toolbox will search the paths specified in the **File Name**, **Help File**, and **Icon** fields based on the following priority:
 - a. Absolute path
 - b. Relative path from current working directory
 - c. The Verdi installation path
- **Help File:** You can prepare an HTML file for the help file and then specify the file name and path in this field.
- **Icon:** You can click the arrow in the **Icon** field to specify the default icon, or click the **Browse** button to select your own icon.

If your script contains arguments, click the  button to automatically add all arguments to the form. Then, you can configure the arguments in the **Arguments** table (refer to the [Configure Applications](#) section for details). If the argument is an output file, you can turn on the **Display Log** option to automatically load the file into the *Smart Log* form after executing the scripts.

You can also create a new folder by enabling the **Is Group** option in the *Config App* form.

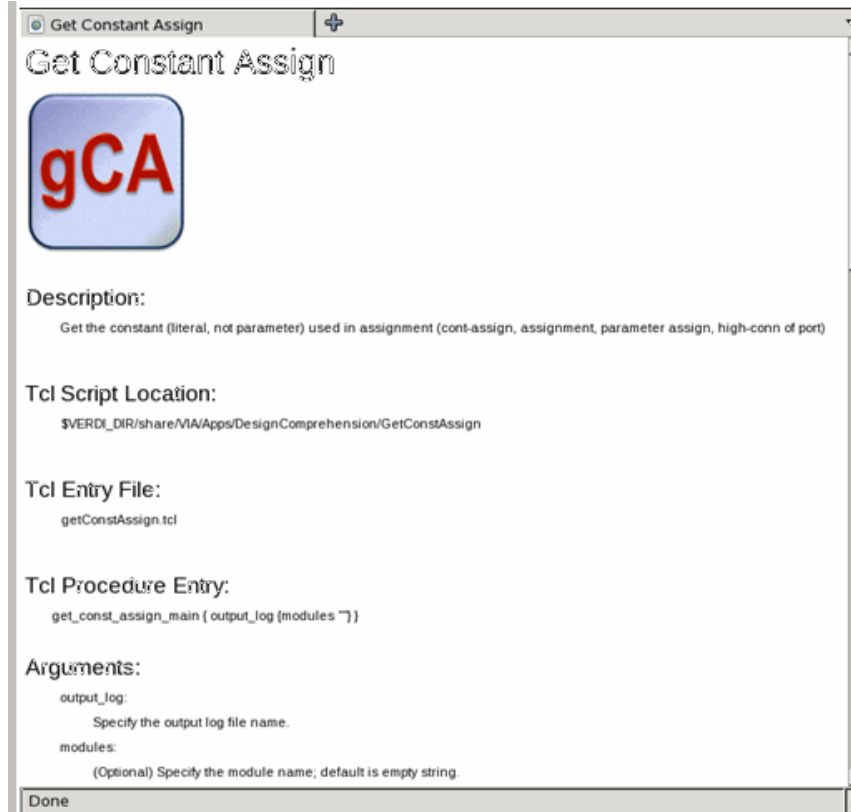
Figure 17 Add New Folder



Get Help for Applications

A help document to explain the detailed usage is provided for each VC App. Select an App icon in the *VC Apps Toolbox* form and use the **Help** command in the right-click command menu to open the related help document. The following figure shows an example of the help document for the **Get Constant Assign** App.

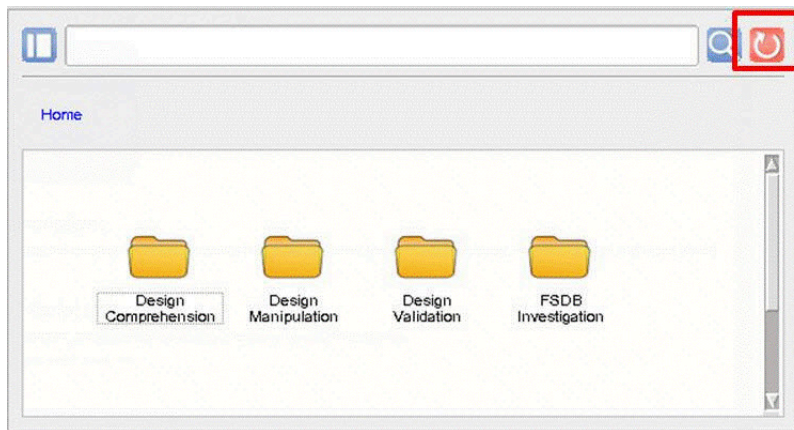
Figure 18 Help Document



Update Applications

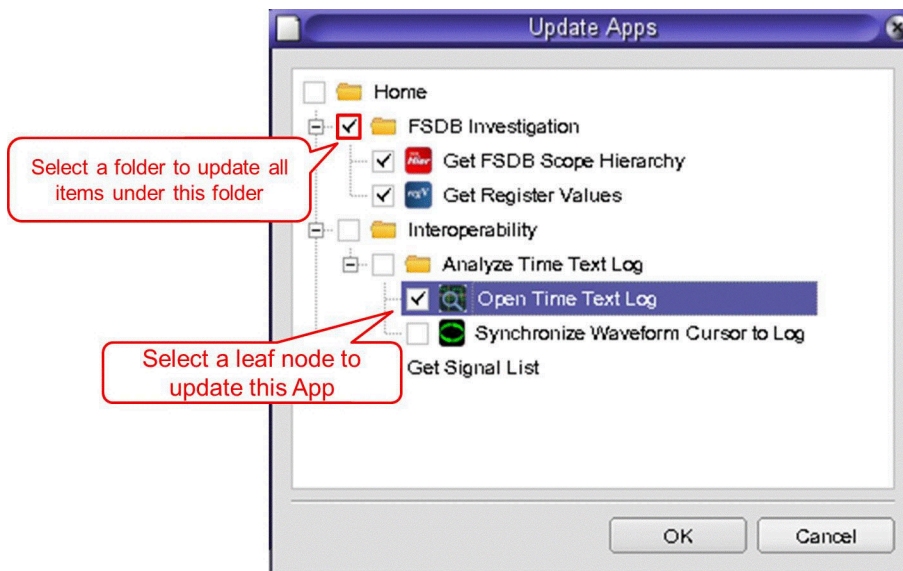
The *VC Apps Toolbox* form provides a mechanism to check the VC Apps and determine whether any are missing or need to be updated. If there are installed VC Apps that do not exist in the working directory, the **Update VC Apps** button is shown.

Figure 19 Update VC Apps in the VC Apps Toolbox Form



After clicking the  button, the *Update Apps* form displays with the list of un-updated VC Apps and their directory path. You can select a folder to update all un-updated items under this folder. You can also select a leaf node to update a VC App only. Then, click the **OK** button to update the VC Apps in the working directory.

Figure 20 Update VC Apps



Practice

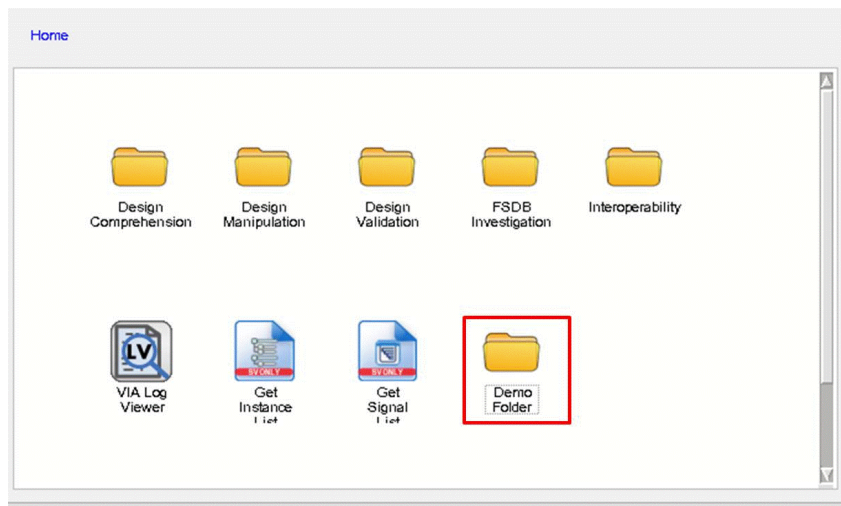
Complete these steps to create your own application and add an entry to the *VC Apps Toolbox* form.

1. In your working directory, create a file called *demo_app.tcl* and add the content as follows:

```
proc my_app_entry {Headline Instance1 Instance2 outputFile} {  
  set fileHandle [open $outputFile "w"]  
  puts $fileHandle $Headline  
  puts $fileHandle $Instance1  
  puts $fileHandle $Instance2  
  close $fileHandle  
}
```

2. Create a folder under the *Home* directory in the *VC Apps Toolbox* form.
 - a. In the *VC Apps Toolbox* form, right-click in a blank space and then select the **New** command to open a blank *Config App* form.
 - b. Input the *Demo Folder* string in the **Display Name** field.
 - c. Enable the **Is Group** option.
 - d. Click the **OK** button.

Figure 21 Create A Folder under the Home Directory

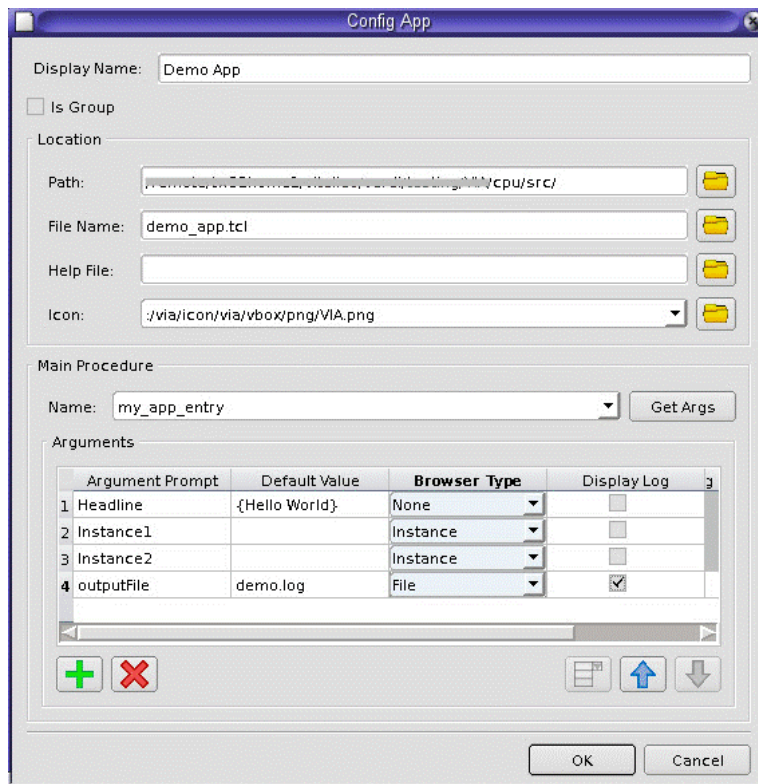


3. Create an entry in the *VC Apps Toolbox* form for your Tcl script.
 - a. In the *VC Apps Toolbox* form, double-click the newly created **Demo Folder** folder. Right-click in a blank space and then select the **New** command to open a blank *Config App* form.
 - b. Input the *Demo App* string in the **Display Name** field.

- c. Click the  button to specify the Tcl file name in the **File Name** field. After the Tcl file is specified in the **File Name** field, the file path will be automatically entered in the **Path** field.
- d. Select the *my_app_entry* main Tcl procedure name in the selection menu for the **Name** field.
- e. Click the  button to automatically insert the arguments.
- f. Input the *{Hello World}* and *demo.log* strings as the default values for the *Headline* and *outputFile* arguments.
- g. Select the **Instance** option as the browser type for the *Instance1* and *Instance2* arguments.
- h. Turn on the **Display Log** option for the *outputFile* argument.

The form will be similar to the following figure:

Figure 22 Create An Entry in the VC Apps Toolbox form for Your Tcl Script



Config App

Display Name: Demo App

☐ Is Group

Location

Path: /usr/bin/.../cpu/src/ 

File Name: demo_app.tcl 

Help File: 

Icon: ./via/icon/via/vbox/png/VIA.png 

Main Procedure

Name: my_app_entry 

Arguments

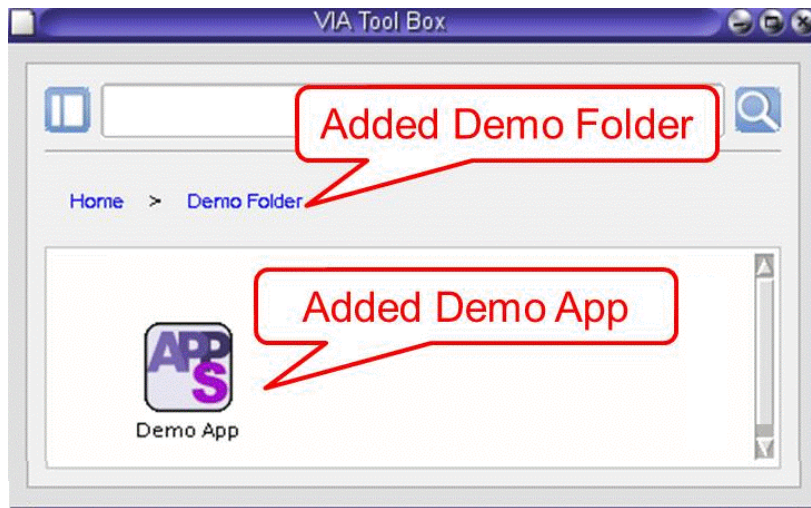
Argument Prompt	Default Value	Browser Type	Display Log
1 Headline	{Hello World}	None	☐
2 Instance1		Instance	☐
3 Instance2		Instance	☐
4 outputFile	demo.log	File	☒

OK Cancel

4. Click the **OK** button, and a new icon will appear in the *VC Apps Toolbox* form.

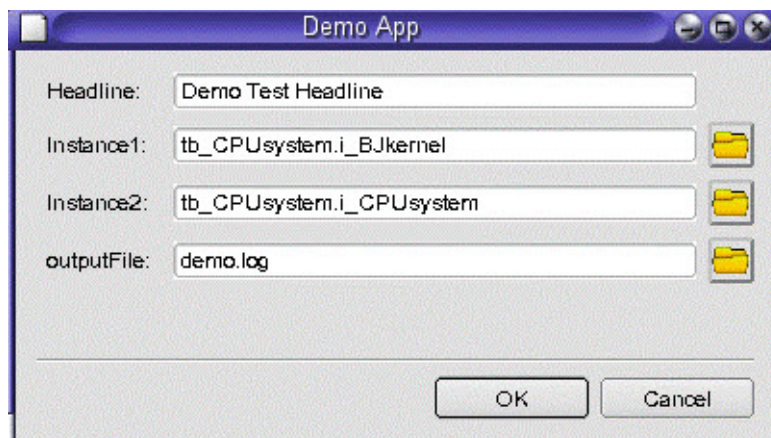
Figure 23 New Icon in the VC Apps Toolbox Form



5. Execute the newly created application.
 - a. Double-click the **Demo App** icon for the newly created application. The *Input* form for the **Demo App** will be opened for you to input arguments.
 - b. Specify *Demo Test Headline* in the **Headline** field. Click the  buttons to specify the instances in the **Instance1** and **Instance2** fields and leave the default value for *Output Log* argument

The form will be similar to the following figure:

Figure 24 Execute the Newly Created Application



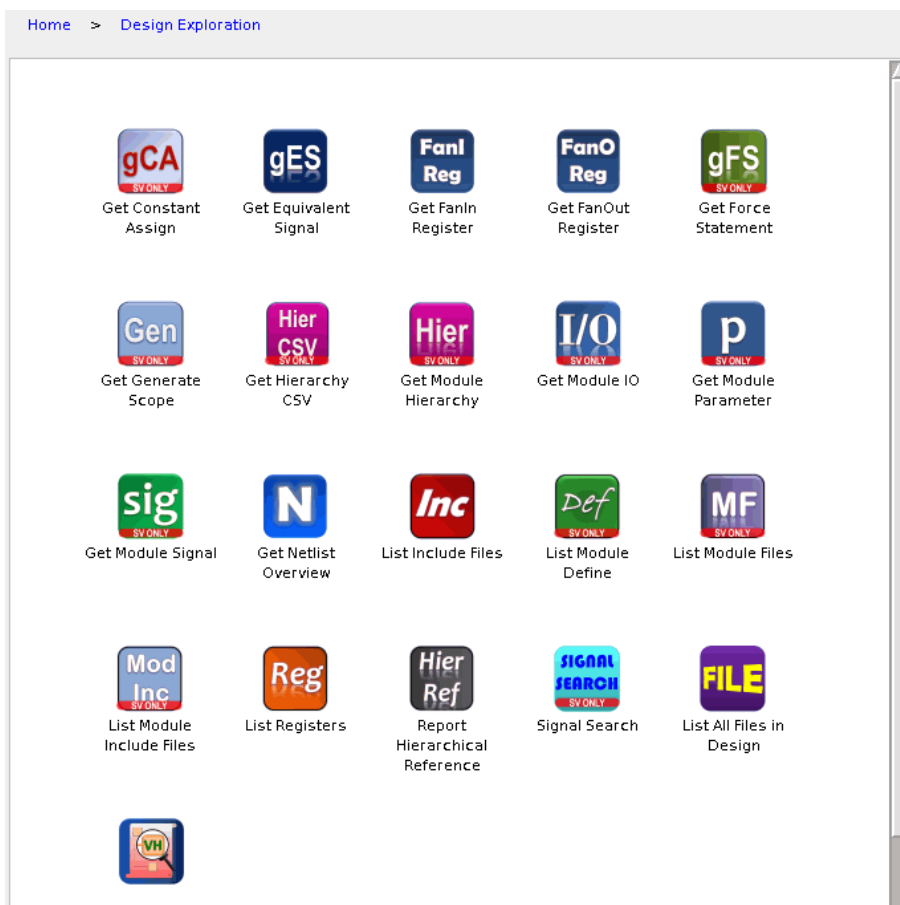
- c. Click the **OK** button to execute the application with the specified arguments.

After executing the application, the output log will be loaded into the Verdi platform and opened in the *Smart Log* frame automatically.

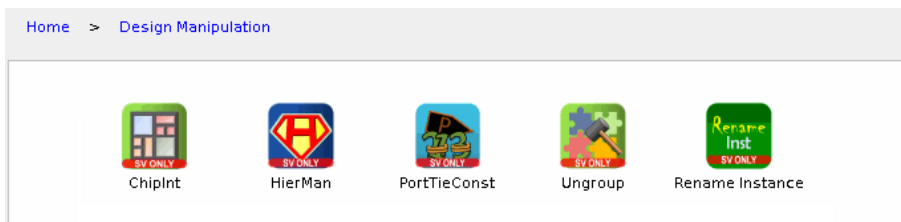
Appendix A: Built-in Apps in VC Apps Toolbox

The following figures shows the built-in applications provided in the *VC Apps Toolbox* form. They are categorized into folders by default. You can refer to the help document for detailed usage for each application using the **Help** command in the right-click command menu.

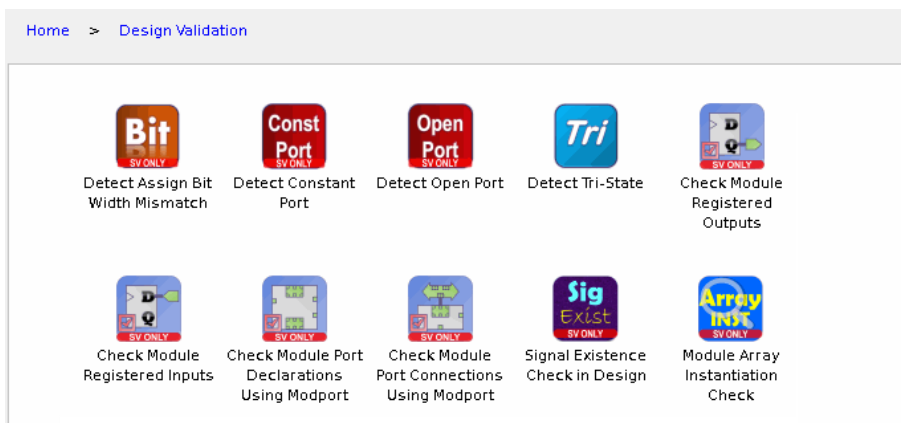
Design Exploration folder:



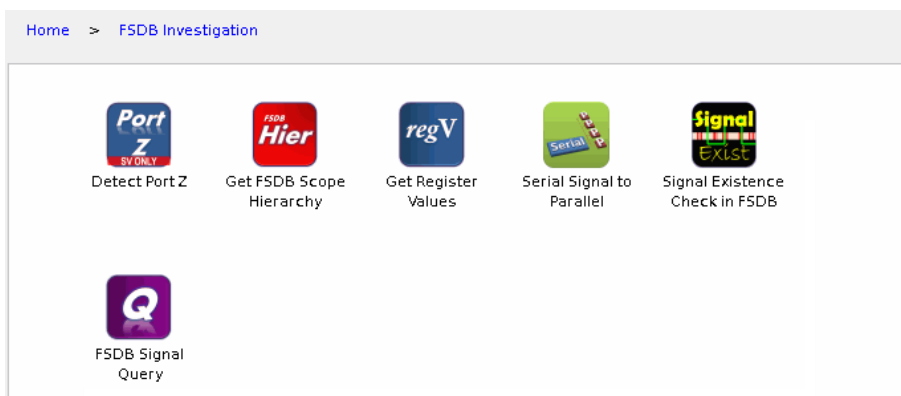
Design Manipulation folder:



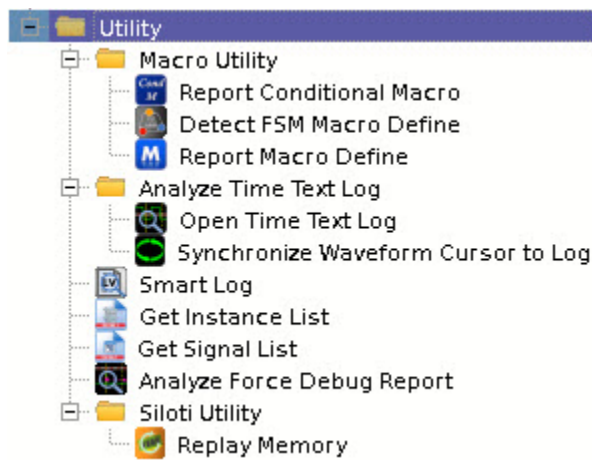
Design Validation folder:



FSDB Investigation folder:



Utility folder:



Home directory:



3

Design Exploration

This section describes about the following topics:

- [Get Constant Assign](#)
- [Get Equivalent Signal](#)
- [Get FanIn Register](#)
- [Get FanOut Register](#)
- [Get Force Statement](#)
- [Get Generate Scope](#)
- [Get Hierarchy CSV](#)
- [Get Module Hierarchy](#)
- [Get Module IO](#)
- [Get Module Parameter](#)
- [Get Module Signal](#)
- [Get Netlist Overview](#)
- [Get PG Pin Connection](#)
- [Get Port Connection](#)
- [List Include Files](#)
- [List Module Define](#)
- [List Module Files](#)
- [List Module Include Files](#)
- [List Registers](#)
- [List Scope Parameter](#)
- [Report Hierarchical Reference](#)

- [Signal Search](#)
- [List All Files in Design](#)
- [Report VHDL Component](#)
- [Find Instance with Module Def Name](#)
- [Report Exception Paths](#)

Get Constant Assign



Description

Get the constant (literal, not parameter) used in assignment (cont-assign, assignment, parameter assign, high-conn of port).

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/GetConstAssign`

Tcl Entry File

`getConstAssign.tcl`

Tcl Procedure Entry

`get_const_assign_main { output_log {modules ""} }`

Arguments

output_log:

Specify the output log file name.

modules:

(Optional) Specify the module name; default is empty string.

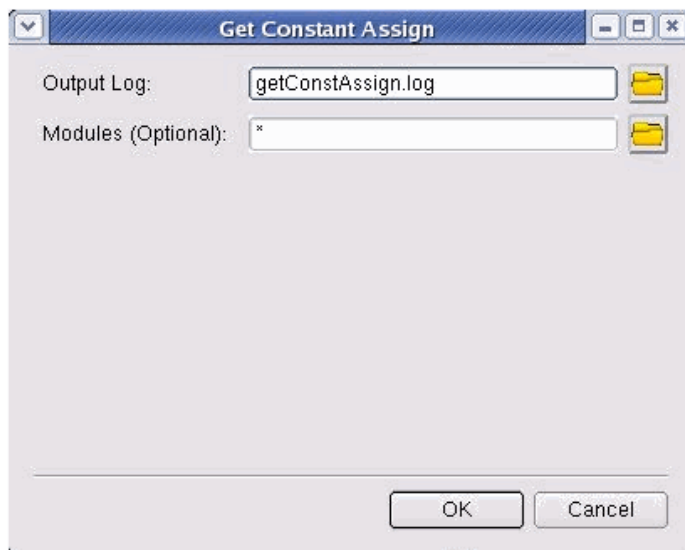
The name can be specified by exact one or wildcard.

The name can be module hierarchy name or module definition name.

If the module name is empty string, then report all modules.

Use space to specify more than one module; e.g.

"system.i_cpu ALU PC* system.i_cpu.i_*.reg*"



Batch Mode Usage

```
Usage: getConstAssign_batch.pl <verdi_import_design> \  
      [-modules "<target_module_names>"] \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./getConstAssign_batch.pl -f run.f -o getConstAssign.log  
./getConstAssign_batch.pl -f run.f -modules "TOP" -o  
getConstAssign.log
```

```
./getConstAssign_batch.pl -f run.f -modules "system.TOP.*  
ALU*" -o getConstAssign.log
```

Limitations

Currently this App won't report the constant assignments in VHDL components.

Example

```
/ ----- example.v ----- //  
  
module top (in1);  
  input in1;  
  wire in1;  
  
  parameter P1 = 0;  
  wire A;  
  wire B = 0;  
  reg C;  
  
  assign A = 0;  
  
  always @(in1) begin  
    if(in1)  
      C = 1;  
    else  
      C = A;  
  end  
  
  sub M1(1'b0);  
  sub M2(P1);  
  sub M3(C);  
endmodule  
  
module sub(A);  
  input A;  
  wire A;  
  wire B;  
  
  assign B = A;  
endmodule  
  
// ----- getConstAssign.log ----- //  
  
*****  
* VIA App: getConstAssign.tcl  
* Report : getConstAssign.log
```

```
* Date      : 2013-03-18 15:24:43
*****
```

```
=====

Target Modules: *
```

```
=====

Scope: top.M1:
```

```
-----
Port High Connection: .A( 'b0 )
                      ( example.v : 19 )
```

```
Scope: top:
```

```
-----
Continuous Assign: assign B = 0
                  ( example.v : 7 )
Continuous Assign: assign A = 0
                  ( example.v : 10 )
Assignment: C = 1;
           ( example.v : 14 )
```

Get Equivalent Signal



Description

It gets the equivalent signals from the specified signal(s).

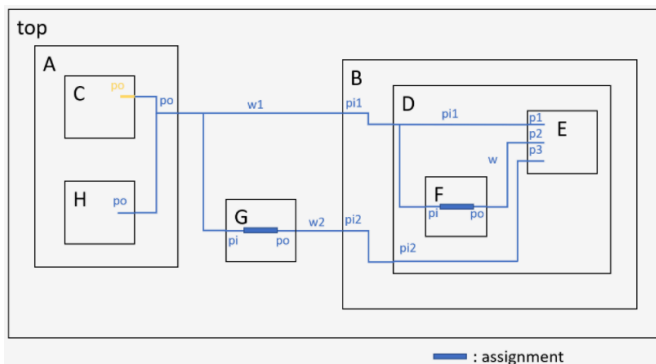
This App traces connectivities of the specified signals. The tracing passes through the module ports and stops at the primitive cells. In the output file, for every input signal, it reports the equivalent signals. The output file can be in the following formats:

- **Default:** A general log file with full hierarchical name.
- **Elfile:** An exclusion file, which can be loaded by VerdiCoverage or URG to exclude the signals.
- **Hier:** A hierarchical configuration file, which can be used by VCS (`-cm_hier <hier_config_file>`) to exclude the signals during the compile phase.

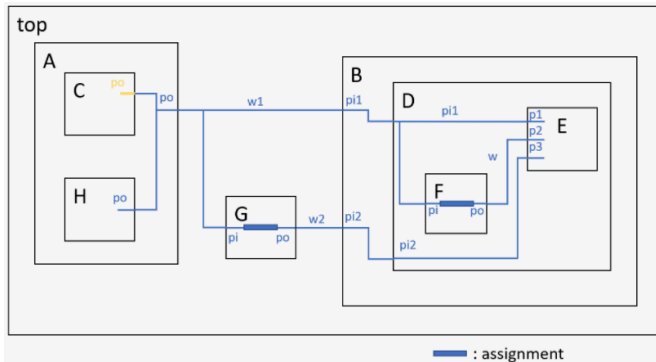
Note:

When the format is default, the tracing does not pass through the assignment cell; it treats assignment cell as the primitive cell.

For example, for a given signal **top.A.C.po**, if the format is **elfile** or **hier**, this App passes through the assignment, and the equivalent signals are top.A.po, top.A.H.po, top.w1, top.G.pi, top.G.po, top.w2, top.B.pi1, top.B.D.pi1, top.B.D.E.p1, top.B.D.F.pi, top.B.D.F.po, top.B.D.w, top.B.D.E.p2, top.B.pi2, top.B.D.pi2, and top.B.D.E.p3.



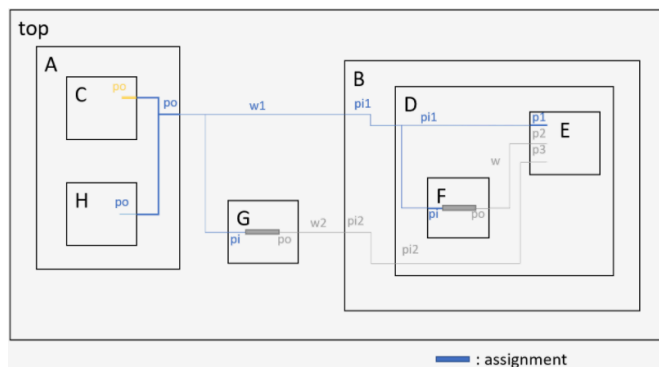
If the format is **default**, this App does not pass through the assignment. Thus, the equivalent signals are top.A.po, top.A.H.po, top.w1, top.G.pi, top.B.pi, top.B.D.pi1, top.B.D.F.pi, and top.B.D.E.p1.



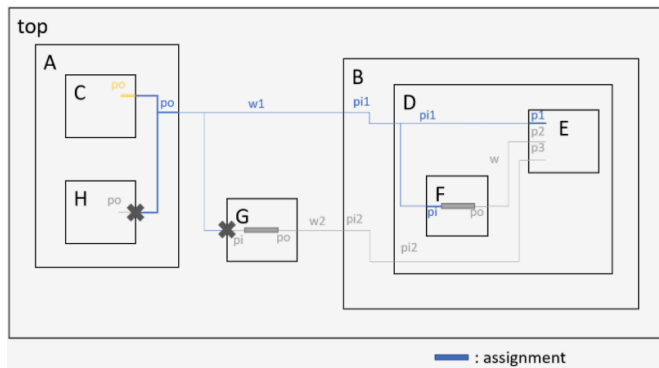
The other available options to filter the result or to control the tracing scopes are as follows:

- In the previous example, **top.w1** is an internal wire and is not shown when this option is *on*.

- In the previous example, if the target instances are top.A and top.B.D.*, the result is signals under top.A, top.B.D.E, and top.B.D.F, which are top.A.po, top.D.E.p1, and top.D.F.pi.



- In the previous example, the target instances are top.A and top.B.D.*, and the **Limit Trace Scope** is set to *on*. The limited passing instances are top, top.A, top.B, top.B.D, top.B.D.E, and top.B.D.F. This reduces the tracing effort in top.A.H and top.G, as the result remains the same, which is top.A.po, top.D.E.p1, and top.D.F.pi.

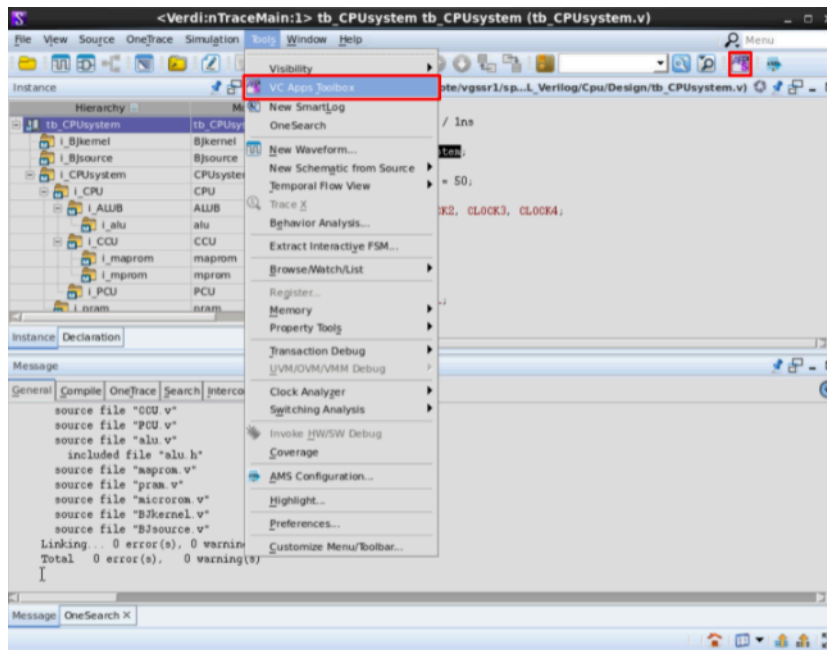


Limitation

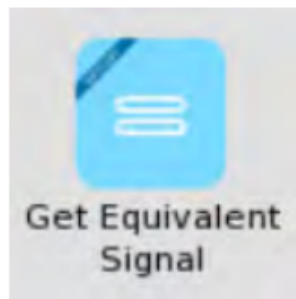
None

How to Open the App in VC Apps Toolbox

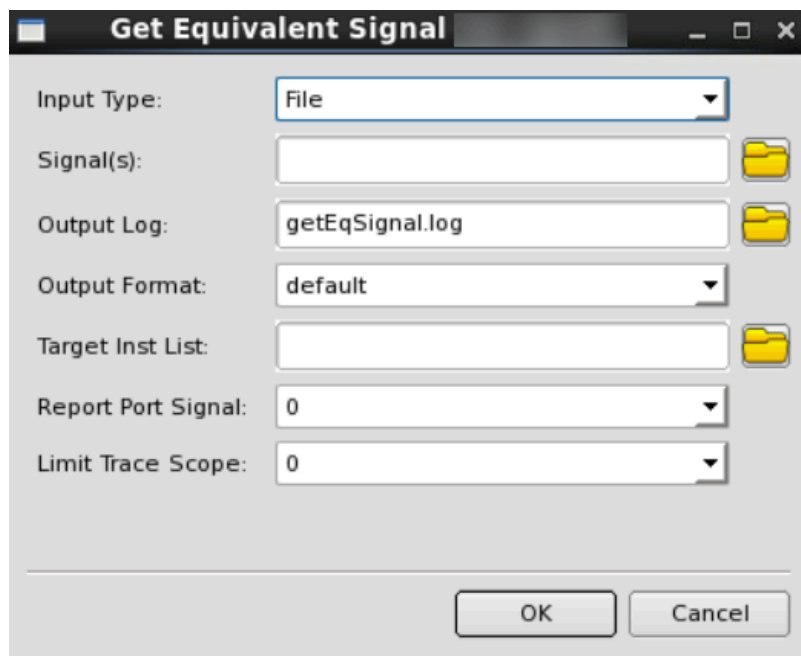
1. Open Verdi with design, select **Tools -> VC Apps Toolbox**;
or click the **VC Apps Toolbox** icon directly.



2. Go to **Design Exploration**, click the **Get Equivalent Signal** icon.

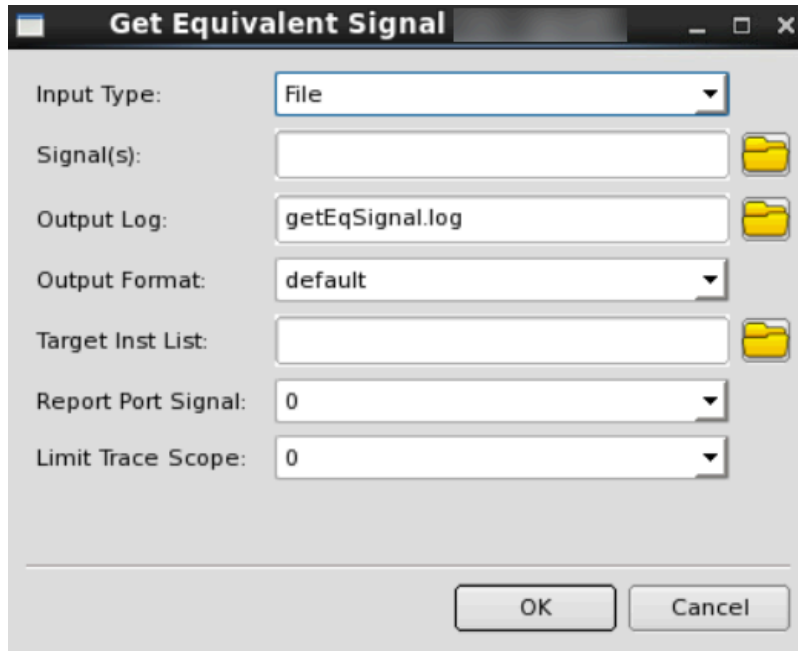


The App opens as shown in the following figure:



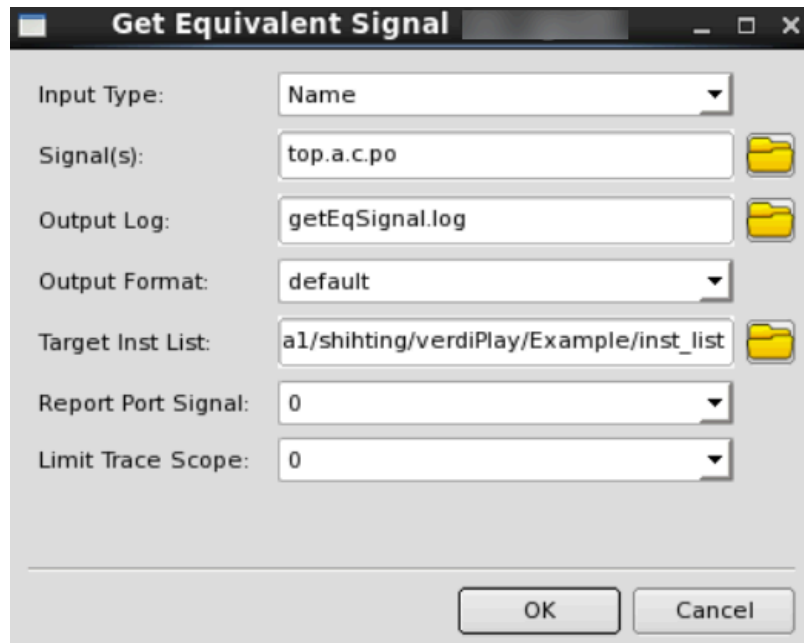
App Usage in Verdi GUI Mode

1. Open App in VC Apps ToolBox.



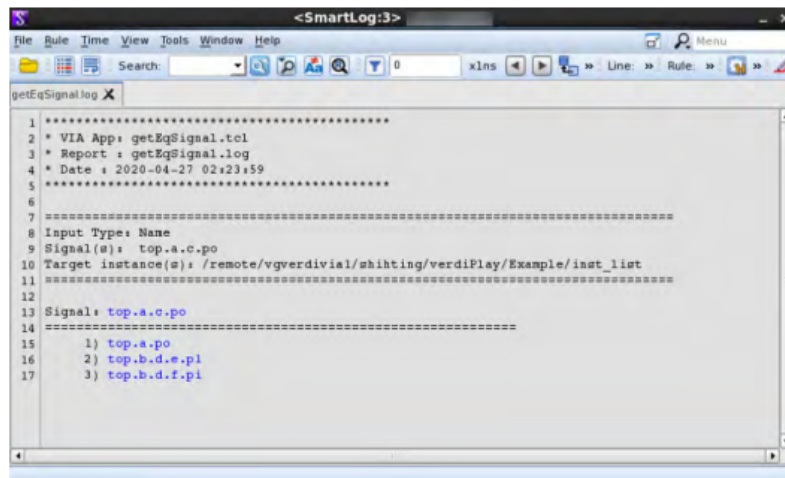
2. Fill in the following fields:
 - **Input Type:** Specify the input type, *Name* or *File*.
 - **Signal(s):** Specify the signal full hierarchical name or file name according to *Input Type*.
 - **Output Log:** Specify the output log file name.
 - **Output Format:** Specify the output file format. The options are *default*, *elfile*, or *hier*.
 - **Target Inst List:** Specify the file name, which contains the instance hierarchical name per line, that filters the result to signals under the specified instances.
 - **Report Port Signal:** Specify whether to filter the result to signal representing a port.

- **Limit Trace Scope:** Specify whether to perform limited tracing only under scopes of instances in *Target Inst List*.



3. Click **OK**.

SmartLog displays the output log with signal full name with its equivalent signals. It ignores the duplicated signals.



App Usage in Verdi Batch Mode

Script location

`$VERDI_HOME/share/VIA/Apps/Bin/getEqSignal.pl`

Usage

```
getEqSignal.pl <design_import>  
-signal "<signal_hier_name>" | -file <signal_file>  
[-o <output_log>]  
[-format default|elfile|hier]  
[-target_inst_list <target_inst_file_name>]  
[-report_port_signal]  
[-limit_trace_scope]
```

Options

- `-signal`: Specify the target signal hierarchical name.
- `-file`: Specify the file name, which describes multiple signals, the file format is one signal hierarchical name per line.

For example,

```
signal_file  
=====  
top.a.c.po  
top.b.pil  
=====
```

Note:

The `-file` and `-signal` options are mutually exclusive; that is, `-file` works only when `-signal` is not given.

- `-o`: Specify the output log name. The default value is `getEqSignal.log`.
- `-format` (Optional): Specify the output file format. The default format is **default**.

default: The general readable log file

elfile: The exclusion file, which can be loaded in VerdiCoverage/URG to exclude connected signals

hier: The hierarchical file, which can be used when VCS compile by `-cm_hier <hier_conf_file>`

- `-target_inst_list` (Optional): Specify the file name, which contains the instance hierarchical name per line, that filters the result to signals under the specified instances.

For example,

```
inst_list
=====
top.a
top.b.d.*
=====
```

Note:

Wildcard for the last hierarchy, like top.B.D.* is supported. It means all instances under top.B.D are supported.

- `-report_port_signal` (Optional): If set, it filters the result to signal representing a port.
- `-limit_trace_scope` (Optional): If set, it does limited tracing only under scopes of instances in `target_inst_list`.

Example

```
/> getEqSignal.pl -sv demo.v -signal "top.a.c.po" -format
default -target_inst_list "inst_list"
```

Get FanIn Register

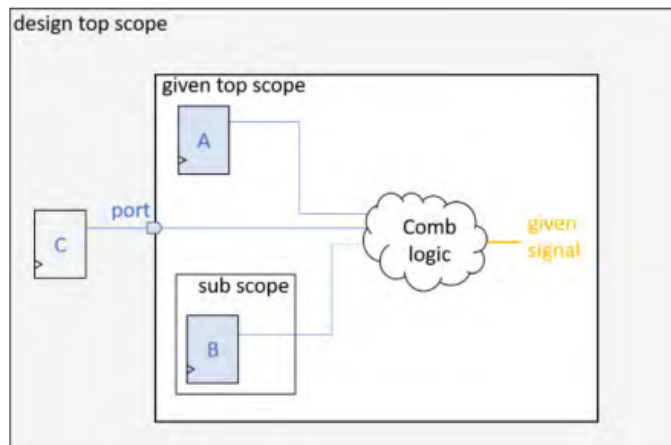


Description

Get the fanIn registers information of a specified signal(s).

This App traces the fan-in of the specified signals under a specified top scope. The tracing stops at the registers and primary ports of the top scope. In the output file, for every input signal, it reports information of the traced registers and ports. And for each register instance, it also includes the information of its ports along with the role.

For example, by given signal and given top scope, the tracing does not cross through the given top scope boundary. Thus, only the registers A and B (the blue ones) are listed in the output log. If the user specifies the option to report primary ports, then the output port of the given top scope (the blue port) is also listed.



The signal can be provided either by single full hierarchical name, or full hierarchical names listed in a file.

If the top scope is not specified, the App reports all registers traced in the design, and by default, it does not report the primary ports.

Limitation

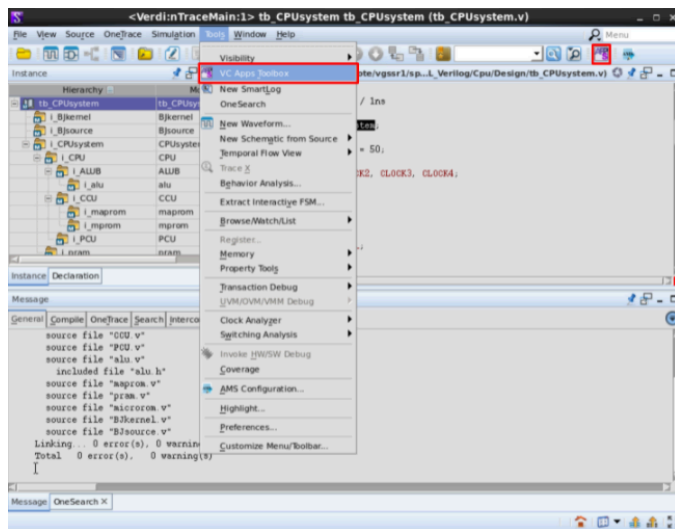
This app does not support gate-level design that nSchema cannot recognize flip-flop or other register type cells.

How to Open the App in VC Apps Toolbox

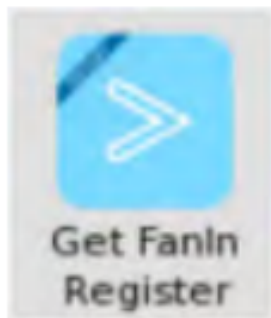
1. Open Verdi with design, select *Tools -> VC Apps Toolbox*;
or click the *VC Apps Toolbox* icon directly.

Chapter 3: Design Exploration

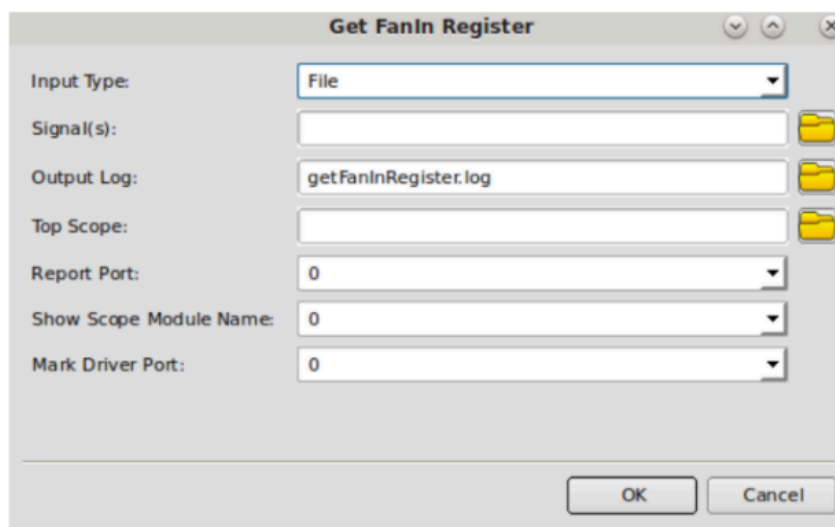
Get FanIn Register



2. Go to *Design Exploration*, click the *Get FanIn Register* icon.

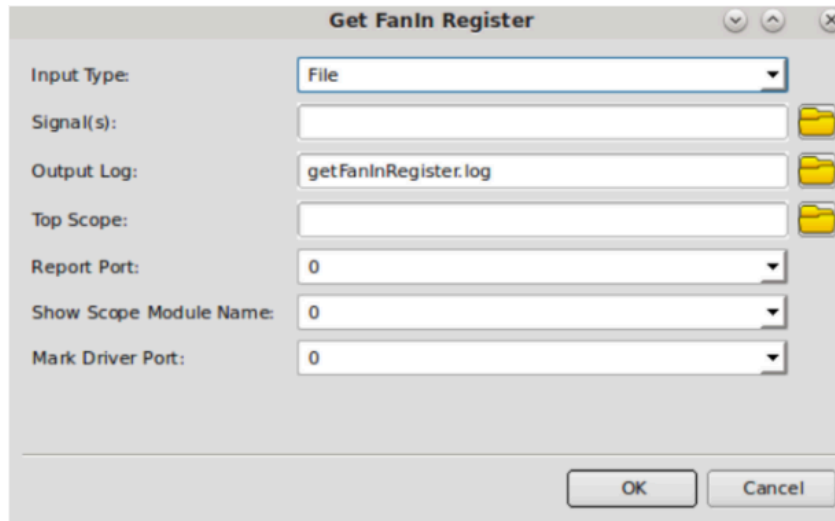


The App opens as shown in the following figure:



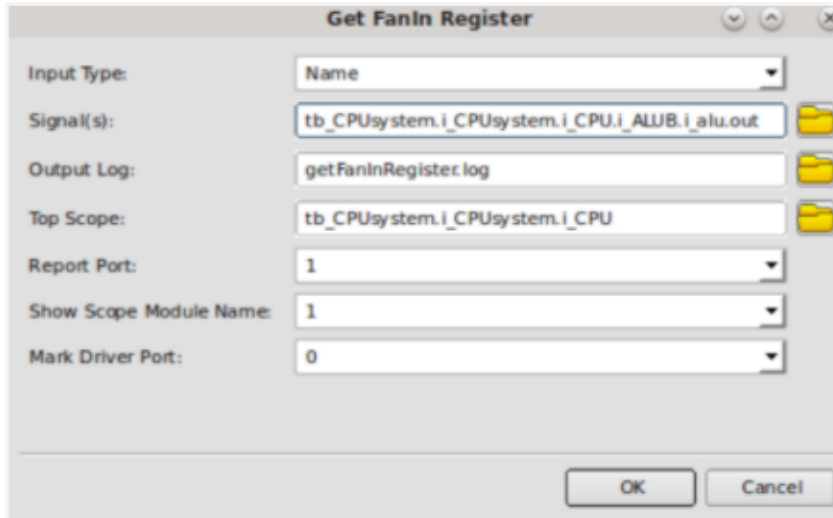
App Usage in Verdi GUI Mode

1. Open App in VC Apps ToolBox.



2. Fill in the following fields:
 - **Input Type:** Specify the input type, **Name** or **File**.
 - **Signal(s):** Specify the signal full hierarchical name or file name according to **Input Type**.
 - **Output Log:** Specify the output log file name.
 - **Top Scope:** Specify the top scope full hierarchical name, which cannot be traversed.
 - **Report Port:** Specify whether to list the primary ports of the top scope traced in the result.
 - **Show Scope Module Name:** Specify whether to show the scope module name of the register in the result.

- **Mark Driver Port:** Specify whether to mark the driver port in the result.



3. Click **OK**.

SmartLog displays the output log with the information of fanIn registers with its ports, and the port information of the top scope.

```
getFanInRegister.log
1 *****
2 * VIA App: getFanInRegister.tcl
3 * Report : getFanInRegister.log
4 * Date : 2021-09-28 17:54:10
5 *****
6
7 *****
8 Input Type: Name
9 Signal(s): tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu.out
10 Top Scope: tb_CPUsystem.i_CPUsystem.i_CPU
11 Report with primary port
12 Show scope module name
13 *****
14
15 Signal: tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu.out
16 *****
17 1) Flip-Flop: tb_CPUsystem.i_CPUsystem.i_CPU.CPU.Always16Always7:217:223:Reg (CPU.v : 217 : 217) (CPU.v : 220 : 220) (CPU.v : 222 : 222)
18   Module Name : CPU
19   1) Async-Reset Port : RESET (Size: 1)
20   2) Clock Port : CLOCK3 (Size: 1)
21   3) Data Port : GEMO_IDM[7:0] (Size: 8)
22   4) Register-Output Port : IDM[7:0] (Size: 8)
23
24 2) Flip-Flop: tb_CPUsystem.i_CPUsystem.i_CPU.CPU.Always44Always1:142:168:Reg (CPU.v : 142 : 142) (CPU.v : 145 : 145) (CPU.v : 147 : 147)
25   Module Name : CPU
26   1) Async-Reset Port : RESET (Size: 1)
27   2) Clock Port : CLOCK3 (Size: 1)
28   3) Data Port : GEMO_PC[7:0] (Size: 8)
29   4) Register-Output Port : PC[7:0] (Size: 8)
30
31 3) Flip-Flop: tb_CPUsystem.i_CPUsystem.i_CPU.CPU.Always58Always2:171:177:Reg (CPU.v : 171 : 171) (CPU.v : 174 : 174) (CPU.v : 176 : 176)
32   Module Name : CPU
33   1) Async-Reset Port : RESET (Size: 1)
34   2) Clock Port : CLOCK3 (Size: 1)
35   3) Data Port : GEMO_ACC[7:0] (Size: 8)
36   4) Register-Output Port : ACC[7:0] (Size: 8)
37
38 4) Flip-Flop: tb_CPUsystem.i_CPUsystem.i_CPU.CPU.Always68Always3:180:186:Reg (CPU.v : 180 : 180) (CPU.v : 183 : 183) (CPU.v : 185 : 185)
39   Module Name : CPU
40   1) Async-Reset Port : RESET (Size: 1)
```

App Usage in Verdi Batch Mode

Script location

```
$VERDI_HOME/share/VIA/Apps/Bin/getFanInRegister.pl
```

Usage

```
getFanInRegister.pl <design_import>
-signal "<signal_hier_name>" | -file <signal_file>
```

```
[-top <top_scope>]  
[-report_port]  
[-scope_module]  
[-drive_port]  
[-o <output_log>]
```

Options

- **-signal:** Specify the target signal hierarchical name.
- **-file:** Specify the file name, which describes multiple signals, the file format is one signal hierarchical name per line.

For example,

```
signal_file  
=====
```

```
tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu.out  
tb_CPUsystem.i_CPUsystem.i_CPU.i_PCU.data  
=====
```

Note:

The `-file` and `-signal` options are mutually exclusive; that is, `-file` works only when `-signal` is not given.

- `-top` (optional): Specify the top scope full hierarchical name, which cannot be traversed.
- `-report_port` (optional): Specify whether to list the primary ports of the top scope traced in the result.
- `-scope_module` (optional): Specify whether to show the module name of the scope instance of the register.
- `-drive_port` (optional): Specify whether to show the driver port of the signal fan-in.
- `-o`: Specify the output log name. The default value is *getFanInRegister.log*.

Example

```
/> getFanInRegister.pl -f run.f -path Design -signal  
"tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu.out" -top  
"tb_CPUsystem.i_CPUsystem.i_CPU" -report_port  
/> getFanInRegister.pl -f run.f -path Design -file  
signal_file -o getFanInRegister.log
```

Get FanOut Register

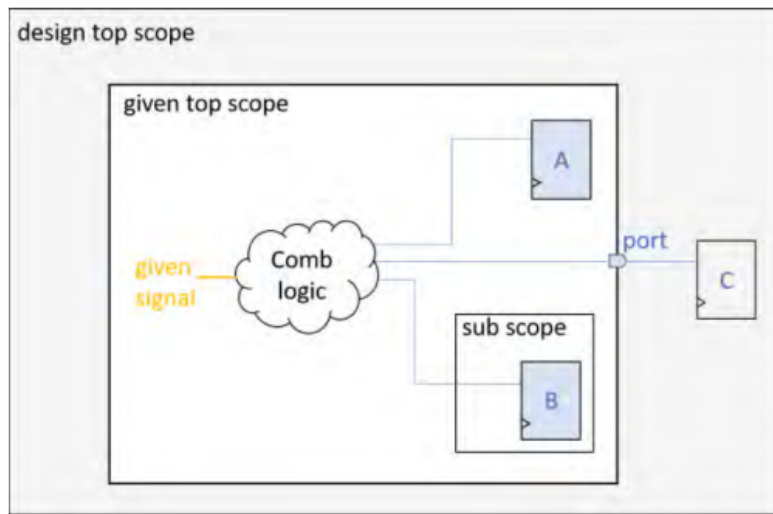


Description

Get the fanOut registers information of the specified signal(s).

This App traces the fan-out of the specified signals under a specified top scope. The tracing stops at registers and primary ports of the top scope. In the output file, for every input signal, it reports information of the traced registers and ports. And for each register instance, it includes the information of its ports along with the role.

For example, for the given signal and given top scope, the tracing does not cross through the given top scope boundary. Thus, only the registers A and B (the blue ones) are listed in the output log. If you specify the option to report the primary ports, then the output port of the given top scope (the blue port) is also listed.



The signal can be provided either by single full hierarchical name or full hierarchical names listed in a file.

If the top scope is not specified, the App reports all registers traced in the design, and by default, it does not report the primary ports.

Limitation

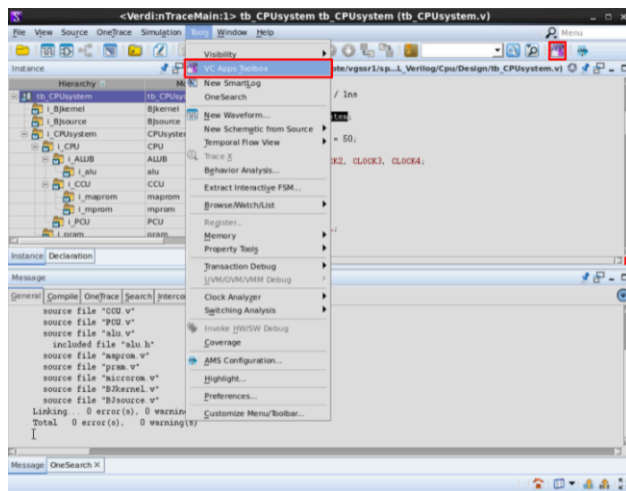
This app does not support gate-level design that nSchema cannot recognize flip-flop or other register type cells.

How to Open the App in VC Apps Toolbox

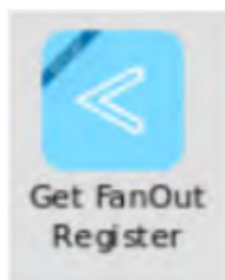
1. Open Verdi with design, select **Tools -> VC Apps Toolbox**;
or click the **VC Apps Toolbox** icon directly

Chapter 3: Design Exploration

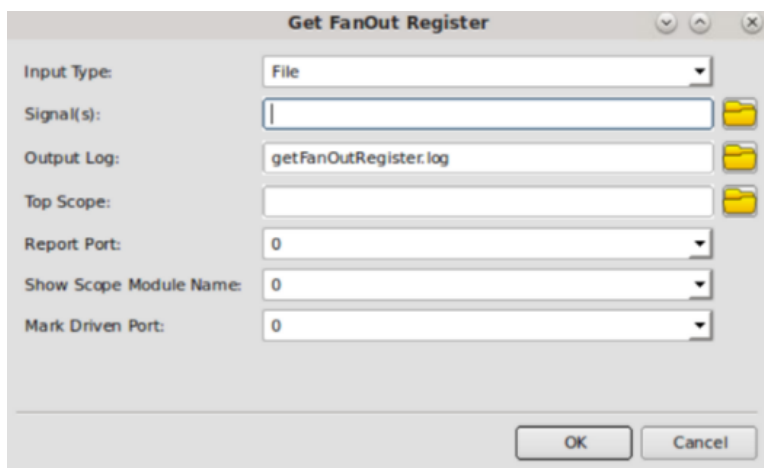
Get FanOut Register



2. Go to **Design Exploration**, click the **Get FanOut Register** icon.

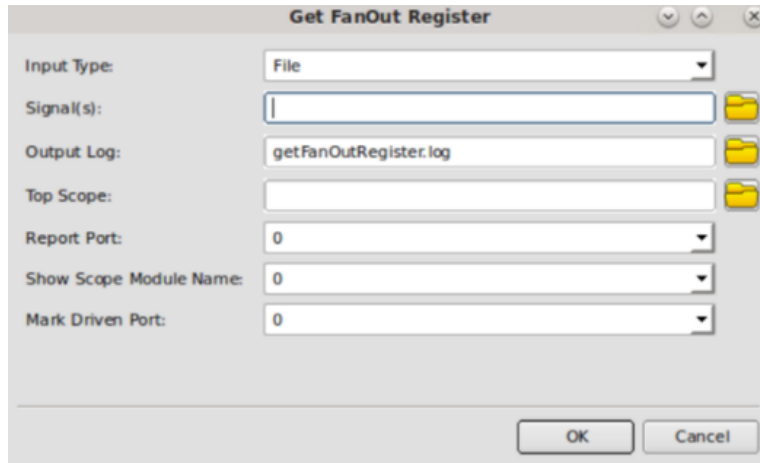


The App opens as shown in the following figure:

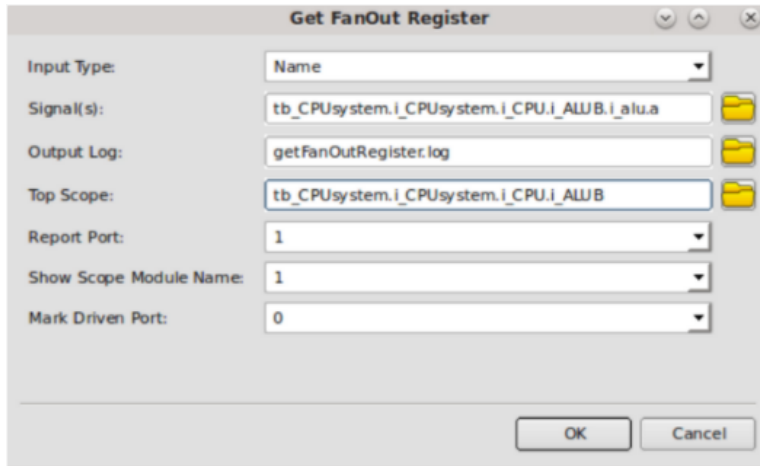


App Usage in Verdi GUI Mode

1. Open App in VC Apps ToolBox.

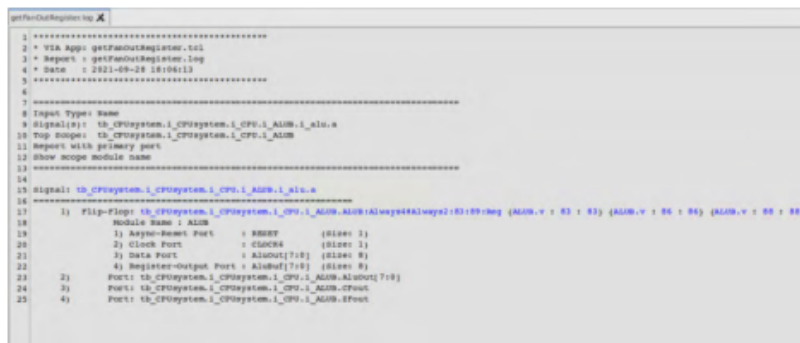


2. Fill in the following fields:
 - **Input Type:** Specify the input type, **Name** or **File**.
 - **Signal(s):** Specify the signal full hierarchical name or file name according to **Input Type**.
 - **Output Log:** Specify the output log file name.
 - **Top Scope:** Specify the top scope full hierarchical name, which cannot be traversed.
 - **Report Port:** Specify whether to list the primary ports of top scope traced in the result.
 - **Show Scope Module Name:** Specify whether to show the scope module name of the register in the result.
 - **Mark Driven Port:** Specify whether to mark the driven port in the result.



3. Click **OK**.

SmartLog displays the output log with the information of fanOut registers with its ports, and the port information of the top scope.



App Usage in Verdi Batch Mode

Script location

\$VERDI_HOME/share/VIA/Apps/Bin/getFanOutRegister.pl

Usage

```
getFanOutRegister.pl <design_import>
-signal "<signal_hier_name>" | -file <signal_file>
[-top <top_scope>]
[-report_port]
[-scope_module]
[-drive_port]
[-o <output_log>]
```

Options

- `-signal`: Specify the target signal hierarchical name.
- `-file`: Specify the file name, which describes multiple signals; the file format is one signal hierarchical name per line.

For example,

```
signal_file
=====
tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu.a
tb_CPUsystem.i_CPUsystem.i_CPU.i_PCU.data
=====
```

Note:

The `-file` and `-signal` options are mutually exclusive; that is, `-file` works only when `-signal` is not given.

- `-top` (Optional): Specify the top scope full hierarchical name, which cannot be traversed.
- `-report_port` (Optional): Specify whether to list the primary ports of the top scope traced in result.
- `-scope_module` (Optional): Specify whether to show the module name of the scope instance of the register.
- `-drive_port` (Optional): Specify whether to show the driven port of the signal fan-out.
- `-o`: Specify the output log name. The default value is `getFanOutRegister.log`.

Example

```
/> getFanOutRegister.pl -f run.f -path Design -signal
"tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu.a" -top
"tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB" -report_port
/> getFanOutRegister.pl -f run.f -path Design -file
signal_file -o getFanOutRegister.log
```

Get Force Statement



Description

Get the force statement in the design.

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/GetForceStmt`

Tcl Entry File

`getForceStmt.tcl``getForceStmt.tcl`

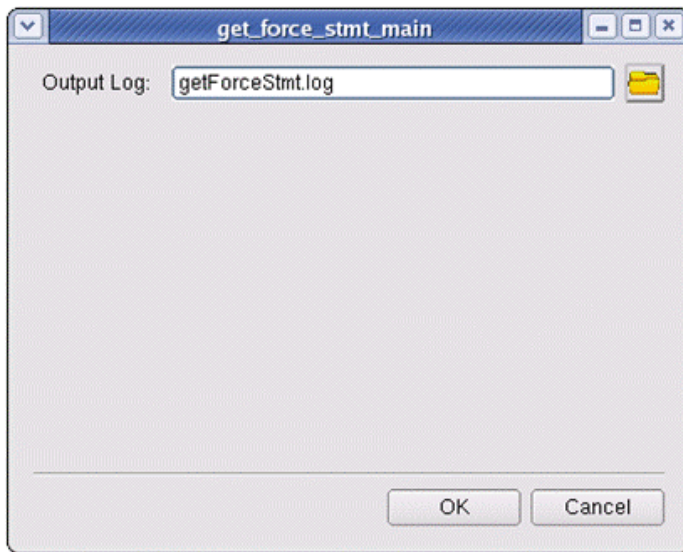
Tcl Procedure Entry

`get_force_stmt_main { output_log }`

Arguments

`output_log`:

Specify the output log file name.



Batch Mode Usage

```
Usage: getForceStmt_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./getForceStmt_batch.pl -f run.f -o getForceStmt.log
```

Limitations

Currently this App won't report the force statements in VHDL components.

Example

```
// ----- example.v ----- //  
  
module TOP;  
  
  wire w1, w2;  
  
  initial force w2 = 1'bz;  
  
  sub M1(w1);  
  sub M2(w2);  
  
endmodule  
  
module sub(input w);
```

```
        initial force w = 1'b1;

endmodule

// ----- getForceStmt.log ----- //

*****
* VIA App: getForceStmt.tcl
* Report : getForceStmt.log
* Date   : 2013-03-19 09:21:52
*****

File: example.v:
-----
    5: initial force w2 = 1'bz;

   14:    initial force w = 1'b1;

   14:    initial force w = 1'b1;
```

Get Generate Scope



Description

Get the design generate scope information.

Tcl Script Location

\$VERDI_HOME/share/VIA/Apps/DesignComprehension/GetGenScope

Tcl Entry File

getGenScope.tcl

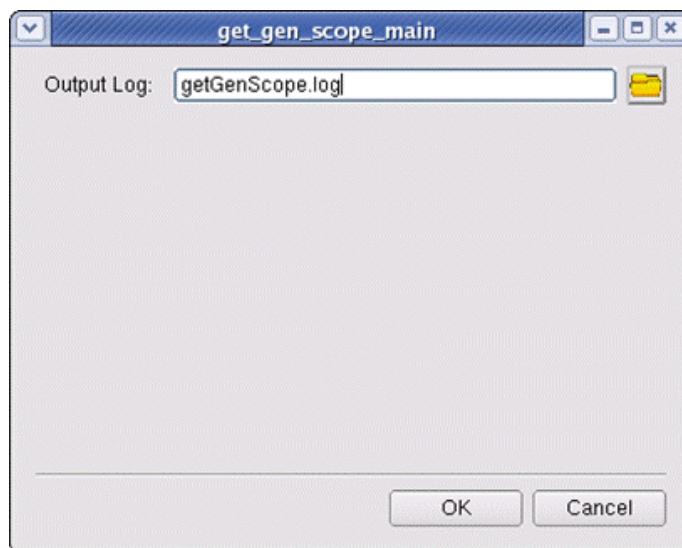
Tcl Procedure Entry

get_gen_scope_main { output_log }

Arguments

output_log:

Specify the output log file name.



Batch Mode Usage

```
Usage: getGenScope_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./getGenScope_batch.pl -f run.f -o getGenScope.log
```

Limitations

Currently this App won't report the generate scope in VHDL components.

Example

```
// ----- example.v ----- //
```

```
module top;
  parameter p = 3;
  wire [7:0] a;
  wire [7:0] aa;
  wire b;

  generate
    if(p > 0)
      begin: B      /* gen scope, top.B */
        and u0 (a[0], a[1],b);
      end
    endgenerate

  genvar i;
  generate
    for (i=0; i<p; i=i+1)
      begin: BB /* gen scope, top.BB[0] ~ top.BB[3] */
        assign a[i] = ^a[p-1:i];
      end
    endgenerate

  generate
    for (i=0; i<p; i=i+1)
      begin /* unamed gen scope, top.genblk3[0] ~ top.genblk3[3] */
        assign aa[i] = ^aa[p-1:i];
      end
    endgenerate

  case(p)
    0:
      begin : u1
        assign a[0] = b;
      end

    1:
      begin : u1
        assign a[1] = b;
      end

    2,3:
      begin : u1
        assign a[2] = b;
      end

    default :
      begin : u1
      end
  end
```



```
        endcase

    endmodule

// ----- getGenScope.log ----- //

*****
* VIA App: getGenScope.tcl
* Report : getGenScope.log
* Date   : 2013-03-19 09:44:48
*****

Scope: top
=====
1) B                (example.v : 9)
2) BB[0]            (example.v : 17)
3) BB[1]            (example.v : 17)
4) BB[2]            (example.v : 17)
5) genblk3[0]        (example.v : 24)
6) genblk3[1]        (example.v : 24)
7) genblk3[2]        (example.v : 24)
8) ul                (example.v : 42)
```

Get Hierarchy CSV



Description

Dump the hierarchy tree in CSV format.

Tcl Script Location

\$VERDI_HOME/share/VIA/Apps/DesignComprehension/GetModHierCSV

Tcl Entry File

getModHierCSV.tcl

Tcl Procedure Entry

```
get_mod_hier_csv_main { target_scope output_file }
```

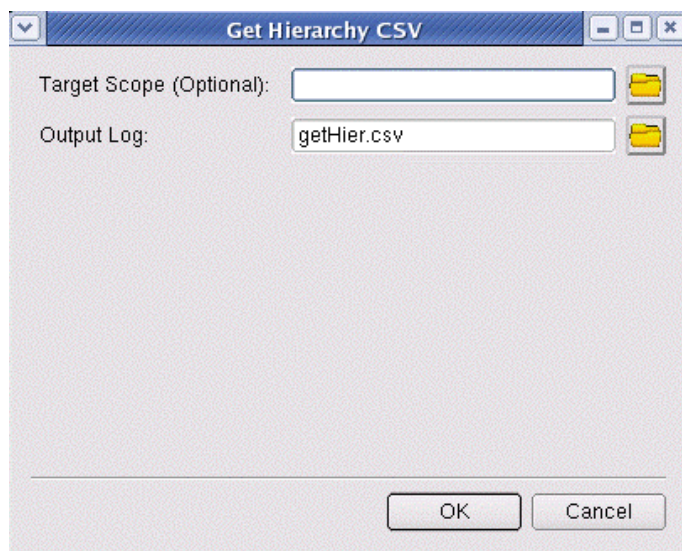
Arguments

target_scope:

Specify the root scope of the module hierarchy tree.

output_log:

Specify the output csv file name.



Batch Mode Usage

```
Usage: getModHierCSV_batch.pl <verdi_import_design> \  
      -target_scope "<design_scope_hier_name>" \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./getModHierCSV_batch.pl -f run.f -target_scope  
"system.top.i_CPU" -o getModHier.csv
```

Limitations

Currently this App won't report the VHDL component hierarchy.

Example

```
// ----- example.v ----- //
```

```
module MOD;  
    task tsk(output integer y, input integer x);  
        endtask  
endmodule  
  
module TOP;  
    wire a, b;  
    and and1[1:0] (a, b);  
  
    generate  
        genvar i;  
        for(i=0; i<2; i=i+1) begin: gen  
            MOD inst();  
        end  
    endgenerate  
  
    MOD inst2 [1:0] ();  
    MOD inst1();  
endmodule  
  
// ----- getHier.csv ----- //
```

```
TOP (TOP)  
  
,inst2[1] (MOD)  
  
,,tsk  
  
,inst2[0] (MOD)  
  
,,tsk  
  
,inst1 (MOD)  
  
,,tsk  
  
,gen[0]  
  
,,inst (MOD)  
  
,,,tsk
```

```
,gen[1]  
,,inst (MOD)  
,,,tsk
```

Get Module Hierarchy



Description

Get the module instance full hierarchy name in the design.

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/GetModHier`

Tcl Entry File

`getModHier.tcl`

Tcl Procedure Entry

```
get_mod_hier_main { target_scope output_log {skip_lib_inst "Yes"} }
```

Arguments

target_scope:

(Optional) Specify the root scope of the module hierarchy tree. Traverse all hierarchy when target_scope is empty.

output_log:

Specify the output log file name.

skip_lib_inst:

"Yes" to skip library cell instance; otherwise keep them.

Batch Mode Usage

```
Usage: getModHier_batch.pl <verdi_import_design> \  
                        -target_scope "<design_scope_hier_name>" \  
                        [-skip_lib_inst] \  
                        [-o <output_log>] \  
                        [-h]
```

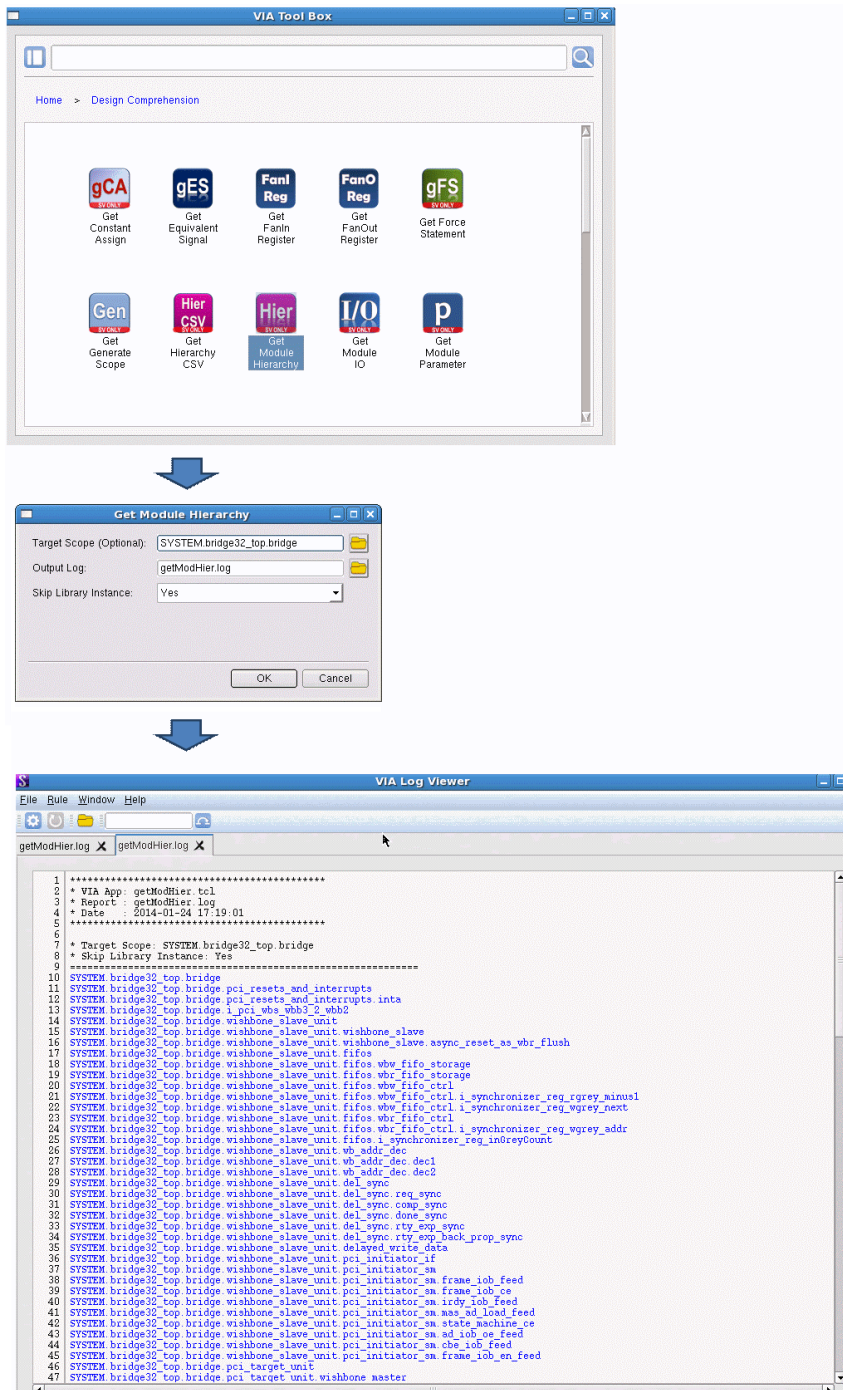
For example:

```
./getModHier_batch.pl -f run.f -target_scope  
"system.top.i_CPU" -o getModHier.log  
./getModHier_batch.pl -f run.f -target_scope  
"system.top.i_CPU" -o getModHier.log -skip_lib_inst
```

Limitations

Currently this App won't report the VHDL component hierarchy.

Example



Get Module IO



Description

Get the module instance I/O port information.

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/GetModIO`

Tcl Entry File

`getModIO.tcl`

Tcl Procedure Entry

`get_mod_io_main { modules output_log }`

Arguments

modules:

(Optional) Specify the module name; default is empty string.

The name can be specified by exact one or wildcard.

The name can be module hierarchy name or module definition name.

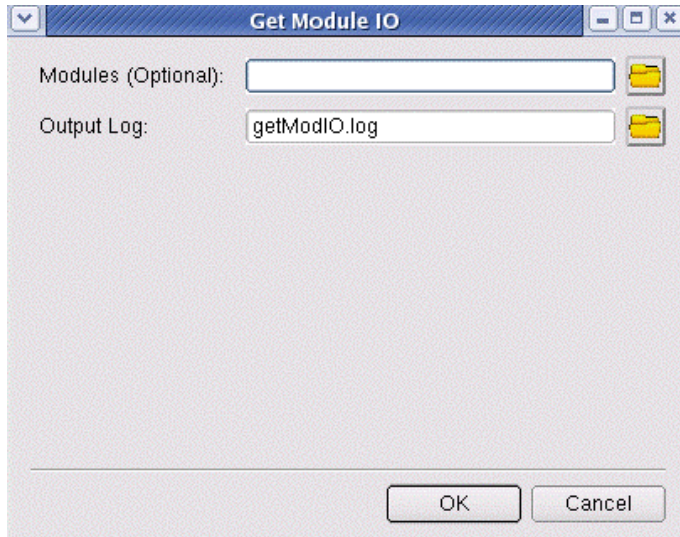
If the module name is empty string, then report all modules.

Use space to specify more than one module; e.g.

"system.i_cpu ALU PC* system.i_cpu.i_*.reg*"

output_log:

Specify the output log file name.



Batch Mode Usage

```
Usage: getModIO_batch.pl <verdi_import_design> \  
        [-modules "<target_module_names>"] \  
        [-o <output_log>] \  
        [-h]
```

For example:

```
./getModIO_batch.pl -f run.f -modules "TOP" -o getModIO.log  
./getModIO_batch.pl -f run.f -modules "system.TOP.* ALU*"  
./getModIO_batch.pl -f run.f
```

Limitations

Currently this App won't report the VHDL component I/O ports.

Example

```
// ----- example.v ----- //  
  
module FF(clk, D, Q);  
  input clk;  
  input [3:0] D;  
  output [3:0] Q;
```



```

    wire clk;
    wire [3:0] D;
    reg [3:0] Q;

    always @(posedge clk) begin
        Q[3:0] = D[3:0];
    end

endmodule

module TOP(clk, data, result);
    input clk;
    input [7:0] data;
    output [7:0] result;
    wire clk;
    wire [7:0] data;
    wire [7:0] result;

    FF X1(clk, data[7:4], result[7:4]);
    FF X2(clk, data[3:0], result[3:0]);

endmodule

// ----- getModIO.log ----- //

*****
* VIA App: getModIO.tcl
* Report : getModIO.log
* Date   : 2013-03-19 11:03:44
*****

=====

Target Modules: ALL

=====

Module: TOP (TOP)
-----
1) Input : clk                (Size: 1)
2) Input : data                (Size: 8)
3) Output: result              (Size: 8)

Module: TOP.X1 (FF)
-----
1) Input : clk                (Size: 1)
2) Input : D                  (Size: 4)
3) Output: Q                  (Size: 4)

```

Module: TOP.X2 (FF)

1) Input : clk	(Size: 1)
2) Input : D	(Size: 4)
3) Output: Q	(Size: 4)

Get Module Parameter



Description

Get the module parameters information.

Tcl Script Location

\$VERDI_HOME/share/VIA/Apps/DesignComprehension/GetModParam

Tcl Entry File

getModParam.tcl

Tcl Procedure Entry

get_mod_param_main { modules output_log }

Arguments

modules:

(Optional) Specify the module name; default is empty string.

The name can be specified by exact one or wildcard.

The name can be module hierarchy name or module definition name.

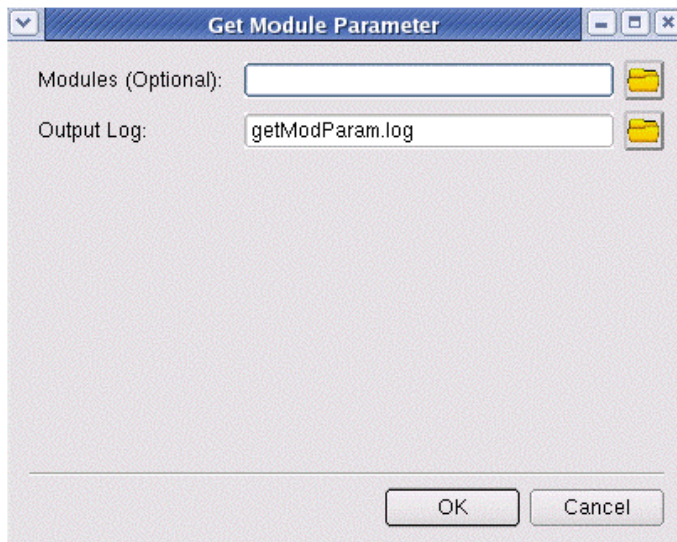
If the module name is empty string, then report all modules.

Use space to specify more than one module; e.g.

"system.i_cpu ALU PC* system.i_cpu.i_*.reg*"

output_log:

Specify the output log file name.



Batch Mode Usage

```
Usage: getModParam_batch.pl <verdi_import_design> \  
        [-modules "<target_module_names>"] \  
        [-o <output_log>] \  
        [-h]
```

For example:

```
./getModParam_batch.pl -f run.f -modules "TOP" -o  
getModParam.log  
./getModParam_batch.pl -f run.f -modules "system.TOP.* ALU*"  
./getModParam_batch.pl -f run.f
```

Limitations

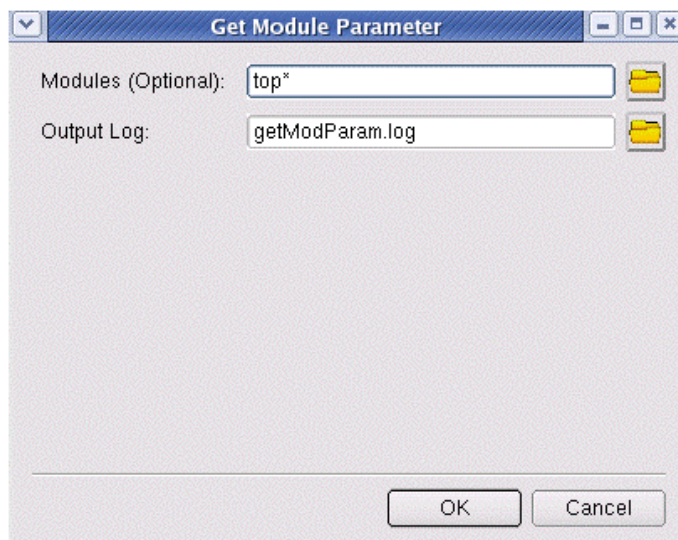
Currently this App won't report the parameters (generics) in VHDL components.

Example

```
// ----- example.v ----- //
```

```
module sub_mod(clk);
    input clk;
    parameter param1 = 512;          /* parameter */
    parameter signed NUM = 4'b1;
    parameter int RANGE = 1024;
    localparam int BOB = 3;         /* local parameter */
    parameter int param2 = RANGE + 256;
    typedef logic signed [5:0] LOGIC;
    parameter type TYPE = LOGIC;    /* type parameter */
endmodule

module top;
    sub_mod #(.RANGE(512)/* param assign */) subMod1(.clk(clk));
    sub_mod subMod2(.clk(clk));
endmodule
```



```
// ----- getModParam.log ----- //
```

```
*****
* VIA App: getModParam.tcl
* Report : getModParam.log
* Date   : 2013-03-19 11:20:39
*****
```

```
=====

Target Modules: top*

=====
```

Module: top.subMod1 (sub_mod)

1) Parameter: param1	Value: 32'h00000200
(example.v : 3)	
2) Parameter: NUM	Value: 4'h1
(example.v : 4)	
3) Parameter: RANGE	Value: 32'hffffffff
(example.v : 5)	
4) Parameter: BOB	Value: 32'h00000003
(example.v : 6)	
5) Parameter: param2	Value: 32'h00000300
(example.v : 7)	
6) Type Param: TYPE	Type : logic signed [5:0] TYPE;
(example.v : 9)	

Module: top.subMod2 (sub_mod)

1) Parameter: param1	Value: 32'h00000200
(example.v : 3)	
2) Parameter: NUM	Value: 4'h1
(example.v : 4)	
3) Parameter: RANGE	Value: 32'h00000400
(example.v : 5)	
4) Parameter: BOB	Value: 32'h00000003
(example.v : 6)	
5) Parameter: param2	Value: 32'h00000300
(example.v : 7)	
6) Type Param: TYPE	Type : logic signed [5:0] TYPE;
(example.v : 9)	

Get Module Signal



Description

Get the module instance signal (net/variable) information

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/GetModSignal`

Tcl Entry File

`getModSignal.tcl`

Tcl Procedure Entry

`get_mod_signal_main { modules output_log }`

Arguments

modules:

(Optional) Specify the module name; default is empty string.

The name can be specified by exact one or wildcard.

The name can be module hierarchy name or module definition name.

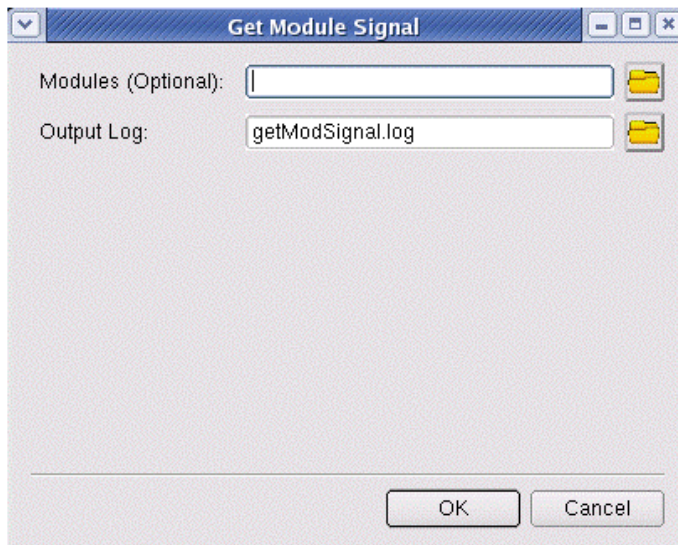
If the module name is empty string, then report all modules.

Use space to specify more than one module; e.g.

`"system.i_cpu ALU PC* system.i_cpu.i_*.reg*"`

output_log:

Specify the output log file name.



Batch Mode Usage

```
Usage: getModSignal_batch.pl <verdi_import_design> \  
      [-modules "<target_module_names>"] \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./getModSignal_batch.pl -f run.f -modules "TOP" -o getModSignal.log  
./getModSignal_batch.pl -f run.f -modules "system.TOP.* ALU*" -o getModSignal.log  
./getModSignal_batch.pl -f run.f
```

Limitations

Currently this App won't report the signals or variables in VHDL components.

Example

```
// ----- example.v -----  
  
module FF(clock, D, Q);  
  input clock, D;  
  output Q;  
  wire clock, D;  
  reg Q;  
  
  always @(posedge clock) begin  
    Q <= D;  
  end
```

```
endmodule

module INV(in, out);
  input in;
  output out;
  wire in, out;

  assign out = ~in;
endmodule

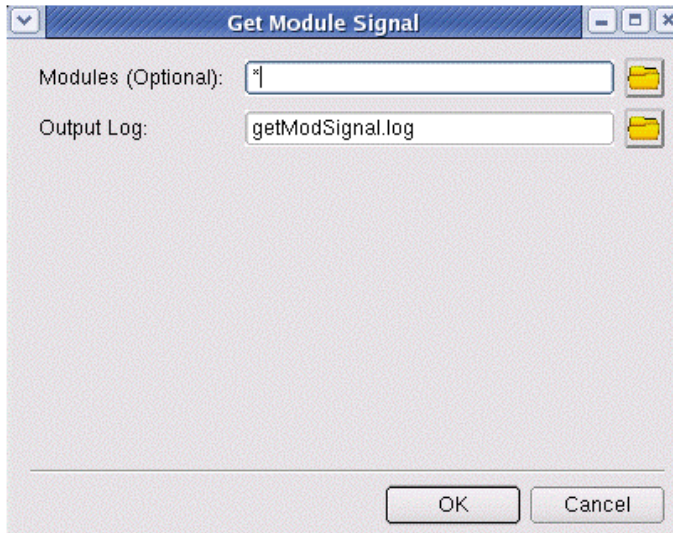
module TOP(clock, PO);
  input clock;
  output PO;
  wire clock, PO;

  wire D1, D2, D3;
  wire Q1, Q2, Q3;
  wire O1;
  wire INV2;

  assign O1 = (Q1 & Q3 | INV2);
  assign D1 = (Q3 & INV2);
  assign D2 = O1;
  assign D3 = O1;
  assign PO = O1;

  FF M1(clock, D1, Q1);
  FF M2(clock, D2, Q2);
  FF M3(clock, D3, Q3);
  INV M4(Q2, IMP);

  INV M5(IMP, INV2);
endmodule
```

```
// ----- getModSignal.log ----- //
```

```
*****
* VIA App: getModSignal.tcl
* Report : getModSignal.log
* Date   : 2013-03-19 11:31:11
*****
```

```
=====
```

```
Target Modules: *
```

```
=====
```

```
Module: TOP (TOP)
```

```
-----
```

1) net: clock	Type: logic (logic unsigned clock;)
2) net: P0	Type: logic (logic unsigned P0;)
3) net: D1	Type: logic (logic unsigned D1;)
4) net: D2	Type: logic (logic unsigned D2;)
5) net: D3	Type: logic (logic unsigned D3;)
6) net: Q1	Type: logic (logic unsigned Q1;)
7) net: Q2	Type: logic (logic unsigned Q2;)
8) net: Q3	Type: logic (logic unsigned Q3;)
9) net: O1	Type: logic (logic unsigned O1;)
10) net: INV2	Type: logic (logic unsigned INV2;)
11) net: IMP	Type: logic (logic unsigned IMP;)

```
Module: TOP.M1 (FF)
```

```
-----
```

1) net: clock	Type: logic (logic unsigned clock;)
2) net: D	Type: logic (logic unsigned D;)

```

3)  var: Q                                Type: reg ( logic unsigned Q; )

Module: TOP.M2 (FF)
-----
1)  net: clock                            Type: logic ( logic unsigned clock; )
2)  net: D                                Type: logic ( logic unsigned D; )
3)  var: Q                                Type: reg ( logic unsigned Q; )

Module: TOP.M3 (FF)
-----
1)  net: clock                            Type: logic ( logic unsigned clock; )
2)  net: D                                Type: logic ( logic unsigned D; )
3)  var: Q                                Type: reg ( logic unsigned Q; )

Module: TOP.M4 (INV)
-----
1)  net: in                               Type: logic ( logic unsigned in; )
2)  net: out                              Type: logic ( logic unsigned out; )

Module: TOP.M5 (INV)
-----

1)  net: in                               Type: logic ( logic unsigned in; )

2)  net: out                              Type: logic ( logic unsigned out; )

```

Get Netlist Overview



Retrieves the netlist overview information of the whole design.

Description

This App reports the overview of design's netlist. For each module, it reports the total gate count, and the individual gate count usage. Moreover, for each gate, it reports the total count used in the design.

The definition of gate is: modules compiled by -v/-y or celldefine, and the object type in NPI Language model is npPrimitive.

(npGate, npSwitch, npUdp)

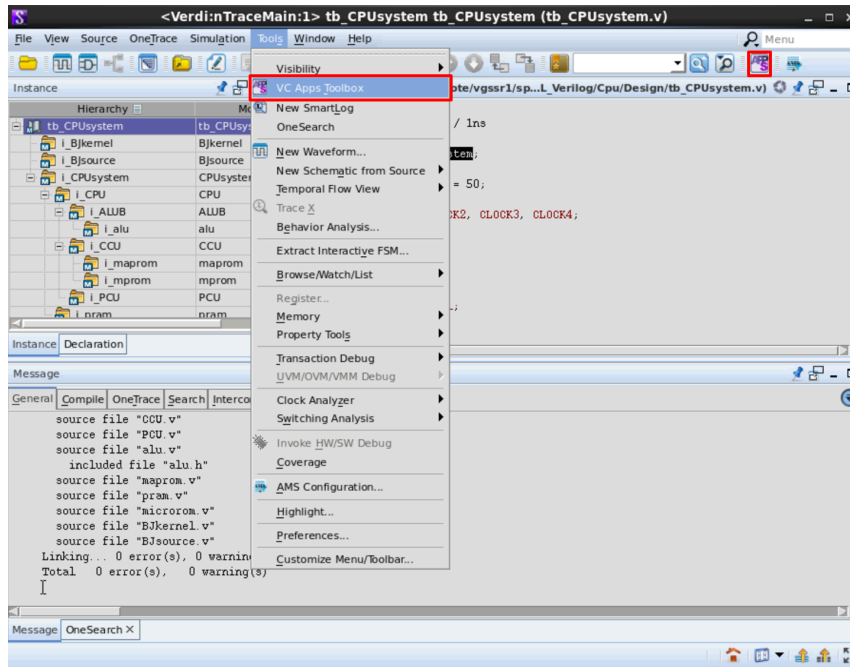
Limitation

None.

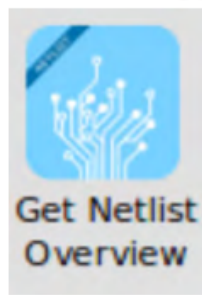
How to Open the App in VC Apps Toolbox

To open the App in the VC Apps toolbox, perform the following steps:

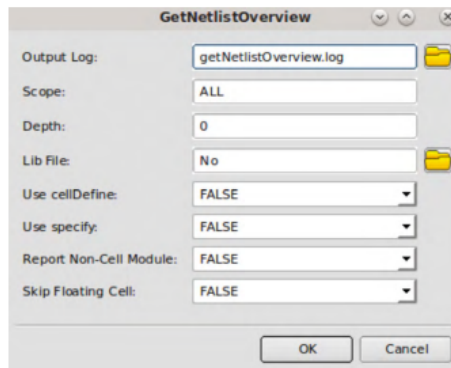
1. Open Verdi with design by selecting **Tools > VC Apps Toolbox**. Otherwise, you can click the **VC Apps Toolbox** icon directly.



2. Go to **Design Exploration** and click the **Get Netlist Overview** icon.



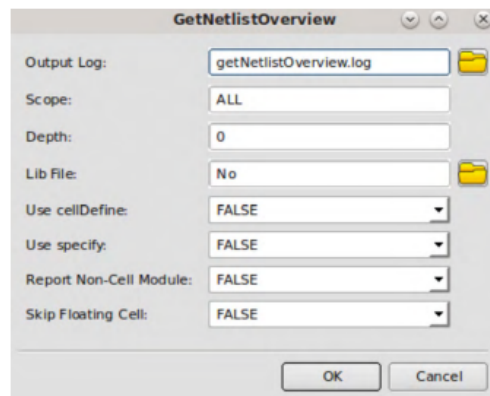
The Get Netlist Overview App opens.



App Usage in Verdi GUI Mode

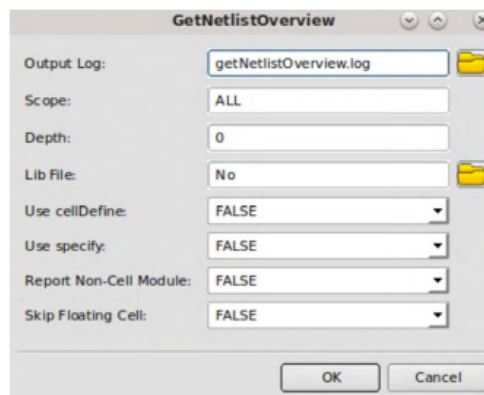
To use the App in the Verdi GUI mode, perform the following steps:

1. From the **VC Apps toolbox**, open the **Get Netlist Overview** App.



2. Fill in the following fields:

- **Output Log:** Specify the output log file name. The default value is `getNetlistOverview.log`.
- **Scope:** Specify the target scope.
- **Depth:** Specify the traverse level. The default value is `traverse all`.
- **Lib File:** Specify the library file name, which defines lib cells.
- **Use celldefine:** Specify whether to use celldefine to recognize a lib cell.
- **Use specify:** Specify whether to use contain specify block to recognize a lib cell.
- **Report Non-Cell Module:** When specified, it reports non cell sub module and its number.
- **Skip Floating Cell:** Specify whether to skip the floating cell, which has no high conn of its ports.



3. Click **OK**.

The SmartLog displays the output log, with information of fanOut registers with its ports, and port information of the top scope.

```

1 *****
2 * VC App: getNetlistOverview
3 * Report : getNetlistOverview.log
4 * Date : 2021-11-08 19:04:22
5 *****
6
7 *****
8 *
9 * Date Usage Information in a Module *
10 *
11 *****
12
13 Module: top (top)
14 -----
15 1) and : 1
16 2) cmos : 2
17 3) latch : 3
18 4) nand : 1
19 5) nmos : 1
20 6) pmos : 1
21 Total Gate Count: 9
22
23 *****
24 *
25 * Date Information in the entire Netlist *
26 *
27 *****
28
29 1) and : 1
30 2) cmos : 2
31 3) latch : 3
32 4) nand : 1
33 5) nmos : 1
34 6) pmos : 1
35 Total Gate Count: 9

```

App Usage in Verdi Batch Mode

Script location

\$VERDI_HOME/share/VIA/Apps/Bin/getNetlistOverview.pl

Usage

```

getNetlistOverview.pl <design_import>
[-o <output_log>]
[-scope <scope>]
[-depth <depth>]
[-lib_file <library_file>]
[-use_cell_def]
[-use_specify]
[-report_non_cell_module]
[-skip_floating_cell]

```

Options

- -o: Specify the output log. The default value is listRegisters.log.
- -scope: (Optional) Specify the target scope.
- -depth: (Optional) Specify the traverse level. The default value is traverse all.

- `-lib_file`: (Optional) Specify the library file name, which defines lib cells.
- `-use_cell_def`: (Optional) Specify whether to use `celldefine` to recognize a lib cell. The default value is `false`.
- `-use_specify`: (Optional) Specify whether to use `contain specify` block to recognize a lib cell. The default value is `false`.
- `-report_non_cell_module`: (Optional) When specified, it reports non cell sub module and its number.
- `-skip_floating_cell`: (Optional) Specify whether to skip the floating cell, which has no high conn of its ports.

Example

```
/> getNetlistOverview.pl -sv demo.v -o getNetlistOverview.log
```

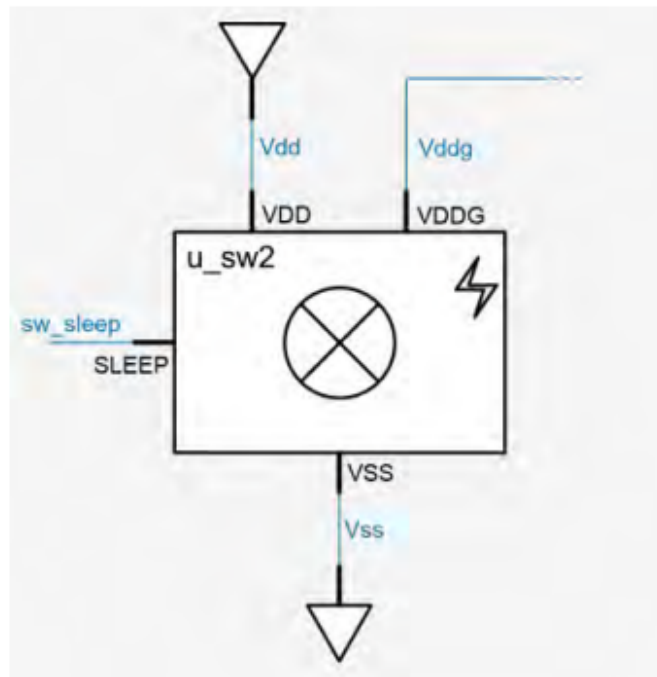
Get PG Pin Connection



Description

This App reports the `pg_pins` and its connected signal.

You can specify module instance either by name or by file. If instance is not specified, this App traverses all instances and reports all `pg_pins` with its connected net. If a `pg_pin` has no connection, then the App does not report.



Limitation

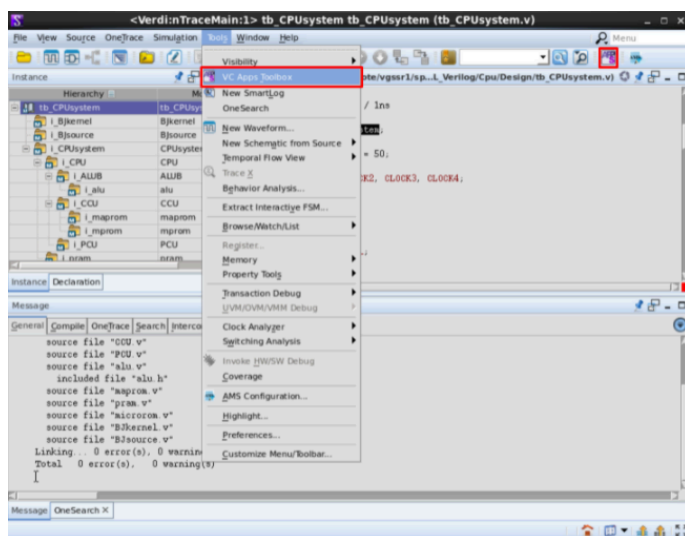
None

How to Open the App in VC Apps Toolbox

1. Open Verdi with design, select **Tools -> VC Apps Toolbox**;
or click the **VC Apps Toolbox** icon directly.

Chapter 3: Design Exploration

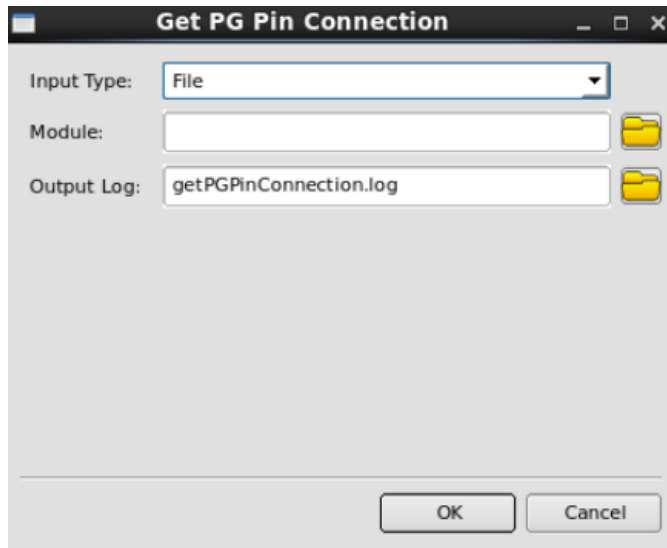
Get PG Pin Connection



2. Go to **Design Validation**, click the **Get PG Pin Connection** icon.

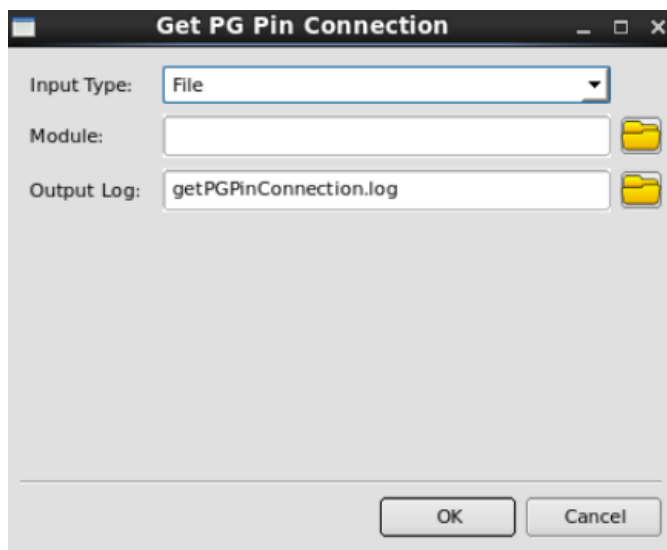


The App opens as shown in the following figure:



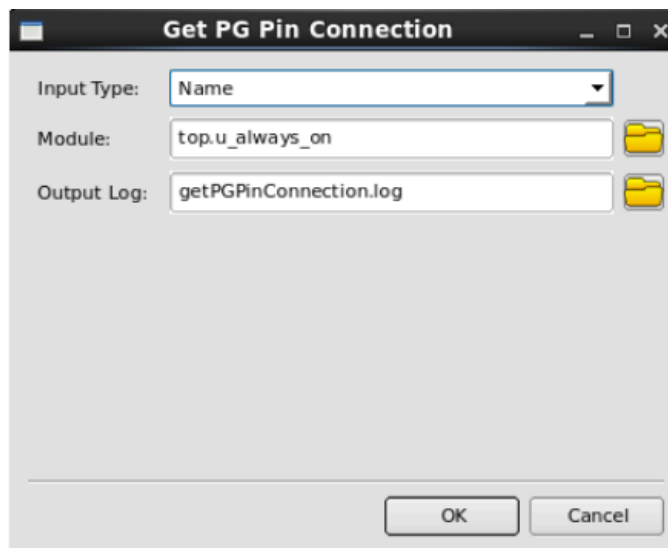
App Usage in Verdi GUI Mode

1. Open App in VC Apps ToolBox.



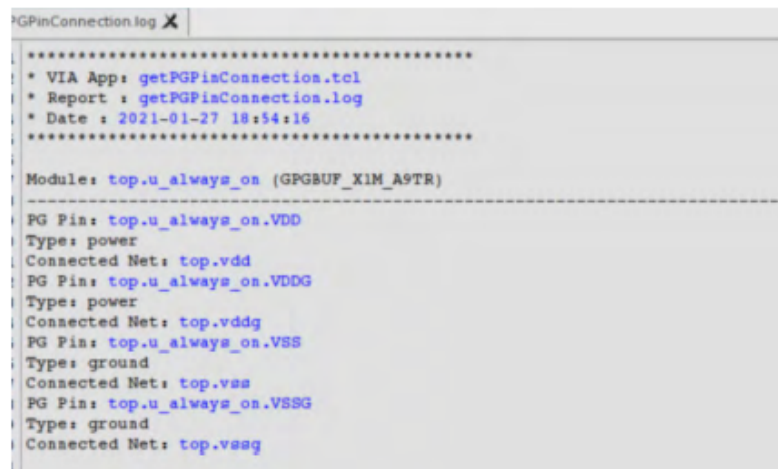
2. Fill in the following fields:
 - **Input Type:** Specify the input type, **Name** or **File**.
 - **Module:** Specify the target module full hierarchical name.

- **Output Log:** Specify the output log file name.



3. Click **OK**.

SmartLog displays the output log, with the information of the dangling port and where it is dangling.



App Usage in Verdi Batch Mode

Script location

\$VERDI_HOME/share/VIA/Apps/Bin/getPGPinConnection.pl

Usage

```
getPGPinConnection.pl <design_import>  
-libpaths <lib_paths>  
-libs <lib_names>  
[-module <inst_full_name> | -file <file_of_inst_full_name>]  
[-o <output_log>]  
[-checkLib]
```

Options

- **-libpaths:** Specify the symbol library path(s). Multiple path strings can be separated by either a space or a colon (:) character.
- **-libs:** Specify the symbol library name(s). Multiple library names can be separated by either a space or a colon (:) character.
- **-module (Optional):** Specify the target module full hierarchical name.
- **-file:** Specify the file name which describes multiple instances name. The file format is one instance hierarchical name per line.

For example,

```
inst_file  
=====
```

```
top.u_always_on  
top.u_iso  
=====
```

Note:

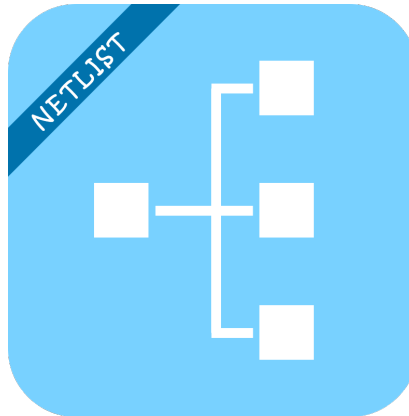
The **-file** and **-module** options are mutually exclusive; **-file** works only when **-module** is not given. If both **-file** and **-module** are not specified, by default it will traverse all module instances.

- **-o:** Specify the output log name. The default value is `getPGPinConnection.log`.
- **-checkLib (Optional):** When specified, it checks the ERROR and Failed to import string in `libCompiler.log`.

Example

```
/> getPGPinConnection.pl -sv top.v -libpaths "./symlib" -libs  
"c9_cln45gs_pmk_rvt_ff_typical_min_0p99v_125c:sc9_cln45gs_pmk  
_rvt_ff_typical_min_0p99v_0p99v_125c" -module  
"top.u_always_on"
```

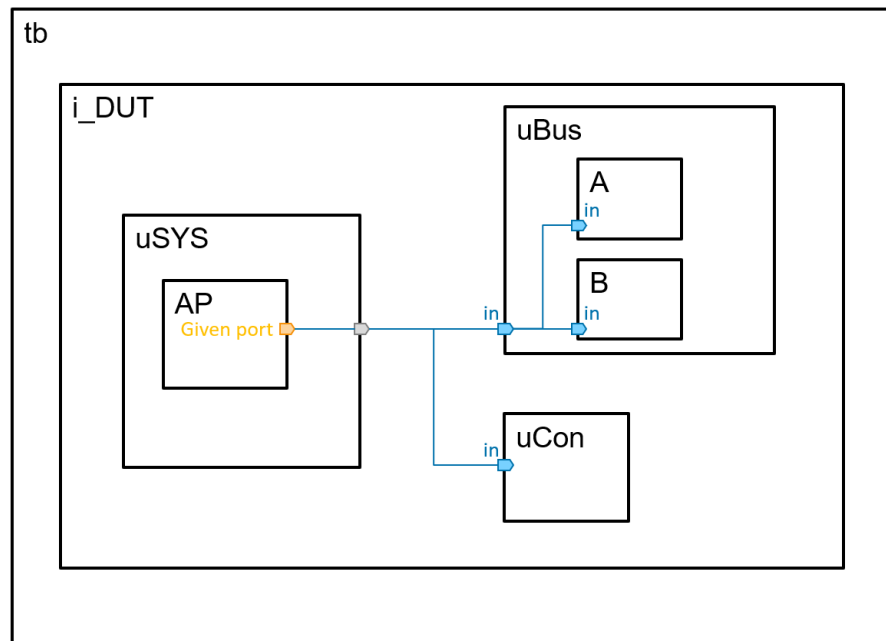
Get Port Connection



Description

This App will traverse one-level upper and then one-level lower of the connected ports, and report the ports with opposite direction of specified port. The module name are also reported to help user check the connection between modules. User can specify the port either by name or by file.

For example, given output port "tb.i_DUT.uSYS.AP.out", this APP will try to find connected input/inout ports of instances in the scope "tb.i_DUT" and instances in one scope under "tb.i_DUT". In this case, the port "tb.i_DUT.uBus.in", "tb.i_DUT.uBus.A.in", "tb.i_DUT.uBus.B.in", and "tb.i_DUT.uCon.in" are reported.

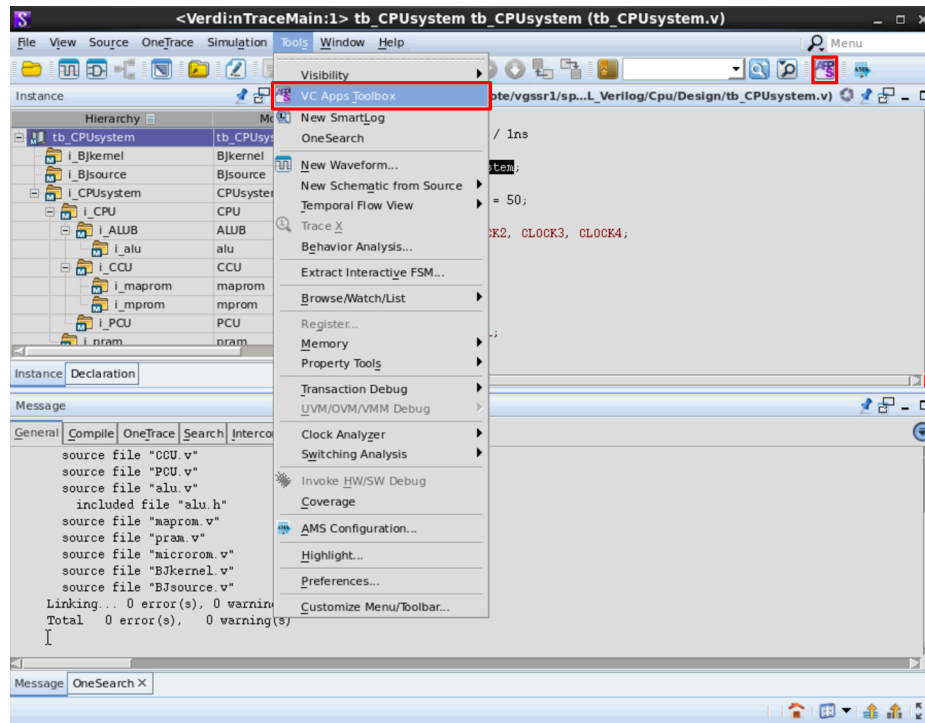


Limitation

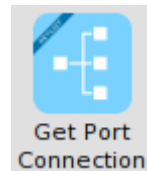
None.

How to Open the App in VC Apps Toolbox

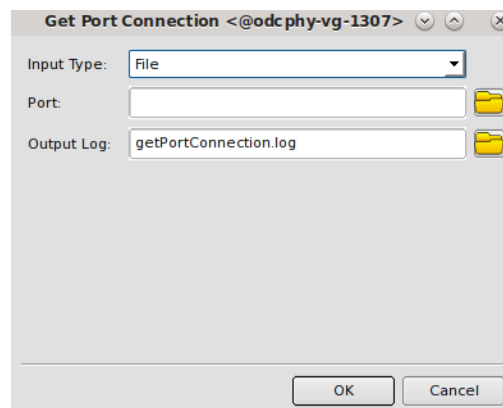
1. Open Verdi with design, select **Tools -> VC Apps Toolbox**;
or click the **VC Apps Toolbox** icon directly.



2. Go to **Design Validation**, click the **Get Port Connection** icon.

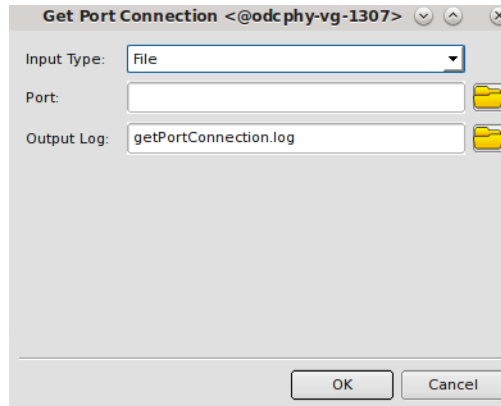


The App is opens:



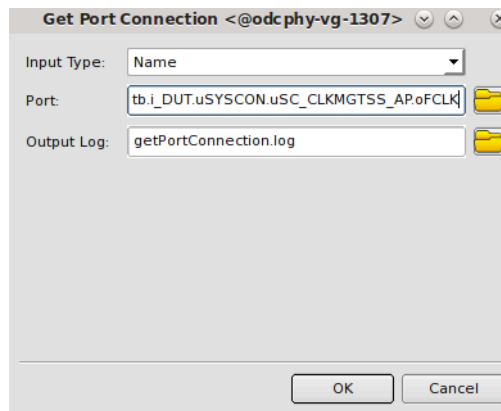
App Usage in Verdi GUI Mode

1. Open App in VC Apps ToolBox



2. Fill in the Arguments

- **Input Type:** Specify the input type, "Name" or "File"
- **Port:** Specify the target port full hierarchical name
- **Output Log:** Specify the output log file name



3. Click OK.

SmartLog displays the output log, with connected ports and the module names.

```
getPortConnection.log X
1 *****
2 * VIA App: getPortConnection.tcl
3 * Report : getPortConnection.log
4 * Date : 2021-03-03 19:57:11
5 *****
6
7 =====
8 Input Type: Name
9 Port(s): tb.i_DUT.uSYSCON.uSC_CLKMGTSS_AP.oFCLK
10 =====
11
12 Port: tb.i_DUT.uSYSCON.uSC_CLKMGTSS_AP.oFCLK
13 Direction: npinOutput
14 -----
15 1) tb.i_DUT.uBUS_SC000.HCLK (BUS)
16 1) tb.i_DUT.uBUS_SC000.uAHB_SLAVE_MUX.HCLK (AHB_SLAVE_MUX)
17 2) tb.i_DUT.uBUS_SC000.uPERIBUS.HCLK (PERIBUS)
18 3) tb.i_DUT.uBUS_SC000.uSC_DECODER_SYSTEM_AH.HCLK (SC_DECODER_SYSTEM_AH)
19 2) tb.i_DUT.uCONNECT.CPUCLK (CONNECTIVITY)
20 3) tb.i_DUT.uSECURITY.FCLK (SECURITY)
21 4) tb.i_DUT.uSYSPERI.CPUCLK (SYSPERI)
22 5) tb.i_DUT.uSYSCON.uSC_RESCNTL_AP.PCLK (SC_RESCNTL_AP)
23
```

App Usage in Verdi Batch Mode

Script location

\$VERDI_HOME/share/VIA/Apps/Bin/getPortConnection.pl

Usage

```
getPortConnection.pl <design_import>
-port <port_full_name> | -file <file_of_port_full_name>
[-o <output_log>]
```

Options

- -port: Specify the complete hierarchical name of the target port.
- -file: Specify the file name, which describes multiple ports name.

The file format is one port hierarchical name per line.

For example,

```
port_file
=====
tb.i_DUT.uSYSCON.uSC_CLKMGTSS_AP.oFCLK
tb.i_DUT.uBUS_SC000.uAHB_SLAVE_MUX.HCLK
=====
```

Note:

The `-file` and `-port` options are mutually exclusive; `file` works only when `-port` is not provided. `-o` specifies the output log name; default is `getPortConnection.log`.

Example

```
/> getPortConnection.pl -sv -f run.f -port  
"tb.i_DUT.uSYSCON.uSC_CLKMGTSS_AP.oFCLK"
```

List Include Files



Description

List the include files used in each file of a design.

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/ListIncludeFiles`

Tcl Entry File

`listIncludeFiles.tcl`

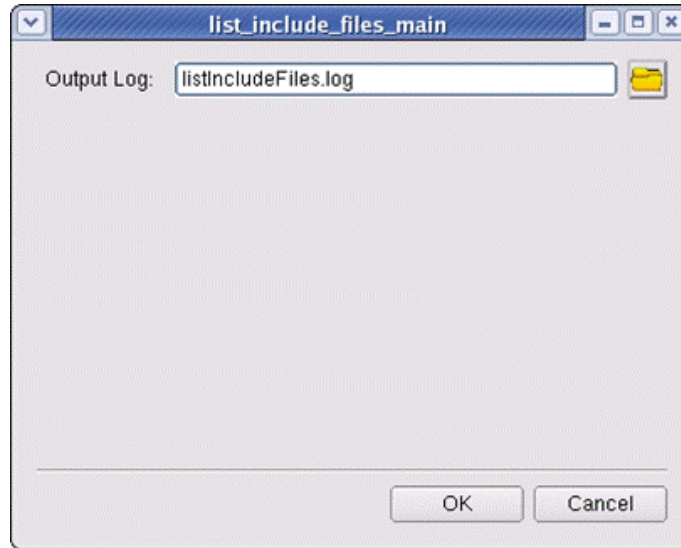
Tcl Procedure Entry

`list_include_files_main { output_log }`

Arguments

output_log:

Specify the output log file name.



Batch Mode Usage

```
Usage: listIncludeFiles_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./listIncludeFiles_batch.pl -f run.f -o listIncludeFiles.log
```

Limitations

N/A

Example

```
// ----- NFF.v ----- //  
  
module NFF(clk, D, Q);  
  input clk, D;  
  output Q;  
  wire clk, D;  
  reg Q;
```

```

        always @(negedge clk) begin
            Q <= D;
        end

    endmodule

// ----- FF.v ----- //

`include "NFF.v"

module FF(clk, D, Q);
    input clk, D;
    output Q;
    wire clk, D;
    reg Q;

    always @(posedge clk) begin
        Q <= D;
    end

endmodule

// ----- example.v ----- //

`include "FF.v"
module TOP(clk);
    input clk;
    wire clk;

    wire D1, D2;
    wire Q1, Q2;

    assign D1 = (Q1 & Q2);
    assign D2 = (Q1 | Q2);

    FF X1(clk, D1, Q1);
    NFF X2(clk, D2, Q2);

endmodule;

// ----- listIncludeFiles.log ----- //

*****
* VIA App: listIncludeFiles.tcl
* Report : listIncludeFiles.log
* Date   : 2013-03-19 14:01:06
*****

File: (example.v)
-----
1    : include "FF.v"

```

File: (FF.v)

```
1 : include "NFF.v"
```

List Module Define



Description

List all the module definition names in the design.

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/ListModDefine`

Tcl Entry File

`listModDefine.tcl`

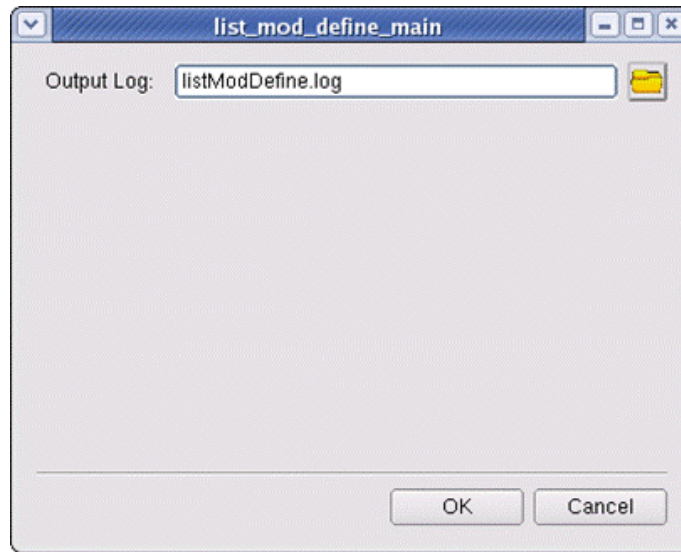
Tcl Procedure Entry

`list_mod_define_main { output_log }`

Arguments

`output_log`:

Specify the output log file name.



Batch Mode Usage

```
Usage: listModDefine_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./listModDefine_batch.pl -f run.f -o listModDefine.log
```

Limitations

Currently this App won't report the entity names for VHDL components.

Example

```
// ----- example.v ----- //  
  
module NFF(clk, D, Q);  
  input clk, D;  
  output Q;  
  wire clk, D;  
  reg Q;  
  
  always @(negedge clk) begin  
    Q <= D;  
  end  
endmodule  
  
module FF(clk, D, Q);
```

```
    input clk, D;
    output Q;
    wire clk, D;
    reg Q;

    always @(posedge clk) begin
        Q <= D;
    end
endmodule

module TOP(clk);

    input clk;
    wire clk;

    wire D1, D2;
    wire Q1, Q2;

    assign D1 = (Q1 & Q2);
    assign D2 = (Q1 | Q2);

    FF X1(clk, D1, Q1);
    NFF X2(clk, D2, Q2);

endmodule;

// ----- listModDefine.log ----- //

*****
* VIA App: listModDefine.tcl
* Report : listModDefine.log
* Date   : 2013-03-19 14:13:03
*****

1) FF
2) NFF
3) TOP
```

List Module Files



Description

List all module definition names and their corresponding text files (including line number).

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/ListModFiles`

Tcl Entry File

`listModFiles.tcl`

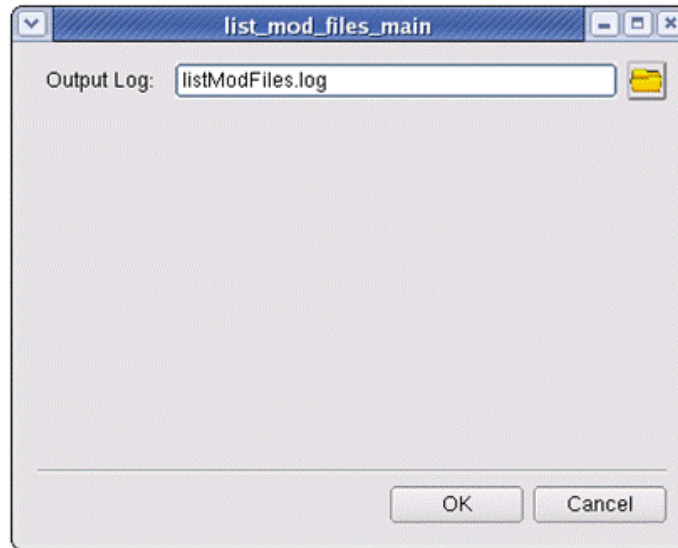
Tcl Procedure Entry

`list_mod_files_main { output_log }`

Arguments

`output_log`:

Specify the output log file name.



Batch Mode Usage

```
Usage: listModFiles_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./listModFiles_batch.pl -f run.f -o listModFiles.log
```

Limitations

Currently this App won't report the files for VHDL components.

Example

```
// ----- NFF.v ----- //  
  
module NFF(clk, D, Q);  
  input clk, D;  
  output Q;  
  wire clk, D;  
  reg Q;  
  
  always @(negedge clk) begin  
    Q <= D;  
  end  
  
endmodule
```

```
// ----- FF.v ----- //
```

```
`include "NFF.v"
```

```
module FF(clk, D, Q);  
  input clk, D;  
  output Q;  
  wire clk, D;  
  reg Q;  
  
  always @(posedge clk) begin  
    Q <= D;  
  end  
  
endmodule
```

```
// ----- example.v ----- //
```

```
`include "FF.v"
```

```
module TOP(clk);  
  input clk;  
  wire clk;  
  
  wire D1, D2;  
  wire Q1, Q2;  
  
  assign D1 = (Q1 & Q2);  
  assign D2 = (Q1 | Q2);  
  
  FF X1(clk, D1, Q1);  
  NFF X2(clk, D2, Q2);  
  
endmodule;
```

```
// ----- listModFiles.log ----- //
```

```
*****  
* VIA App: listModFiles.tcl  
* Report : listModFiles.log  
* Date   : 2013-03-19 15:20:05  
*****
```

```
TOP                ( example.v : 2 )  
FF                  ( FF.v : 3 )  
NFF                  ( NFF.v : 1 )
```

List Module Include Files



Description

List the include files used in the modules

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/ListModIncFiles`

Tcl Entry File

`listModIncFiles.tcl`

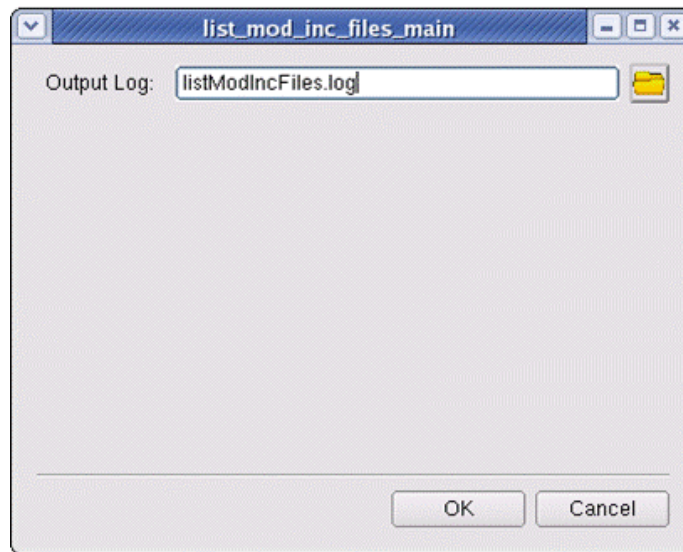
Tcl Procedure Entry

`list_mod_inc_files_main { output_log }`

Arguments

`output_log`:

Specify the output log file name.



Batch Mode Usage

```
Usage: listModIncFiles_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./listModIncFiles_batch.pl -f run.f -o listModIncFiles.log
```

Limitations

Currently this App won't report the include files for VHDL components.

Example

```
// ----- NFF.v ----- //  
  
module NFF(clk, D, Q);  
  input clk, D;  
  output Q;  
  wire clk, D;  
  reg Q;  
  
  always @(negedge clk) begin  
    Q <= D;  
  end  
  
endmodule
```

```
// ----- FF.v ----- //
```

```
module FF(clk, D, Q);  
    input clk, D;  
    output Q;  
    wire clk, D;  
    reg Q;  
  
    always @(posedge clk) begin  
        Q <= D;  
    end  
  
endmodule
```

```
// ----- INC.v ----- //
```

```
parameter param1 = 1;  
parameter param2 = 0;
```

```
// ----- example.v ----- //
```

```
`include "NFF.v"  
  
`include "FF.v"  
  
module TOP(clk);  
    input clk;  
    wire clk;  
  
    wire D1, D2;  
    wire Q1, Q2;  
  
    `include "INC.v"  
  
    assign D1 = (Q1 & Q2 & param1);  
    assign D2 = (Q1 | Q2 | param2);  
  
    FF X1(clk, D1, Q1);  
    NFF X2(clk, D2, Q2);  
  
endmodule;
```

```
// ----- listModIncFiles.log ----- //
```

```
*****  
* VIA App: listModIncFiles.tcl  
* Report  : listModIncFiles.log  
* Date    : 2013-03-19 15:26:59  
*****
```

```
Module: TOP (example.v)  
-----  
11    : include "INC.v"
```

List Registers



Lists the inferred registers in the design.

Description

This App traverses into the design and lists all the registers with port information, under user specified target scope. Furthermore, users can specify "Non Resettable Flop" to only dump the non-resettable flops.

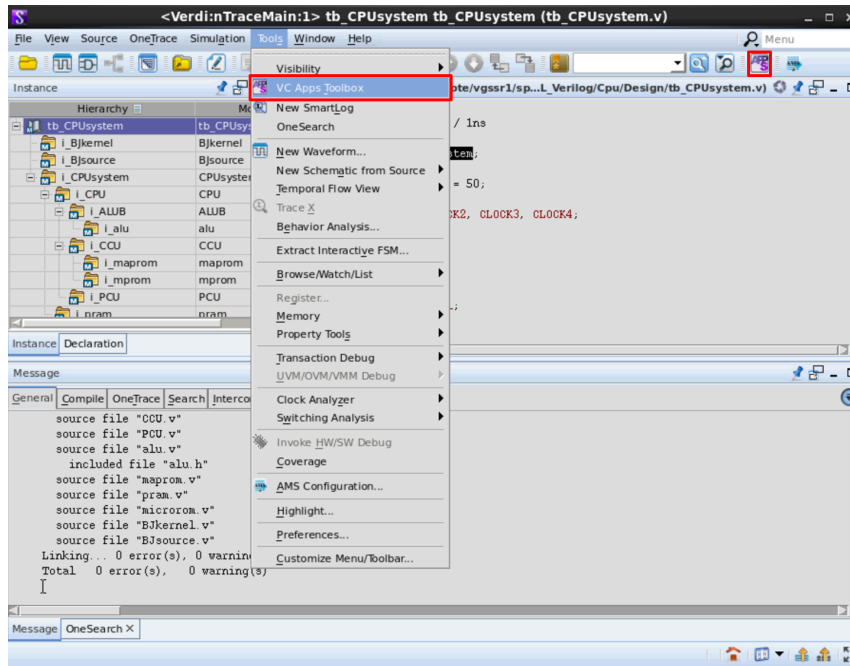
Limitation

This App does not support gate-level design that nSchema cannot recognize as flip-flops or other register type cells.

How to Open the App in VC Apps Toolbox

To open the App in the VC Apps toolbox, perform the following steps:

1. Open Verdi with design by selecting **Tools > VC Apps Toolbox**. Otherwise, you can click the **VC Apps Toolbox** icon directly.



2. Go to **Design Exploration** and click the **List Registers** icon.



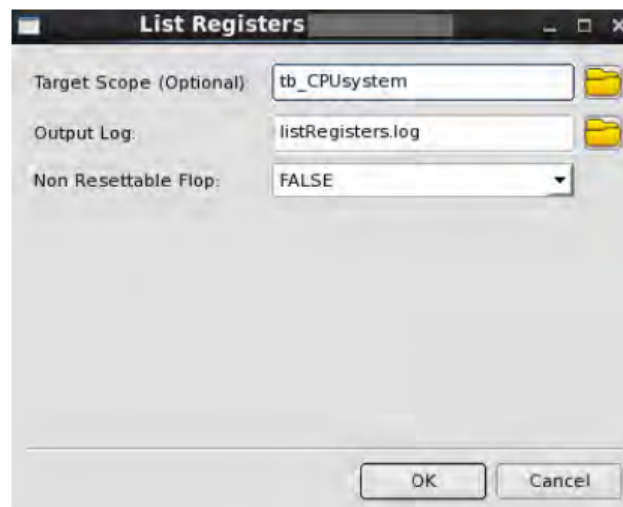
The List Registers App opens.



App Usage in Verdi GUI Mode

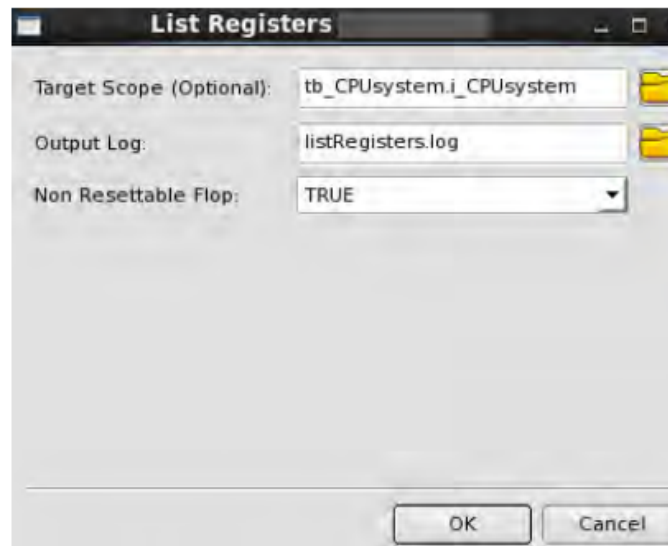
To use the App in the Verdi GUI mode, perform the following steps:

1. From the **VC Apps toolbox**, open the **List Registers App**.

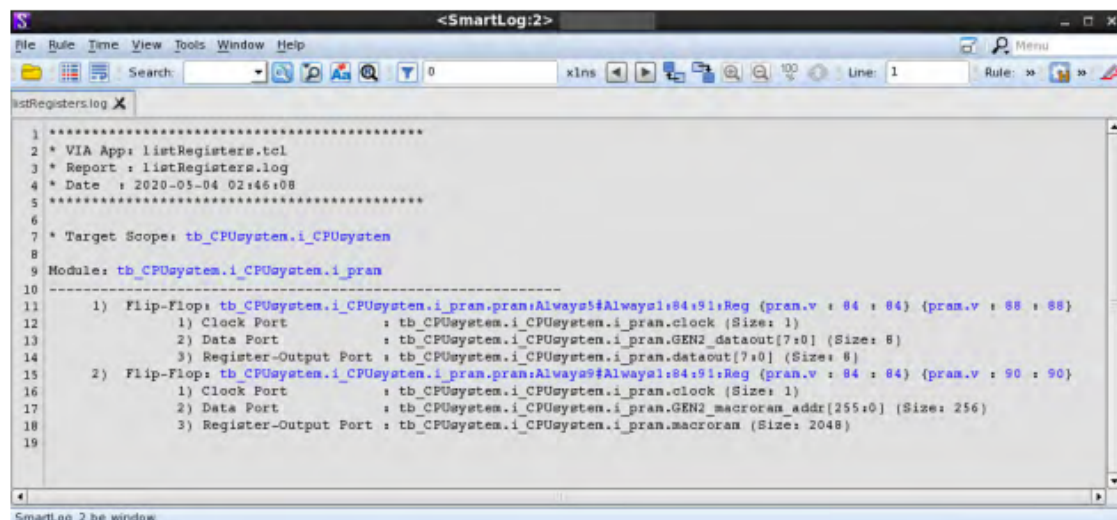


2. Fill in the following fields:
 - **Target Scope:** Specify the target scope full hierarchical name to list registers.
 - **Output Log:** Specify the output log file name.

- **Non Resettable Flop:** Specify whether to only dump the non-resettable flop.



3. Click **OK**. The SmartLog displays the output log, with information of all registers with its ports and port type in each. modules.



App Usage in Verdi Batch Mode

Script location

\$VERDI_HOME/share/VIA/Apps/Bin/getFanInRegister.pl

Usage

```
listRegisters.pl <design_import>  
[-target_scope <target_scope>]  
[-o <output_log>]  
[-non_reset_flop]
```

Options

- `-target_scope`: (Optional) Specify the target scope full hierarchical name to list registers.
- `-o`: Specify the output log. The default value is `listRegisters.log`.
- `-non_reset_flop`: (Optional) Specify whether to only dump the non-resettable flop.

Example

```
/> listRegisters.pl -f run.f -path Design -target_scope  
"tb_CPUsystem.i_CPUsystem" -non_reset_flop
```

List Scope Parameter



Lists detail parameter information of specified scopes and NPI types.

Description

The **List Scope Parameter** VC App reports detail parameter information, including decompile string, final value, and default value.

1. Decompile: The RHS string of parameter definition.
2. Final value: The parameter value after evaluating parameter assignment and defparam.
3. Default value: The RHS value of parameter definition.

This VC App calculates parameter value if it is an operation, such as "1+2+3".

```
1 `define STR "NewString"
2
3 interface intf;
4     parameter P_INTF = 8;
5 endinterface
6
7 module sub;
8     parameter PARAM_1 = 1+2+3;
9     parameter PARAM_2 = 2;
10    parameter PARAM_3 = 3.0;
11    parameter PARAM_4 = "string";
12    genvar i;
13    for (i=0; i<4; i++) begin
14        parameter P = i;
15    end
16 endmodule
17
18 module top;
19     intf #(.P_INTF(10)) i();
20     defparam sub_inst.PARAM_2 = 222;
21     sub #(.PARAM_3(3.33), .PARAM_4(`STR)) sub_inst();
22 endmodule
```

Report Format:

```
Scope: top.sub_inst (example.v:21)
-----
#1 PARAM_1 (example.v:8)
  Decompile    = ((1 + 2) + 3)
  Final value  = 6
  Default value = 6
#2 PARAM_2 (example.v:9)
  Decompile    = 2
  Final value  = 222
  Default value = 2
#3 PARAM_3 (example.v:10)
  Decompile    = 3.000000
  Final value  = 3.330000
  Default value = 3.000000
#4 PARAM_4 (example.v:11)
  Decompile    = "string"
  Final value  = NewString
  Default value = string
```

"List Scope Parameter" also supports aggressive options to retrieve parameter information such as target scope, depth, and a list of NPI type.

You can specify a target scope to be traverse. If the target scope is not specified, the VC App traverses design from root.

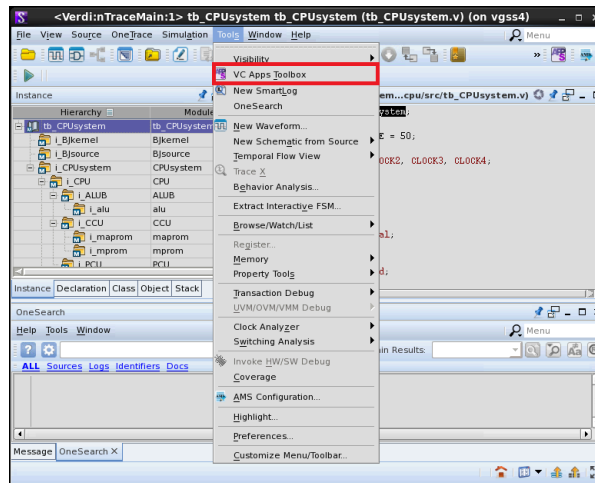
The option "depth" is accompanied by the target scope. depth is used to assign the hierarchy to be traversed. If depth is not specified, the VC App traverses the whole design without level limitation.

The VC App also provides support to report parameter within specified NPI types. For example, you can specify "npiModule npiInterface" as the target NPI types, thus the VC App only reports parameters under modules and interfaces. If the option is not specified, the VC APP reports parameters in all possible scopes.

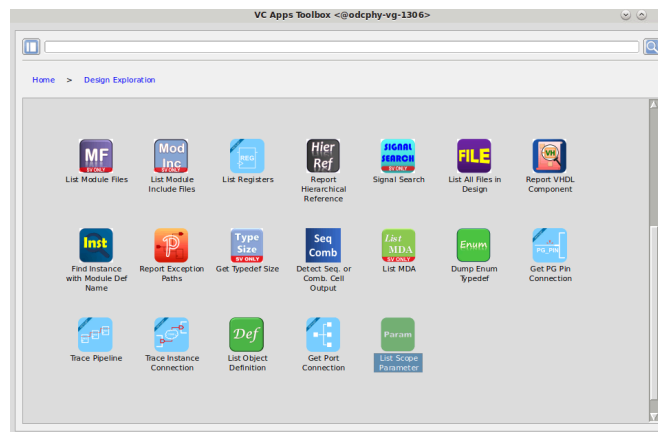
App Usage in Verdi GUI Mode

To use the App in the Verdi GUI mode, perform the following steps:

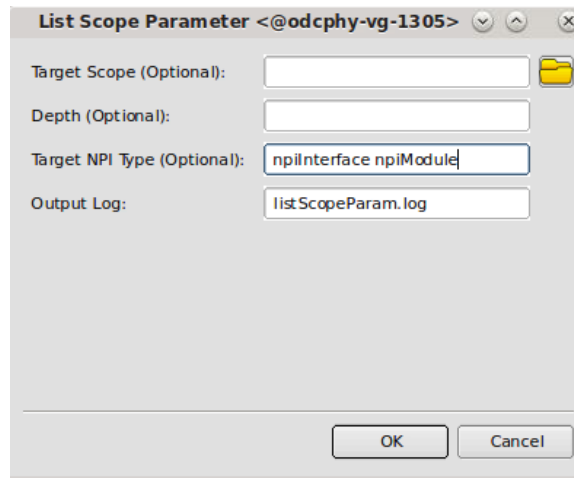
1. Open Verdi with design, select **Tools -> VC Apps Toolbox**.



2. Go to **Design Exploration**, and click the **List Scope Parameter** icon.



3. Fill in the following fields:



- **Target Scope:** The target scope to get parameters. If scope is not specified, traverse from root.
 - **Depth (Optional):** Level to be traversed from the target scope. If depth is not specified, traverse whole design without level limitation.
 - **Target NPI Type (Optional):** Only report parameters within target NPI types, for example, `npiModule`, `npiInterface`. If NPI type is not specified, report parameters within all possible scope types.
 - **Output Log (Optional):** Specify the output log file name. Default log name is `listScopeParam.log`.
4. The output log reports the list of parameter value. Smart Log will display the log file by default..

```
Scope: top.i (example.v:19)
-----
#1 P_INTF (example.v:4)
  Decompile    = 8
  Final value  = 10
  Default value = 8

Scope: top.sub_inst (example.v:21)
-----
#1 PARAM_1 (example.v:8)
  Decompile    = ((1 + 2) + 3)
  Final value  = 6
  Default value = 6
#2 PARAM_2 (example.v:9)
  Decompile    = 2
  Final value  = 222
  Default value = 2
#3 PARAM_3 (example.v:10)
  Decompile    = 3.000000
  Final value  = 3.330000
  Default value = 3.000000
#4 PARAM_4 (example.v:11)
  Decompile    = "string"
  Final value  = NewString
  Default value = string
```

App Usage in Verdi Batch Mode

Script location

`$VERDI_HOME/share/VIA/Apps/Bin/listScopeParam.pl`

Usage

```
listScopeParam.pl <verdi_import_design> \  
[-target_scope <target_scope>] \  
[-depth <depth>] \  
[-target_npi_type <list_of_npi_type>] \  
[-o <output_log>] \  
[-h]
```

Options

- `-target_scope`: (Optional) Specify a target scope to get parameters. If not specified, traverse from root.
- `depth` (Optional): Specify a level limitation to traverse from the target scopes. If not specified, traverse whole design without level limitation. Default value is 0.
- `-o`: Specify the output log file name. Default log name is `listScopeParam.log`.
- `-h`: help information.

Example

```
listScopeParam.pl -f run.f -target_scope "top.i_sub" -  
target_npi_type "npiModule npiInterface"
```

```
listScopeParam.pl -f run.f -target_scope "top.i_sub" -o  
list_parameter.log
```

Report Hierarchical Reference



Description

Report the hierarchical reference signals in the design, with the operator '.'

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/ReportHierRef`

Tcl Entry File

`reportHierRef.tcl`

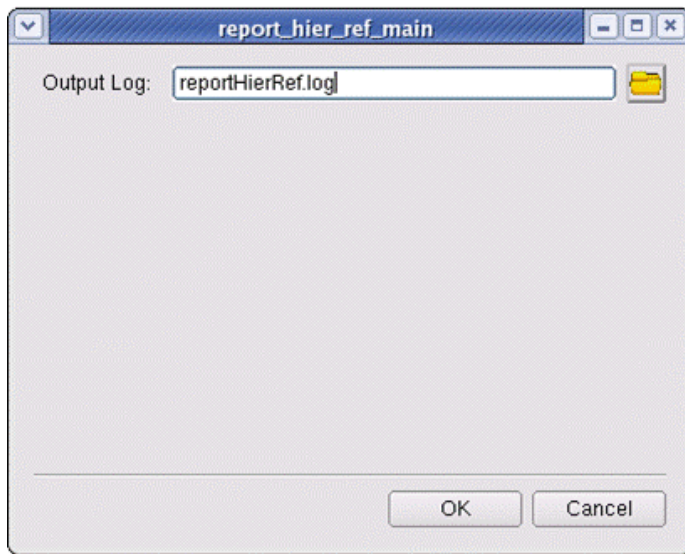
Tcl Procedure Entry

`report_hier_ref_main { output_log }`

Arguments

`output_log:`

Specify the output log file name.



Batch Mode Usage

```
Usage: reportHierRef_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./reportHierRef_batch.pl -f run.f -o reportHierRef.log
```

Limitations

N/A

Example

```
// ----- example.v ----- //  
  
package pkg;  
  bit pa;  
endpackage  
  
interface itf;  
  bit i1;  
endinterface  
  
typedef struct { bit s1; bit s2; } T1;  
  
module top;
```

```
T1 s;
m1 i0();
// 1. package::a
assign pkg::pa = 1;

// 2. top.a.b
assign top.i0.a = 1;

// 3. struct member
assign s.s1 = 1;

// 4. build-in method call
bit total;
byte b[] = { 1, 2, 3, 4 };
assign total = b.sum;

endmodule

module m1( itf i );
  bit a;

  // 5. interface
  assign i.i1 = 1;
endmodule

// ----- reportHierRef.log ----- //
```

```
* VIA App: reportHierRef.tcl
* Report : reportHierRef.log
* Date   : 2013-03-19 16:31:27
*****
```

Module: top (example.v)

```
19: hierarchical reference: top.i0.a
22: hierarchical reference: s.s1
27: hierarchical reference: b.sum
```

Module: m1 (example.v)

```
19: hierarchical reference: top.i0.a
22: hierarchical reference: s.s1
27: hierarchical reference: b.sum
35: hierarchical reference: i.i1
```

Signal Search



Description

Given

1. a list of keywords separated by blank(s),
 - e.g. "TEST.TOP.CPU data rst"
2. target scope,
 - can be full hierarchical name or partial name
 - can be multiple scopes separated by blank(s)
 - empty string means the search of all design scopes
3. search rule: regex or wildcard,
4. output log file name

Result

- Report the signal's full hierarchical name which matches *ALL* the keywords; and those signals should belong to *ONE* of the specified target scopes
- The search be case insensitive

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/SignalSearch`

Tcl Entry File

signalSearch.tcl

Tcl Procedure Entry

```
proc signal_search_main { search_rule keyword_list target_scope output_log }
```

Arguments

search_rule:

“Regex” or “Wildcard”; default is “Regex” (regular expression).

keyword_list:

A list of keywords; use spaces for multiple keywords. For example, “CPU data rst”.

target_scope:

Specify the target scopes; the scope can be a complete string or a keyword with the search rule; use spaces for multiple scopes. For example, “TEST.TOP.CPU TOP2”.

output_log:

Specify the output log file name.

Batch Mode Usage

Usage: `signalSearch_batch.pl` <verdi_import_design> \
[-search_rule <"Regex" | "Wildcard">] \
-keywords <keywords_string> \
-target_scope <target_scopes_string> \
[-o <output_log>] \
[-h]

For example:

```
signalSearch_batch.pl -f run.f -search_rule "Regex" -keywords "cloc  
ccu" -target_scope "alu pcu" -o signalSearch.log  
signalSearch_batch.pl -f run.f -search_rule "Wildcard" -keywords  
"*cloc* *ccu*" -target_scope "*" -o signalSearch.log
```

Limitations

Currently this App won't report signals in VHDL components.

Example

Given the input:

The screenshot shows a 'Signal Search' dialog box with the following fields:

- Search Rule:
- Signal Keywords:
- Target Scope:
- Output Log:

At the bottom right are 'OK' and 'Cancel' buttons.

The output log looks like:

```

1 *****
2 * VIA App: signalSearch.tcl
3 * Report : signalSearch.log
4 * Date   : 2013-10-30 10:57:41
5 *****
6
7 =====
8 Search Rule:  Regex
9 Signal Keywords: wish clk
10 Target Scope: SYSTEM.bridge32_top.bridge
11 =====
12
13 Matched signals:
14
15 0) Signal: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.wishbone_slave.async_reset_as_wbr_flush.clk_in
16    Location: (/slowfs/sp_verify1/regression/whatif/RTL_Verilog/Pci_Bridge/Design/pcl_async_reset_flop.v : 86)
17
18 1) Signal: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.ffios.pcl_clk_sync_inGreyCount
19    Location: (/slowfs/sp_verify1/regression/whatif/RTL_Verilog/Pci_Bridge/Design/pcl_wbw_wbr_ffios.v : 562)
20
21 2) Signal: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.ffios.pcl_clk_inGreyCount
22    Location: (/slowfs/sp_verify1/regression/whatif/RTL_Verilog/Pci_Bridge/Design/pcl_wbw_wbr_ffios.v : 563)
23
24 3) Signal: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.ffios.wbw_ffio_storage.clk_a
25    Location: (/slowfs/sp_verify1/regression/whatif/RTL_Verilog/Pci_Bridge/Design/pcl_wb_tpram.v : 148)
26
27 4) Signal: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.ffios.wbw_ffio_storage.clk_b
28    Location: (/slowfs/sp_verify1/regression/whatif/RTL_Verilog/Pci_Bridge/Design/pcl_wb_tpram.v : 156)
29
30 5) Signal: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.ffios.wbr_ffio_storage.clk_a
31    Location: (/slowfs/sp_verify1/regression/whatif/RTL_Verilog/Pci_Bridge/Design/pcl_wb_tpram.v : 148)
32
33 6) Signal: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.ffios.wbr_ffio_storage.clk_b
34    Location: (/slowfs/sp_verify1/regression/whatif/RTL_Verilog/Pci_Bridge/Design/pcl_wb_tpram.v : 156)
35
36 7) Signal: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.ffios.wbw_ffio_ctrl.wclk_sync_rgrey_minus1
37    Location: (/slowfs/sp_verify1/regression/whatif/RTL_Verilog/Pci_Bridge/Design/pcl_wbw_ffio_control.v : 270)
38
39 8) Signal: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.ffios.wbw_ffio_ctrl.rclk_sync_wgrey_next
40    Location: (/slowfs/sp_verify1/regression/whatif/RTL_Verilog/Pci_Bridge/Design/pcl_wbw_ffio_control.v : 301)
41
42 9) Signal: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.ffios.wbw_ffio_ctrl.wclk_rgrey_minus1
43    Location: (/slowfs/sp_verify1/regression/whatif/RTL_Verilog/Pci_Bridge/Design/pcl_wbw_ffio_control.v : 271)
44
45 10) Signal: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.ffios.wbw_ffio_ctrl.rclk_wgrey_next
46    Location: (/slowfs/sp_verify1/regression/whatif/RTL_Verilog/Pci_Bridge/Design/pcl_wbw_ffio_control.v : 302)

```

List All Files in Design



Description

List all files in design import.

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/ListAllFilesInDesign`

Tcl Entry File

`listAllFilesInDesign.tcl`

Tcl Procedure Entry

`list_all_files_main { output_log }`

Arguments

`output_log:`

Specify the output log file name.

Batch Mode Usage

```
Usage: listAllFilesInDesign_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-h]
```

Chapter 3: Design Exploration

List All Files in Design

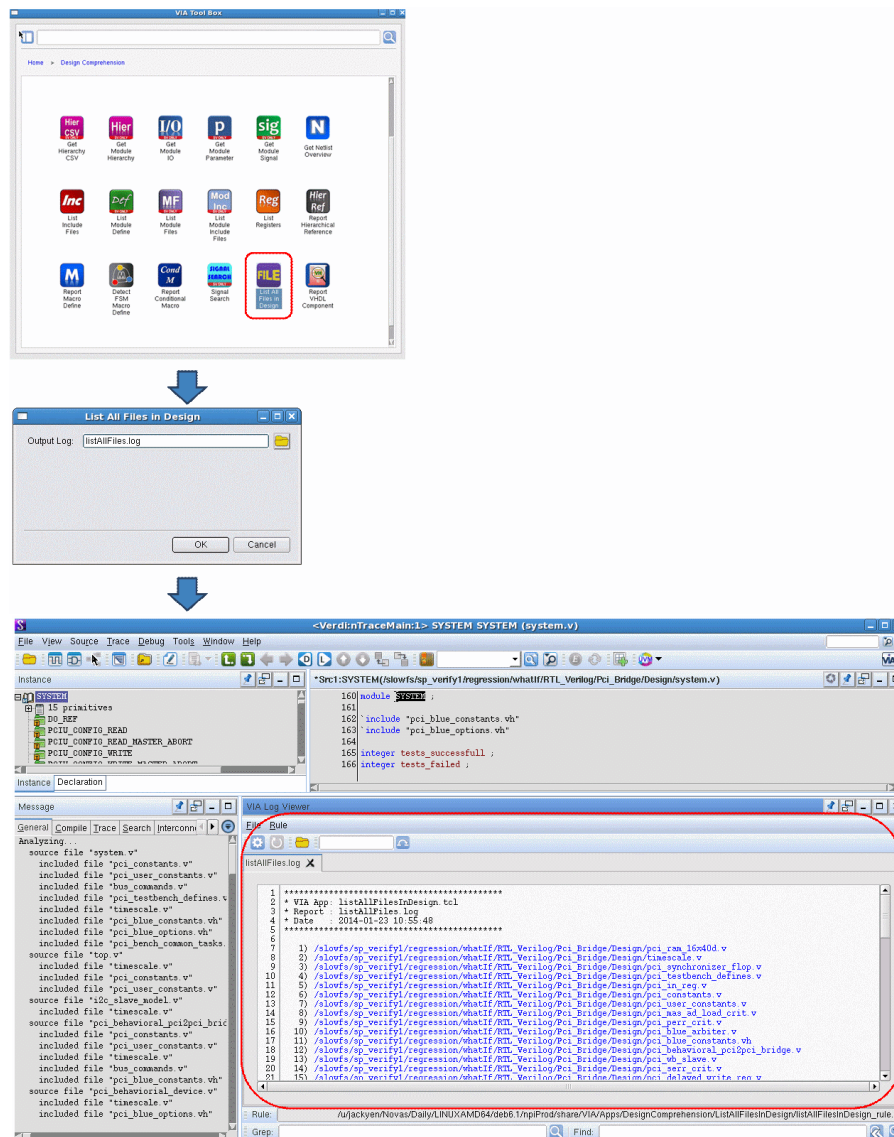
For example:

```
./listAllFilesInDesign_batch.pl -f run.f -o listAllFilesInDesign.log
```

Limitations

N/A

Example



Report VHDL Component



Description

Traverse and report VHDL components of current design.

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/ReportVhdlComp/`

Tcl Entry File

ReportVhdlComp.tcl

Tcl Procedure Entry

```
report_vhdl_component { targetScopeName outputLog }
```

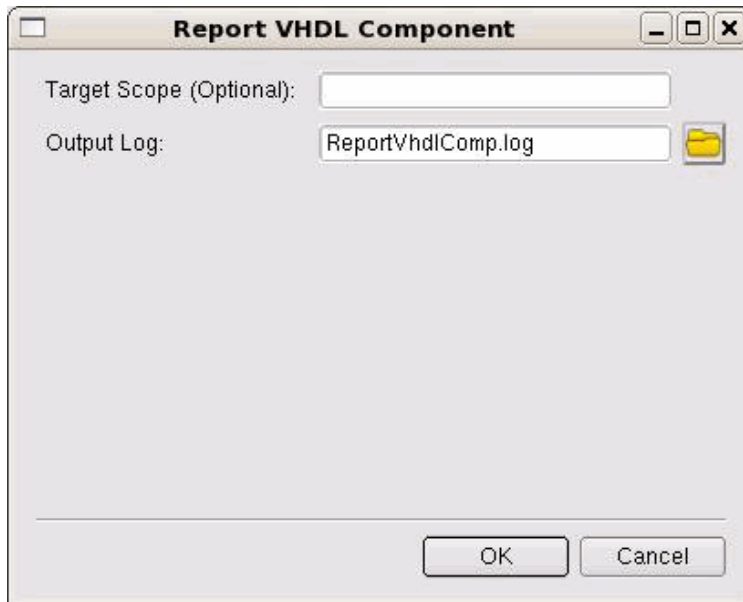
Arguments

Target Scope (Optional):

Target serial signal full name

Output Log:

Specify the output log file name



Batch Mode Usage

```
Usage: ReportVhdlComp_batch.pl <verdi_import_design> \  
      [ -root <target_scope_name> ] \  
      [-o <output_log>] \  
      [-h]
```

For example:

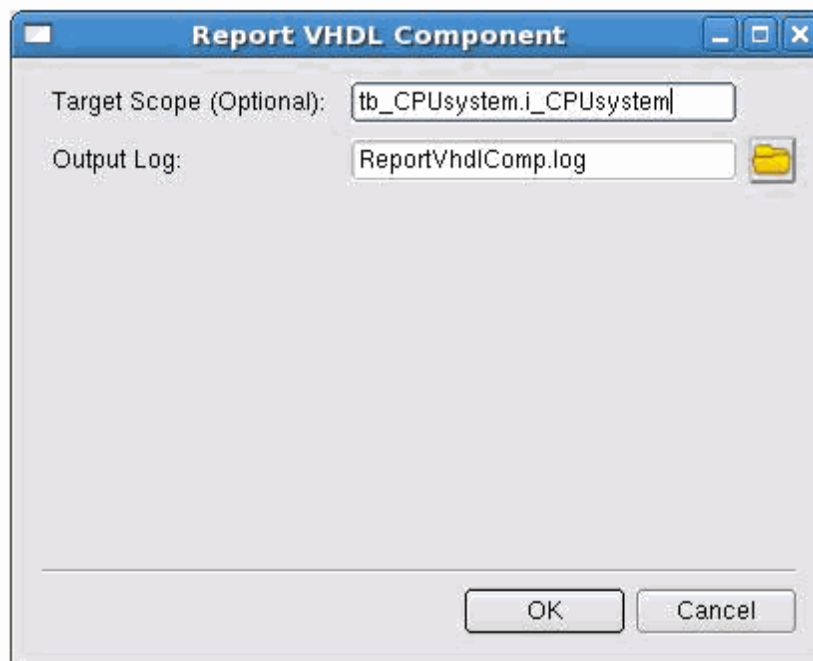
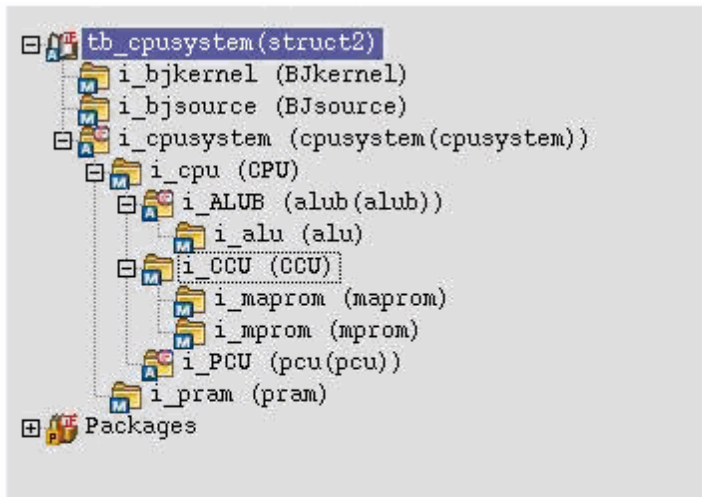
```
./ReportVhdlComp_batch.pl -top tb_CPUsystem -lib work.lib++ -o  
ReportVhdlComp.log  
-root tb_CPUsystem.i_CPUsystem
```

Limitations

N/A

Example

```
// ----- example MIXED design ----- //
```



```
// ----- ReportVhdlComp.log ----- //
```

```
*****
```

```
* VIA App: ReportVhdlComp.tcl
```

```
* Report: ReportVhdlComp.log
```

```
* Date : 2013-12-20 18:19:56
```

```
*****
```

```
Target scope: tb_CPUsystem.i_CPUsystem
```

```
1: tb_CPUsystem.i_CPUsystem
```

```
2: tb_CPUsystem.i_CPUsystem.i_cpu.i_PCU
```

```
3: tb_CPUsystem.i_CPUsystem.i_cpu.i_ALUB
```

Find Instance with Module Def Name



Description

Gets the module instance full hierarchical name by wildcard search for module define name.

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/FindInstDefWild`

Tcl Entry File

`findInstDefWild.tcl`

Tcl Procedure Entry

`find_inst_mod_def_wildcard { pattern output_log }`

Arguments

pattern:

Specify the wildcard pattern of module define name.

Use space to specify more than one module; e.g.

`"*DFF* AOI*"`

output_log:

Specify the output log file name.

Batch Mode Usage

```
Usage: findInstDefWild_batch.pl <verdi_import_design> \  
      -pattern <Wildcard_pattern_of_module_define_name> \  
      [-outlog <output_log>] \  
      [-h]
```

For example:

```
findInstDefWild_batch.pl -f run.f -pattern "*DFF* AOI*" -outlog  
findInstDefWild.log
```

Limitations

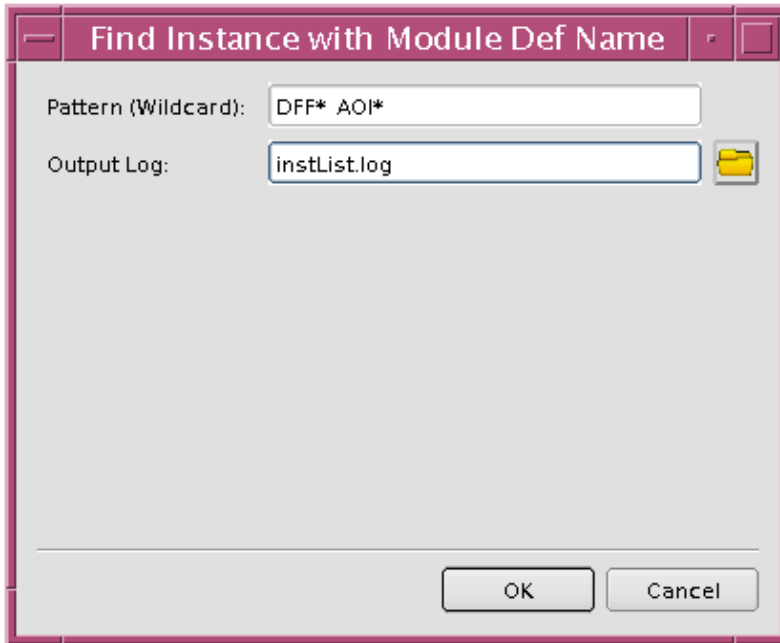
N/A

Example

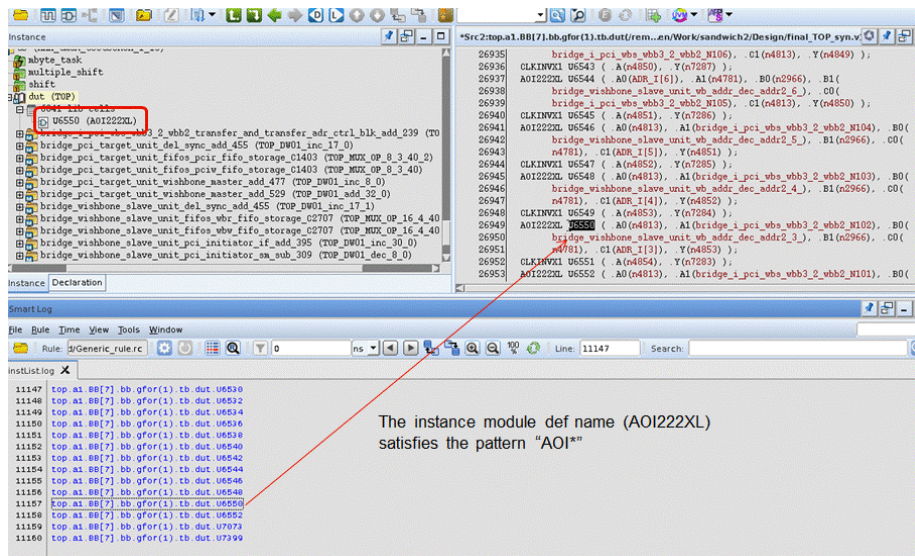
Double click the icon of “Find Instance with Module Def Name” App.



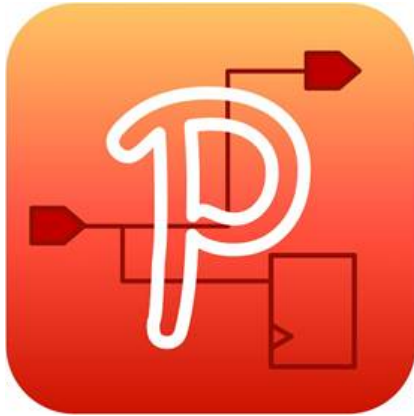
Fill in the inputs.



The results are dumped to the log `instList.log`. By default, Smart Log will also display the log file.



Report Exception Paths



Description

Reports all the exception paths.

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/ReportExceptionPaths`

Tcl Entry File

`reportExceptionPaths.tcl`

Tcl Procedure Entry

```
report_exception_path_main { exception_file clock_report output_log }
```

Arguments

PT exception file:

Specify the file name dump from primeTime `report_exceptions -nosplit` command.

`transform_exceptions -flatten` command is required before `report_exceptions -nosplit` command in PrimeTime.

File format is as shown:

```
From Through To Setup Hold Ignored
```

```
-----
I_ORCA_TOP/I_SDRAM_IF/sd_DQ_en_reg[0]/CP
sdram_DQ_iopad_0/OEN * FALSE FALSE

I_ORCA_TOP/\* /sd_DQ_en_reg[1]/CP
sdram_DQ_iopad_1/OEN * FALSE FALSE

I_ORCA_TOP/I_SDRAM_IF/sd_DQ_en_reg[2]/CP
sdram_DQ_iopad_2/OEN * FALSE FALSE

I_ORCA_TOP/I_SDRAM_IF/sd_DQ_en_reg[3]/CP
sdram_DQ_iopad_3/OEN * FALSE FALSE
1
```

PT clock report:

Specify the file name dump from primeTime `report_clock -nosplit` command.

File format is as shown:

...	Period	Waveform	Attrs
Clock Sources			

PCI_CLK	15.000	{0 7.5}	p
{pclk}			
SD_DDR_CLK	7.500	{0 3.75}	p, G
{sd_ck}			

Output log:

Specify the output log name.

Batch Mode Usage

```
Usage: reportExceptionPath_batch.pl <verdi_import_design> \
      -excep < exception_file> \
      [-clk <clock_report>] \
      [-o <output_log>] \
      [-h]
```

For example:

```
./reportExceptionPath_batch.pl -sv example.v -excep
"exception.log" -clk "clock.log" -o "reportExceptionPath.log"
```

Example

In PrimeTime after setting false paths or multicycle paths, use

```
pt_shell> transform_exceptions -flatten
```

to flatten the exceptions.

Then use,

```
pt_shell>report_exceptions -nosplit > exception_file.log
```

to dump exceptions report to exception_file.log, as an input of this App.

Use

```
pt_shell>report_clock -nosplit > clock.log
```

to dump exceptions report to clock.log, as an input of this App.

This figure shows the PT exception file: exception_file.log

```
*****
Report : exceptions
Design : system
Version: K-2015.06-SP3
Date   : Wed Nov 30 11:49:30 2016
*****

Reasons : f invalid start points
          t invalid end points
          p non-existent paths
          o overridden paths

From      Through      To      Setup      Hold      Ignored
-----
i_cpu/i_PCU/ALU      *      myClock      FALSE      FALSE
1
```

The following figure shows the PT clock report: exception_file.log.

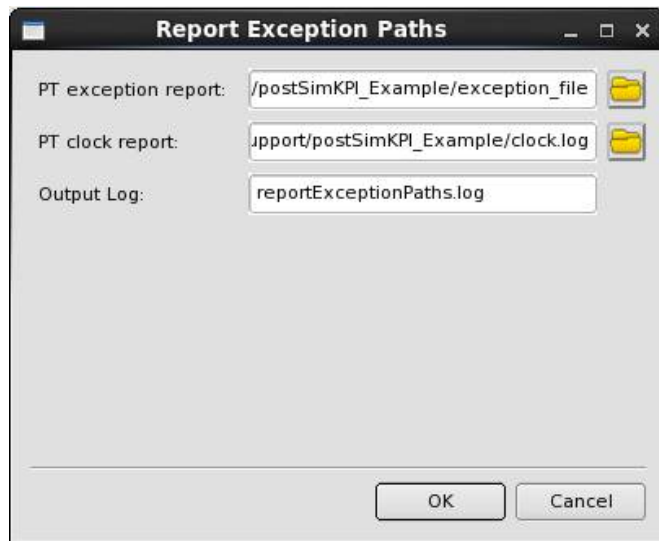
```
*****
Report : clock
Design : system/i cpu
Version: K-2015.06-SP3
Date   : Wed Nov 30 11:49:17 2016
*****

Attributes:
p - Propagated clock
G - Generated clock
I - Inactive clock

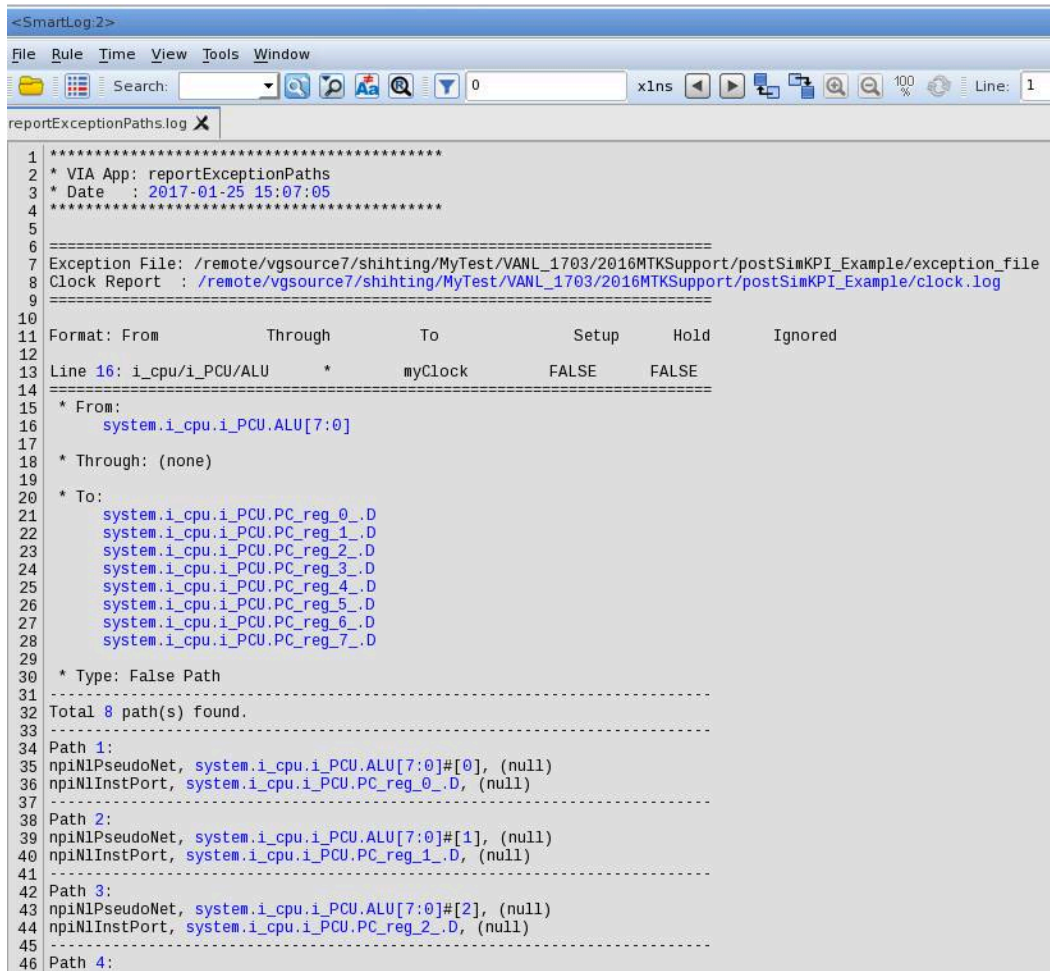
Clock      Period      Waveform      Attrs      Sources
-----
myClock      15.000      {0 7.5}      p      {i_PCU/S1}

Generated      Master      Generated      Master      Waveform
Clock      Source      Source      Clock      Modification
-----
```

Report Exception Paths.



Output log:



```

1 *****
2 * VIA App: reportExceptionPaths
3 * Date : 2017-01-25 15:07:05
4 *****
5
6 =====
7 Exception File: /remote/vgsource7/shiting/MyTest/VANL_1703/2016MTKSupport/postSimKPI_Example/exception_file
8 Clock Report : /remote/vgsource7/shiting/MyTest/VANL_1703/2016MTKSupport/postSimKPI_Example/clock.log
9 =====
10
11 Format: From          Through          To          Setup          Hold          Ignored
12
13 Line 16: i_cpu/i_PCU/ALU      *          myClock      FALSE          FALSE
14
15 * From:
16   system.i_cpu.i_PCU.ALU[7:0]
17
18 * Through: (none)
19
20 * To:
21   system.i_cpu.i_PCU.PC_reg_0_.D
22   system.i_cpu.i_PCU.PC_reg_1_.D
23   system.i_cpu.i_PCU.PC_reg_2_.D
24   system.i_cpu.i_PCU.PC_reg_3_.D
25   system.i_cpu.i_PCU.PC_reg_4_.D
26   system.i_cpu.i_PCU.PC_reg_5_.D
27   system.i_cpu.i_PCU.PC_reg_6_.D
28   system.i_cpu.i_PCU.PC_reg_7_.D
29
30 * Type: False Path
31
32 Total 8 path(s) found.
33
34 Path 1:
35 npINIPseudoNet, system.i_cpu.i_PCU.ALU[7:0]#[0], (null)
36 npINInstPort, system.i_cpu.i_PCU.PC_reg_0_.D, (null)
37
38 Path 2:
39 npINIPseudoNet, system.i_cpu.i_PCU.ALU[7:0]#[1], (null)
40 npINInstPort, system.i_cpu.i_PCU.PC_reg_1_.D, (null)
41
42 Path 3:
43 npINIPseudoNet, system.i_cpu.i_PCU.ALU[7:0]#[2], (null)
44 npINInstPort, system.i_cpu.i_PCU.PC_reg_2_.D, (null)
45
46 Path 4:
  
```

4

Design Validation

This section describes the following topics:

- [Detect Dangling Port](#)
- [Detect Assign Bit Width Mismatch](#)
- [Detect Constant Port](#)
- [Detect Open Port](#)
- [Detect Tri-State](#)
- [Check Module Registered Outputs](#)
- [Check Module Registered Inputs](#)
- [Check Module Port Declaration Using Modport](#)
- [Check Module Port Connections Using Modport](#)
- [Signal Existence Check in Design](#)
- [Module Array Instantiation Check](#)

Detect Dangling Port

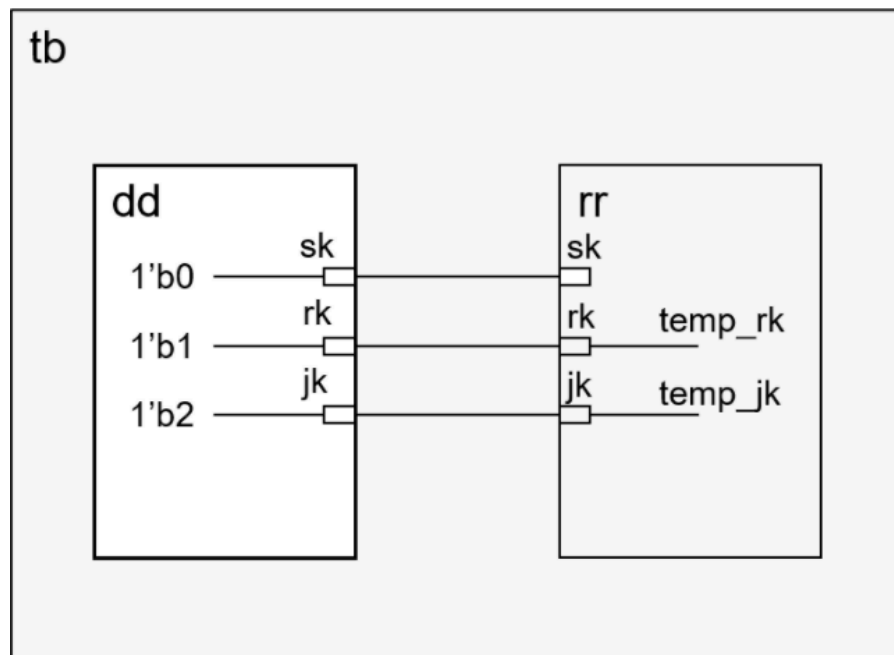


Description

This App detects dangling port which is connected to the specified module instance.

For example, by given module "tb.dd", you can find port "tb.dd.sk" is connected to "tb.rr.sk". However, there is no load for "tb.rr.sk", which can be seen as dangling. Therefore, it is defined that the port "tb.dd.sk" is "left dangling" at "tb.rr.sk". Although "tb.dd.sk" seems having connections to other modules, but it is actually dangling at some point. This App can help you find the dangling port and the dangling point.

On the other hand, "tb.rr.rk" is connected to a net "temp_rk" by assignment. Even the net "temp_rk" is floating at its load, "tb.dd.rk" is not left dangling. If you want to treat a port, with assignment connections to all floating nets, as dangling, set "Report port with floating net" to True, or -float in perl.

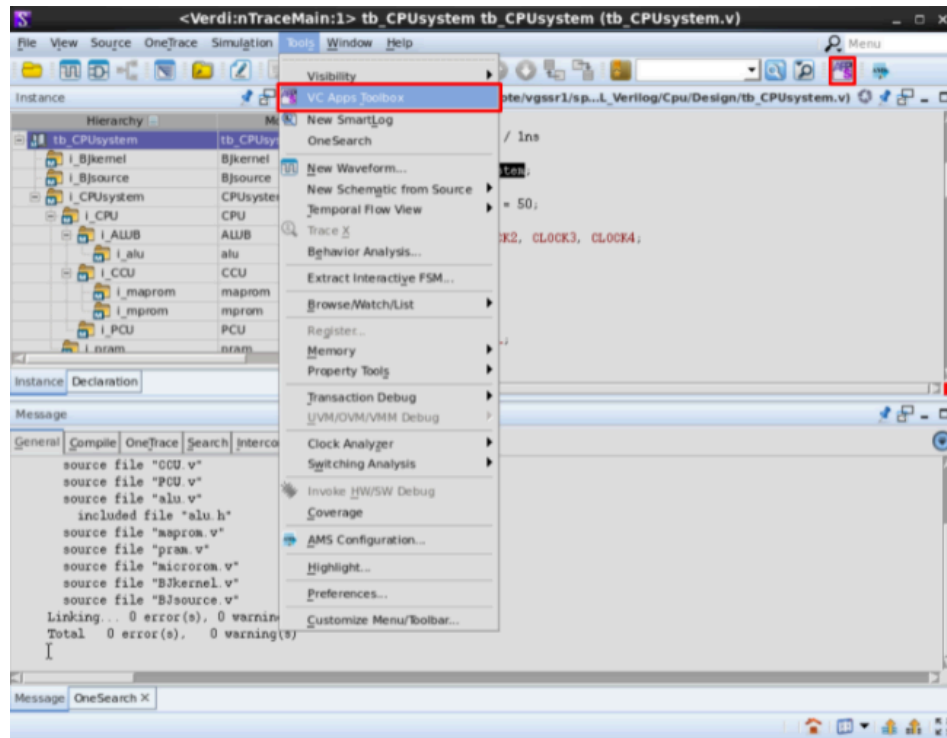


Limitation

None

How to Open the App in VC Apps Toolbox

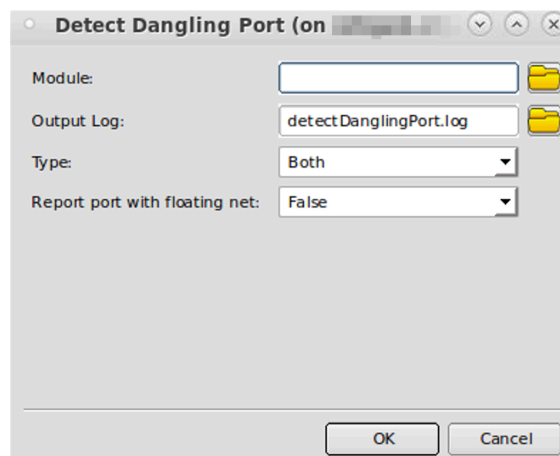
1. Open Verdi with design, select **Tools -> VC Apps Toolbox**;
or click the **VC Apps Toolbox** icon directly.



2. Go to **Design Validation**, click the **Detect Dangling Port** icon.

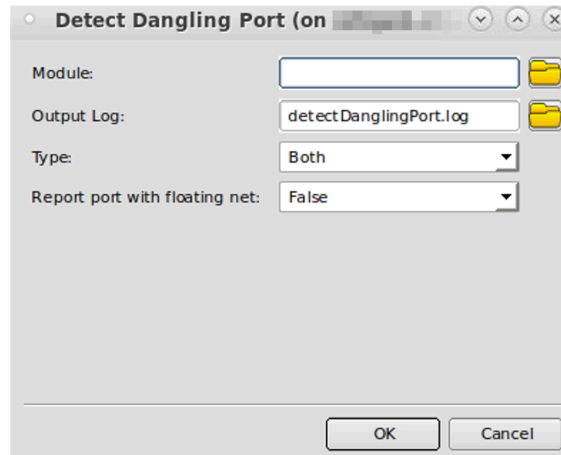


The App opens as shown in the following figure.



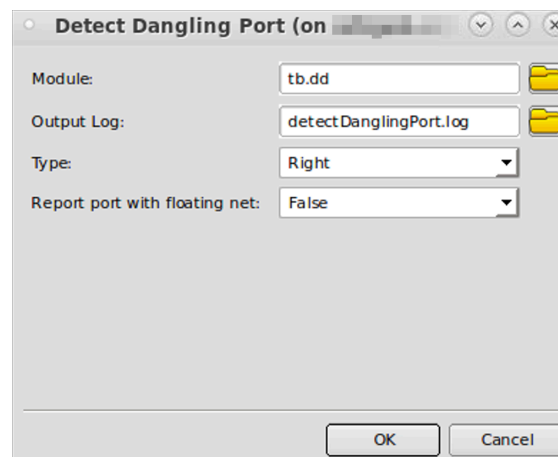
App Usage in Verdi GUI Mode

1. Open App in VC Apps ToolBox.



2. Fill in the following fields:

- **Module:** Specify the target module full hierarchical name.
- **Output Log:** Specify the output log file name.
- **Type:** Specify the dangling type. The default value is **Both**.
- **Report port with floating net:** Specify if it also reports port connected with floating net by assignment.



3. Click **OK**.

SmartLog displays the output log, with the information of the dangling port and where it is dangling.

```
detectDanglingPort.log X
1 *****
2 * VIA App: detectDanglingPort.tcl
3 * Report : detectDanglingPort.log
4 * Date : 2020-07-01 19:02:24
5 *****
6
7 Module: tb.dd (dut)
8 -----
9 1) Output: tb.dd.rk
10    1) Dangling at: tb.ee.rk
11 2) Output: tb.dd.sk
12    1) Dangling at: tb.rr.sk
```

App Usage in Verdi Batch Mode

Script location

`$VERDI_HOME/share/VIA/Apps/Bin/detectDanglingPort.pl`

Usage

```
detectDanglingPort.pl <design_import>
-module <inst_full_name>
[-o <output_log>]
[-left|-right]
```

Options

- `-module`: Specify the target module full hierarchical name.
- `-o`: Specify the output log name. The default value is *detectDanglingPort.log*.
- `-left|-right` (Optional): Set to detect left or right dangling. The default value is **Both**.
- `-float`(Optional): Set to report port connected with the floating net by assignment.

Example

```
/> detectDanglingPort.pl -sv top.v -module "tb.dd" -left
```

Detect Assign Bit Width Mismatch



Description

Detect whether any bit width mismatch occurs in:

- (1) the LHS and RHS of assignment (cont-assign, assignment, parameter assign)
- (2) the high-conn and low-conn of port

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignValidation/DetectAssignBitWidthMismatch`

Tcl Entry File

`detectAssignBitWidthMismatch.tcl`

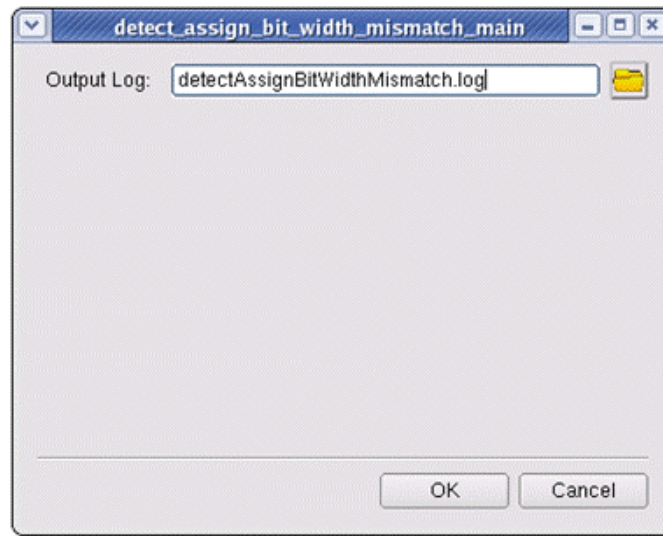
Tcl Procedure Entry

`detect_assign_bit_width_mismatch_main { output_log }`

Arguments

`output_log`:

Specify the output log file name.



Batch Mode Usage

```
Usage: detectAssignBitWidthMismatch_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./detectAssignBitWidthMismatch_batch.pl -f run.f -o  
detectAssignBitWidthMismatch.log
```

Limitations

Currently this App won't report the results in VHDL components.

Example

```
// ----- example.v ----- //  
  
module TOP;  
  
    wire a, z;  
    M m(a, z);  
  
endmodule  
  
module M(a, z);  
  
    input [7:0] a;  
    output z;
```

```
    assign z = ~a;

endmodule

// ----- detectAssignBitWidthMismatch.log ----- //

*****
* VIA App: detectAssignBitWidthMismatch.tcl
* Report : detectAssignBitWidthMismatch.log
* Date   : 2013-03-20 13:33:41
*****

1) assign z = (~a) (Lhs = 1, RhS = 8)
   ( example.v : 13 )
2) .a( a ) (High-Conn = 1, Low-Conn = 8)
   ( example.v : 4 )
```

Detect Constant Port



Description

Detect whether any high-conn of the module instance is constant.

Tcl Script Location

\$VERDI_HOME/share/VIA/Apps/DesignValidation/DetectConstPort

Tcl Entry File

detectConstPort.tcl

Tcl Procedure Entry

```
detect_const_port_main { target_scope target_scope_file depth modules modules_file  
exclude_modules exclude_modules_file output_log {partial_const "FALSE"} }
```

Arguments

target_scope:

Specify the full name of the target scope or a list of scopes.

target_scope_file:

A file contains the list of target scopes; if both **target_scope** and **target_scope_file** are not specified, all module instances are handled.

depth:

Specify depth to traverse; if not specified, traverse without depth limitation.

modules:

Specify the module name, can be exact name or wildcard; can be hierarchical name or module definition name; can be multiple modules separated by space.

modules_file:

A file contains the list of modules; if both **modules** and **modules_file** are not specified, all modules are handled.

exclude_modules:

Specify the module name not reported.

exclude_modules_file:

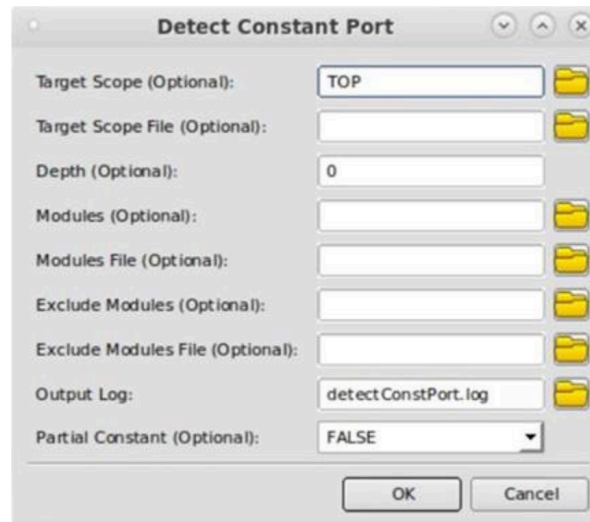
A file contains the list of modules not reported; if both **exclude_modules** and **exclude_modules_file** are not specified, all modules are handled.

output_log:

Specify the output log file name.

partial_const:

<"TRUE" | "FALSE"> to specify whether also report partial constants. Default: "FALSE".



Batch Mode Usage

```
Usage: detectConstPort_batch.pl <verdi_import_design> \  
  
[-target_scope "<target_instance_fullnames>"] \  
[-target_scope_file "<file_of_scope_list>"] \  
[-depth "<depth_to_traverse>"] \  
[-modules "<target_module_names>"] \  
[-modules_file "<file_of_module_names>"] \  
[-exclude_modules "<exclude_module_names>"] \  
[-exclude_modules_file "<file_of_exclude_module_names>"] \  
[-include_partial_const] \  
[-o <output_log>] \  
[-h]
```

For example:

```
./detectConstPort_batch.pl -f run.f -o detectConstPort.log  
./detectConstPort.pl -f run.f -modules "system.TOP.* ALU*" \  
./detectConstPort.pl -f run.f -include_partial_const
```

Limitations

Currently this App does not report the results in VHDL components.

If a module is both included and excluded, then the module is not reported.

Example

```
// ----- example.v ----- //
```

```

module TOP;

    wire b;
    wire [0:3] c;
    wire [2:7] d;
    parameter [1:0] e = 4;
    wire z;

    function [1:0] cpy (input [1:0] data);
        cpy = data + data;
    endfunction

    M m (.a( {b, c, 5'h10, d} ), .h( cpy(e)+6'h11 ), .z(1'b0));

endmodule

module M (a, z, h);

    input wire [7:0] h;
    input wire [15:0] a;
    output wire z;

    assign z = ~a;

endmodule

// ----- detectConstPort.log ----- //

*****
* VIA App                : Detect Constant Port
* Report                 : detectConstPort.log
* Target Scope           : TOP
* Target Scope File      :
* Depth                  : 0
* Modules                :
* Modules File           :
* Exclude Modules        :
* Exclude Modules File   :
* Partial Const          : FALSE
* Date                   : Fri Dec 23 11:53:17 2022
*****
=====
Target Scopes: TOP
Target Modules: ALL
Target Exclude Modules: NONE
=====
Module: TOP.m (M)
-----
    1) z (1'b0)
    2) h ((cpy(e) + 'h11))

```

Detect Open Port



Description

Detect whether any high-conn of the module instance is open (floating).

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignValidation/DetectOpenPort`

Tcl Entry File

`detectOpenPort.tcl`

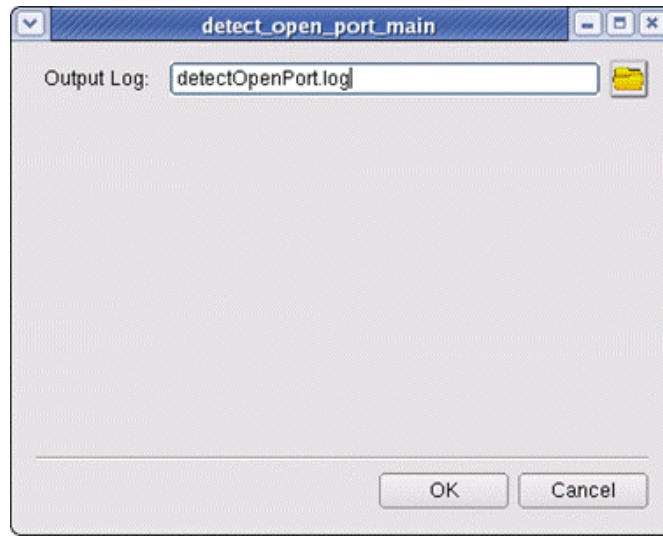
Tcl Procedure Entry

`detect_open_port_main { output_log }`

Arguments

`output_log`:

Specify the output log file name.



Batch Mode Usage

```
Usage: detectOpenPort_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./detectOpenPort_batch.pl -f run.f -o detectOpenPort.log
```

Limitations

Currently this App won't report the results in VHDL components.

Example

```
// ----- example.v ----- //  
  
module TOP;  
  
    wire b;  
    wire [0:3] c;  
    wire [2:7] d;  
    wire z;  
  
    M m (.a( {b, c, 5'h10, d} ), .z(z));  
  
endmodule  
  
module M (a, z, h);
```

```
input wire [7:0] h;
input wire [15:0] a;
output wire z;

assign z = ~a;

endmodule

// ----- detectOpenPort.log ----- //

*****
* VIA App: detectOpenPort.tcl
* Report : detectOpenPort.log
* Date   : 2013-03-20 13:34:23
*****

Module: TOP.m (M)
-----
1) h
```

Detect Tri-State



Description

It detects any inferred tri-state buffer in the design.

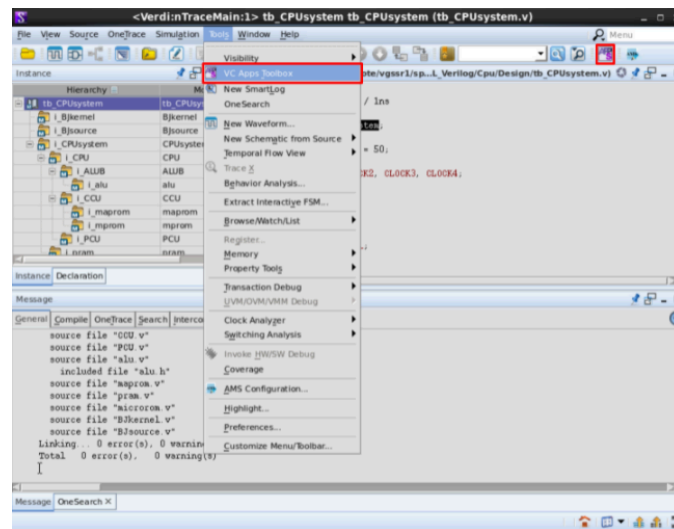
This App traverses the design hierarchy of all the instances, and finds the filename and line number of the detected tri-state buffers.

Limitation

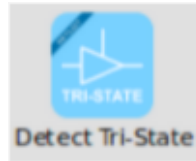
None

How to Open the App in VC Apps Toolbox

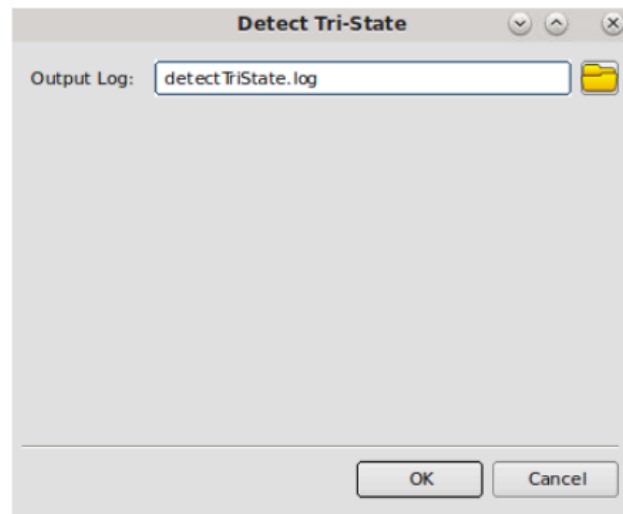
1. Open Verdi with design, select **Tools -> VC Apps Toolbox**;
or click the **VC Apps Toolbox** icon directly.



2. Go to **Design Validation**, click the **Detect Tri-State** icon.

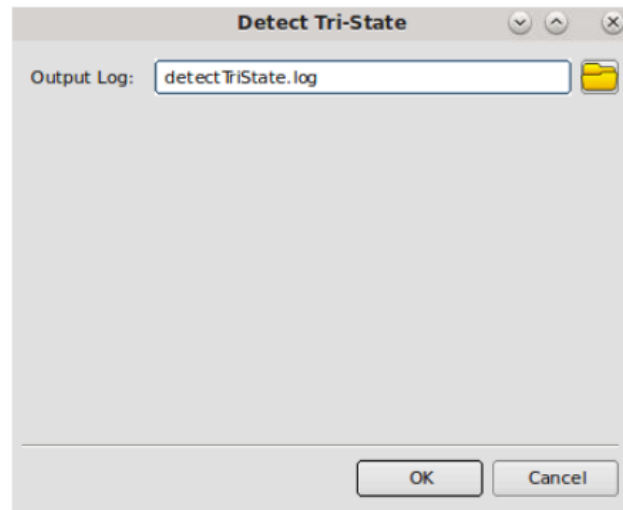


The App opens as shown in the following figure:



App Usage in Verdi GUI Mode

1. Open App in VC Apps ToolBox.



2. On the **Detect Tri-State** dialog, specify the output log file name.

3. Click **OK**.

SmartLog displays the output log of tri-state buffers detected with its filename and line number.

```
detectTriState.log X
1 *****
2 * VIA App: detectTriState.tcl
3 * Report : detectTriState.log
4 * Date   : 2021-11-22 17:32:34
5 *****
6
7 Module: top.u_down_counter_1
8 -----
9      1) (example.v : 25 : 25)
10
11 Module: top.u_up_counter_1
12 -----
13      1) (example.v : 13 : 13)
14
```

App Usage in Verdi Batch Mode

Script location

\$VERDI_HOME/share/VIA/Apps/Bin/detectTriState.pl

Usage

```
detectTriState.pl <design_import>
[-o <output_log>]
```

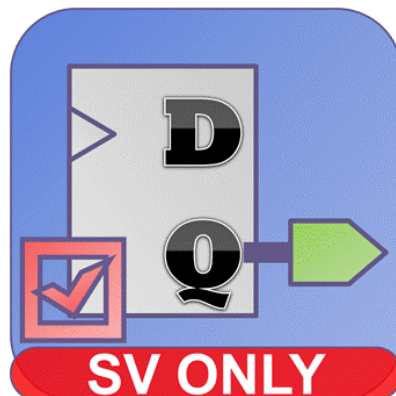
Options

-o: Specify the output log name. The default value is detectTriState.log.

Example

```
/> detectTriState.pl -sv example.v -o detectTriState.log
```

Check Module Registered Outputs



Description

Given

- Output log file name.
- File name contains the list of module names to be checked. Each line of file contains one module name.
 - Optional argument, all the modules will be checked whether the file name is not given.

Result

- Report violation(s) whether there is any module output that is not directly connected to registered outputs.
 - Combinational logics or assignments in front of outputs are not allowed.

Note

- Schematic configuration will be changed.
 - schSetPreference -detailRTL on -detailMux on -detailLevel 99999 -recogFSM off -expandGenBlock on

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignValidation/CheckModRegOut`

Tcl Entry File

`checkModRegOut.tcl`

Tcl Procedure Entry

```
proc check_mod_reg_out_main { outFile { modListFile "" } }
```

Arguments

`outFile:`

Specify the output log file name, default is "coding_rule_check.rpt".

modListFile (Optional):

Specify the file name that contains the list of module names to be checked. Each line of file contains one module name. All the modules will be checked to confirm whether the file name is not given.

Batch Mode Usage

Usage: `checkModRegOut_batch.pl` <verdi_import_design> \
[-o <output_log>] \
[-mod_list <mod_list_file>] \
[-h]

For example:

```
checkModRegOut_batch.pl -f run.f
checkModRegOut_batch.pl -f run.f -o coding_rule_check.rpt
checkModRegOut_batch.pl -f run.f -o coding_rule_check.rpt
-mod_list modlist
```

Limitations

Currently this App won't report violations for VHDL components.

Example

[demo.sv](#)

1	module demo_top (out, in, clk);
2	output out;
3	input clk;
4	input in;
5	sub1 sub1 (.out(out), .in(in), .clk(clk));
6	endmodule
7	
8	module sub1 (out, in, clk);
9	output out; // out must be a registered output
10	input clk;
11	input in;
12	
12	reg vld, tmp_out;
14	wire out = vld & tmp_out;
15	
16	always @(posedge clk)
17	tmp_out = in + 1;
18	endmodule

[demo.list](#)

1	sub1
---	------

Given the input:

Check Module Registered Outputs

Output Log:

coding_rule_check.rpt

Module List File (Optional):

demo.list

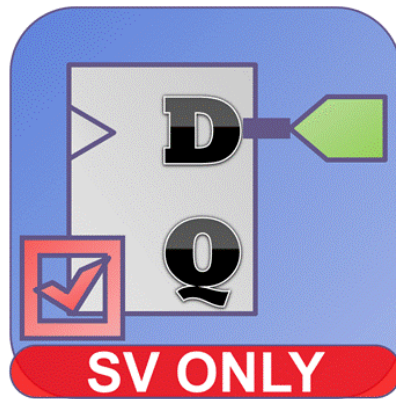
OK

Cancel

The output log looks like:

1	Report file = coding_rule_check.rpt
2	
3	Building design information...
4	Design information is built successfully.
5	
6	Checking coding rule...
7	1 violation(s) detected.
8	
9	/demo/CheckModRegOut/demo.sv(14) : Error : output signal "demo_top.sub1.out" is not directly connected to a registered out.

Check Module Registered Inputs



Description

Given

- Output log file name
- File name contains the list of module names to be checked. Each line of file contains one module name.
 - Optional argument, all the modules will be checked to confirm whether the file name is given.

Result

- Report violation(s) whether there is any module input that is not directly latched by flop-flops.
 - Combinational logics or assignments in front of inputs are not allowed.
 - Input signals connected to clock port will be ignored.

Note

- Schematic configuration will be changed.
 - schSetPreference -detailRTL on -detailMux on -detailLevel 99999 -recogFSM off -expandGenBlock on

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignValidation/CheckModRegIn`

Tcl Entry File

checkModRegIn.tcl

Tcl Procedure Entry

```
proc check_mod_reg_in_main { outFile { modListFile "" } }
```

Arguments

outFile:

Specify the output log file name, default is "coding_rule_check.rpt".

modListFile (Optional):

Specify the file name that contains the list of module names to be checked. Each line of file contains one module name. All the modules will be checked to confirm whether the file name is given.

Batch Mode Usage

```
Usage: checkModRegIn_batch.pl <verdi_import_design> \
      [-o <output_log>] \
      [-mod_list <mod_list_file>] \
      [-h]
```

For example:

```
checkModRegIn_batch.pl -f run.f
checkModRegIn_batch.pl -f run.f -o coding_rule_check.rpt
checkModRegIn_batch.pl -f run.f -o coding_rule_check.rpt
-mod_list modlist
```

Limitations

Currently this App won't report violations for VHDL components.

Example

[demo.sv](#)

1	module demo_top (out, in, clk);
2	output out;
3	input clk;
4	input in;
5	sub1 sub1 (.out(out), .in(in), .clk(clk));
6	endmodule
7	
8	module sub1 (out, in, clk);
9	output out; // out must be a registered output
10	input clk;
11	input in;
12	
12	reg vld, tmp_out;
14	wire out = vld & tmp_out;
15	
16	always @(posedge clk)
17	tmp_out = in + 1;
18	endmodule

[demo.list](#)

1	sub1
---	------

Given the input:



The output log looks like:

1	Report file = coding_rule_check.rpt
2	
3	Building design information...
4	Design information is built successfully.
5	
6	Checking coding rule...
7	1 violation(s) detected.
8	
9	/demo/CheckModRegOut/demo.sv(14) : Error : output signal "demo_top.sub1.out" is not directly connected to a registered out.

Check Module Port Declaration Using Modport



Description

Given

- Output log file name.
- File name that contains the list of module names to be checked. Each line of file contains one module name.
 - Optional argument, all the modules will be checked to confirm whether the file name is given.

Result

- Report violation(s) whether there is module port declaration using modport.
 - Generic interface is not allowed.

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignValidation/CheckModPortDeclModport`

Tcl Entry File

`checkModPortDeclModport.tcl`

Tcl Procedure Entry

```
proc check_mod_port_decl_modport_main { outFile { modListFile "" } }
```

Arguments

outFile:

Specify the output log file name, default is "coding_rule_check.rpt".

modListFile (Optional):

Specify the file name that contains the list of module names to be checked. Each line of file contains one module name. All the modules will be checked to confirm whether the file name is given.

Batch Mode Usage

```
Usage: checkModPortDeclModport_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-mod_list <mod_list_file>] \  
      [-h]
```

For example:

```
checkModPortDeclModport_batch.pl -f run.f  
checkModPortDeclModport_batch.pl -f run.f  
-o coding_rule_check.rpt  
checkModPortDeclModport_batch.pl -f run.f  
-o coding_rule_check.rpt -mod_list modlist
```

Limitations

Currently this App won't report signals in VHDL components.

Example

[demo.sv](#)

```

1  module pcs (pcs_mac_intf.from_mode pcs_from,
2      input logic [7:0] data_in,
3      output logic [7:0] data_out);
4      assign data_out = pcs_from.pcs_mac_axibus_intf;
5      assign pcs_from.pcs_mac_handshake_intf = data_in;
6  endmodule
7
8  module mac (pcs_mac_intf.to_mode mac_to,
9      input logic [7:0] data_in,
10     output logic [7:0] data_out);
11     assign data_out = mac_to.pcs_mac_handshake_intf;
12     assign mac_to.pcs_mac_axibus_intf = data_in;
13 endmodule
14
15 interface pcs_mac_intf;
16     logic [7:0] pcs_mac_axibus_intf;
17     logic [7:0] pcs_mac_handshake_intf;
18
19     modport from_mode (output pcs_mac_handshake_intf, input pcs_mac_axibus_intf);
20     modport to_mode (input pcs_mac_handshake_intf, output pcs_mac_axibus_intf);
21 endinterface
22
23 // ports should be declared using modport
24 module mod_err (input clk,
25     input logic [7:0] data_in,
26     output logic [7:0] data_out);
27     pcs_mac_intf test_intf();
28     always @(posedge clk)
29         data_out = data_in;
30 endmodule
31
32 // ports should be declared using modport, generic interface is not allowed
33 module mod_err2(interface itf/*generic*/, simple_itf sitf);
34 endmodule
35
36 interface automatic simple_itf;
37 endinterface
38
39 interface intf(simple_itf sbidf);
40 endinterface

```

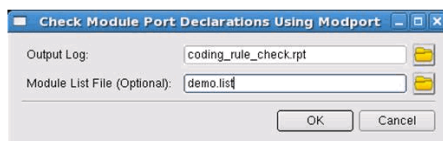
[demo.list](#)

```

1  mod_err
2  mod_err2

```

Given the input:



The output log looks like:

```

1  Report file = coding_rule_check.rpt
2
3  Building design information...
4  Design information is built successfully.
5
6  Checking coding rule...
7  2 violation(s) detected.
8
9  /demo/CheckModPortDecModport/demo.sv(24) : Error : module declaration "mod_err" must be SystemVerilog format and must write modport declaration.
10 /demo/CheckModPortDecModport/demo.sv(33) : Error : module declaration "mod_err2" must be SystemVerilog format and must write modport declaration.

```

Check Module Port Connections Using Modport



Description

Given

- Output log file name.
- File name that contains the list of module names to be checked. Each line of file contains one module name.
 - Optional argument, all the modules will be checked to confirm whether the file name is not be given.

Result

- Report violation(s) whether there is module port connection (high-connection of module port) using modport.
 - Generic interface is not allowed.

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignValidation/CheckModPortConnModport`

Tcl Entry File

`checkModPortDeclModport.tcl`

Tcl Procedure Entry

```
proc check_mod_port_conn_modport_main { outFile { modListFile "" } }
```

Arguments

utFile:

Specify the output log file name, default is "coding_rule_check.rpt".

modListFile (Optional):

Specify the file name that contains the list of module names to be checked. Each line of file contains one module name. All the modules will be checked to confirm whether the file name is given.

Batch Mode Usage

```
Usage: checkModPortConnModport_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-mod_list <mod_list_file>] \  
      [-h]
```

For example:

```
checkModPortConnModport_batch.pl -f run.f  
checkModPortConnModport_batch.pl -f run.f  
-o coding_rule_check.rpt  
checkModPortConnModport_batch.pl -f run.f  
-o coding_rule_check.rpt -mod_list modlist
```

Limitations

Currently this App won't report signals in VHDL components.

Example

[demo.sv](#)

```

1  module pcs (pcs_mac_intf.from_mode pcs_from,
2      input logic [7:0] data_in,
3      output logic [7:0] data_out);
4      assign data_out = pcs_from.pcs_mac_axibus_intf;
5      assign pcs_from.pcs_mac_handshake_intf = data_in;
6  endmodule
7
8  module mac (pcs_mac_intf.to_mode mac_to,
9      input logic [7:0] data_in,
10     output logic [7:0] data_out);
11     assign data_out = mac_to.pcs_mac_handshake_intf;
12     assign mac_to.pcs_mac_axibus_intf = data_in;
13 endmodule
14
15 interface pcs_mac_intf;
16     logic [7:0] pcs_mac_axibus_intf;
17     logic [7:0] pcs_mac_handshake_intf;
18
19     modport from_mode (output pcs_mac_handshake_intf, input pcs_mac_axibus_intf);
20     modport to_mode (input pcs_mac_handshake_intf, output pcs_mac_axibus_intf);
21 endinterface
22
23 interface simple_bus(input logic clk);
24 endinterface
25
26 module mod_demo;
27     logic [7:0] pcs_data_in, mac_data_in;
28     logic [7:0] pcs_data_out, mac_data_out;
29
30     simple_bus u_sb();
31     pcs_mac_intf u_pcs_mac_intf();
32     // high-conn of instance "u_pcs" uses modport
33     pcs u_pcs (.pcs_from(u_pcs_mac_intf.from_mode),
34         .data_in(pcs_data_in),
35         .data_out(pcs_data_out));
36     // high-conn of instance "u_mac" should use modport
37     mac u_mac (//.mac_to(u_pcs_mac_intf.to_mode),
38         .data_in(mac_data_in),
39         .data_out(mac_data_out));
40
41 endmodule

```

[demo.list](#)

1	pcs
2	mac

Given the input:



The output log looks like:

```

1  Report file = coding_rule_check.rpt
2
3  Building design information...
4  Design information is built successfully.
5
6  Checking coding rule...
7  1 violation(s) detected.
8
9  /remote/hw52home1/hzchou/tmp/CRC/CheckModPortConnModport/demo.sv(37) : Error : instance "mod_demo.u_mac" must be instantiated with modport.

```

Signal Existence Check in Design



Description

Given a file which lists the signal full hierarchical name per line, the script will check whether the signal actually exists in the design.

The signal file looks like the following format:

```
=====
system.i_cpu.i_ALUB.i_alu.U133
system.i_cpu.i_ALUB.i_alu.U140
system.i_cpu.i_ALUB.U243
system.i_cpu.i_CCU.C21_reg
system.i_cpu.i_CCU.add_84.U1_3
=====
```

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignValidation/SignalExistCheckDesign`

Tcl Entry File

`signalExistCheckDesign.tcl`

Tcl Procedure Entry

```
signal_exist_check_design_main { signal_list_file output_log }
```

Arguments

signal_list_file:

Specify the file that list the signal's full hierarchical name per line.

output_log:

Specify the output log file name.

Batch Mode Usage

```
Usage: signalExistCheckDesign_batch.pl <verdi_import_design> \  
      -file <signal_file> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./signalExistCheckDesign_batch.pl -f run.f -file sig.txt -o  
signalExistCheckDesign.log
```

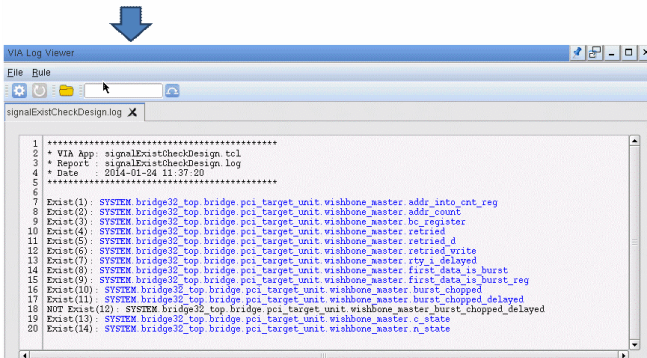
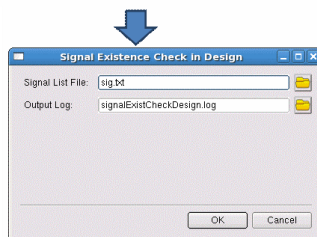
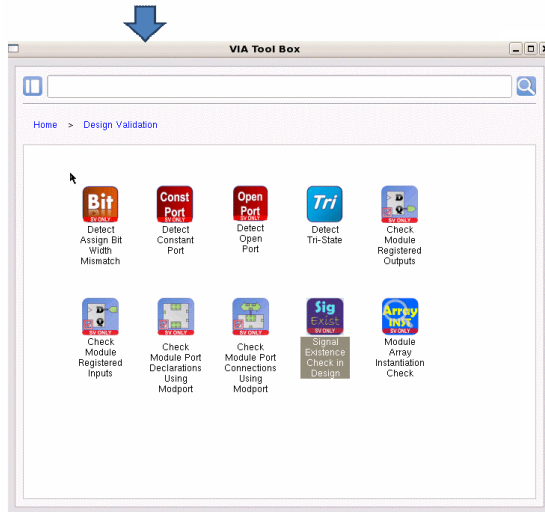
Limitations

N/A

Example

Signal file

```
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.addr_into_cnt_reg  
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.addr_count  
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.bc_register  
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.retried  
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.retried_d  
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.retried_write  
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.rty_i_delayed  
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.first_data_is_burst  
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.burst_chopped  
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.burst_chopped_delayed  
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.burst_chopped_delayed  
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.c_state  
SYSTEM.bridge32_top.bridge_pci_target_unit.wishbone_master.n_state  
-  
"sig.txt" 14L, 1031C
```



Module Array Instantiation Check



Description

Check whether the import design has the coding style of module array instantiation. For example,

```
fulladder fa[3:0]
  (.sum(sum[3:0]), .c_out(c_out[3:0]), .x(x[3:0]), .y(y[3:0]), .c_in(c_in[
3:0]));
```

Since some EDA tools do not allow the coding style, so the script will report it

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignValidation/ModuleArrayInstantiationCheck`

Tcl Entry File

`moduleArrayInstantiationCheck.tcl`

Tcl Procedure Entry

`module_array_instantiation_check_main { output_log }`

Arguments

`output_log`:

Specify the output log file name.

Batch Mode Usage

```
Usage: moduleArrayInstantiationCheck_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-h]
```

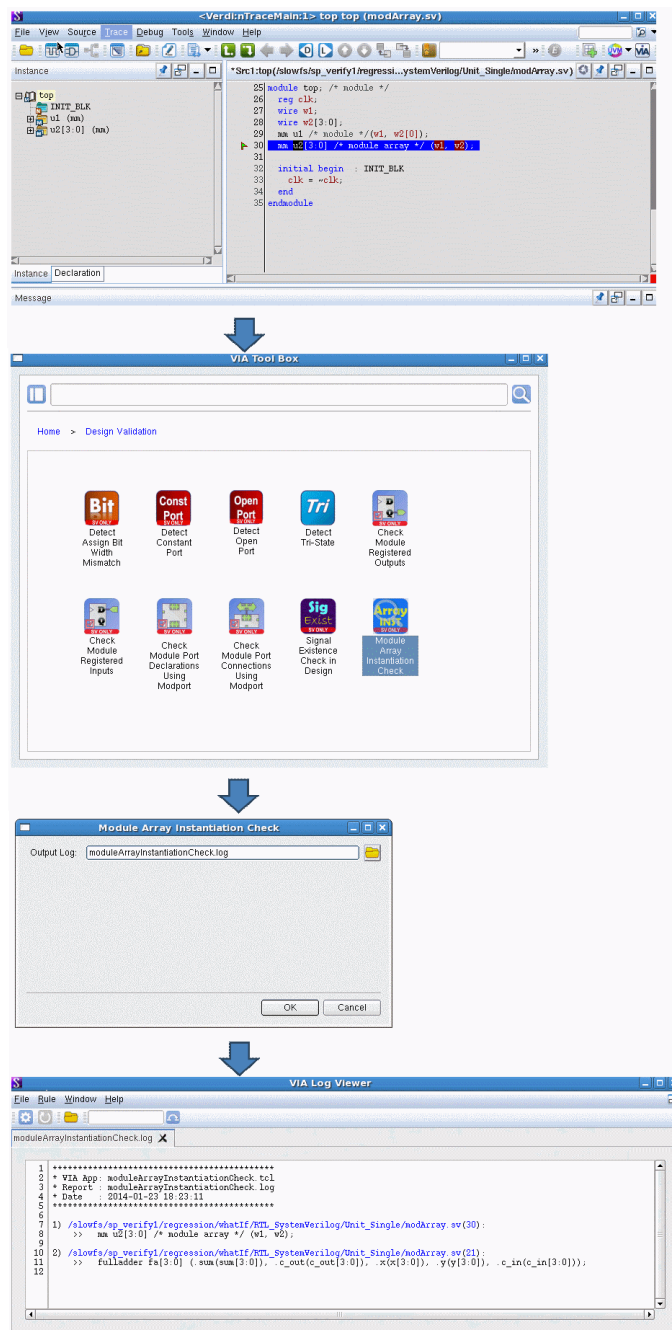
For example:

```
./moduleArrayInstantiationCheck_batch.pl -f run.f -o  
moduleArrayInstantiationCheck.log
```

Limitations

Verilog/SystemVerilog only. No VHDL support.

Example



5

FSDB Investigation

This section describes the following topics:

- [Detect PortZ](#)
- [Get FSDB Scope Hierarchy](#)
- [Get Register Values](#)
- [Serial Signal to Parallel Value](#)
- [Signal Existence Check in FSDB](#)
- [FSDB Signal Query](#)

Detect PortZ



Description:

Detect whether module ports have the Z values in the specified FSDB between the (begin_time, end_time) region.

Tcl Script Location:

```
$VERDI_HOME/share/VIA/Apps/FsdbInvestigation/DetectPortZ
```

Tcl Entry File:

detectPortZ.tcl

Tcl Procedure Entry:

```
detect_port_z_main { modules fsdb_file begin_time end_time time_unit output_log }
```

Arguments:

modules:

(Optional) Specify the module name; default is empty string.

The name can be specified by exact one or wildcard.

The name can be module hierarchy name or module definition name.

If the module name is empty string, then report all modules.

Use space to specify more than one module. For example,

"system.i_cpu ALU PC* system.i_cpu.i_*.reg*"

FSDB:

Specify the FSDB name.

begin_time:

The begin time.

end_time:

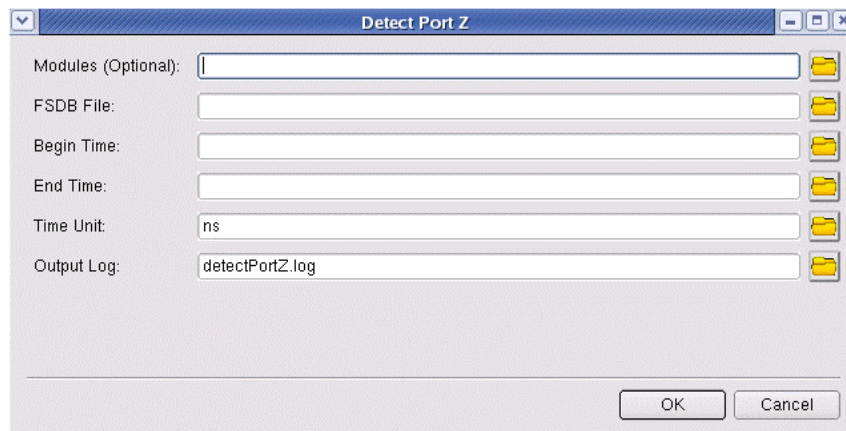
The end time.

time_unit:

The time unit of begin/end time; it must be ms/us/ns/ps/fs.

output_log:

Specify the output log file name.



Batch Mode Usage:

```
Usage: detectPortZ_batch.pl <verdi_import_design> \
      -fsdb <fsdb_file> \
      -begin_time <begin_time> \
      -end_time <end_time> \
      -time_unit <ms|us|ns|ps|fs> \
      [-o <output_log>] \
      [-modules "<target_module_names>"] \
      [-h]
```

For example:

```
./detectPortZ_batch.pl -f run.f -fsdb cpu.fsdb -begin_time 10
-end_time 50 -time_unit ns -modules "TOP" -o detectPortZ.log
./detectPortZ_batch.pl -f run.f -fsdb cpu.fsdb -begin_time 10
-end_time 50 -time_unit ns -modules "system.TOP.* ALU*"
./detectPortZ_batch.pl -f run.f -fsdb cpu.fsdb -begin_time 10
-end_time 50 -time_unit ns -o detectPortZ.log
```

Limitations:

This App currently won't detect the Z values of the ports in VHDL components.

Example:

```
// ----- FSDB ----- //
```




```
Target Modules: SYSTEM.bridge32_top
FSDB           : ./pci.fsdb
Begin Time     : 1000 ns
End Time       : 10000 ns
```

[illegible]

Get FSDB Scope Hierarchy



Description:

Get the hierarchical scope tree in FSDB.

Tcl Script Location:

`$VERDI_HOME/share/VIA/Apps/FsdbInvestigation/GetFsdbScopeHier`

Tcl Entry File:

`getFsdbScopeHier.tcl`

Tcl Procedure Entry:

`get_fsdb_scope_hier_main { fsdb_name output_log }`

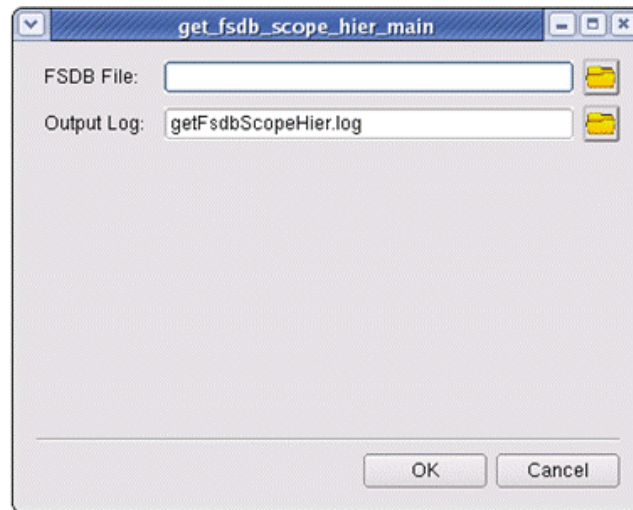
Arguments:

`fsdb_file:`

Specify the FSDB name.

`output_log:`

Specify the output log file name.



Batch Mode Usage:

```
Usage: getFsdbScopeHier_batch.pl -fsdb <fsdb_name> \  
      [-o <output_log>] \  
      [-h]
```

For example:

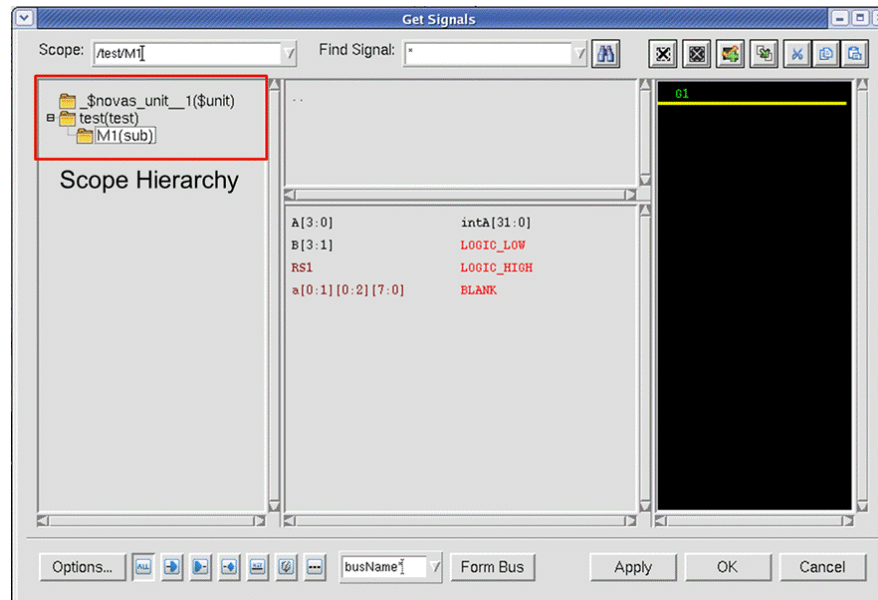
```
./getFsdbScopeHier_batch.pl -fsdb <fsdb_name> -o  
getFsdbScopeHier.log
```

Limitations:

N/A

Example:

```
// ----- example FSDB ----- //
```



```
// ----- getFsdbScopeHier.log ----- //
```

```
*****
* VIA App: getFsdbScopeHier.tcl
* Report : getFsdbScopeHier.log
* Date   : 2013-03-22 09:52:00
*****
```

```
FSDB Name: ./test.fsdb
-----
Scope: test
Scope: test.M1
Scope: _$novas_unit__1
```

Get Register Values



Description:

Get the values of inferred register outputs at a specific time of an FSDB.

Tcl Script Location:

`$VERDI_HOME/share/VIA/Apps/FsdbInvestigation/GetRegValues`

Tcl Entry File:

`getRegValues.tcl`

Tcl Procedure Entry:

`get_reg_values_main { fsdb_file time unit output_log }`

Arguments:

`fsdb_file`:

Specify the fsdb file name.

`time`:

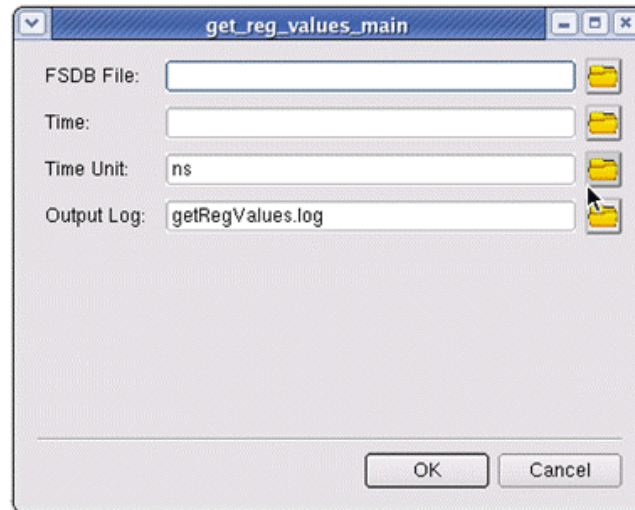
Specify the time to report the register values

`unit`:

Specify the time unit of the time; it must be ms, us, ns, ps, fs

`output_log`:

Specify the output log file name.



Batch Mode Usage:

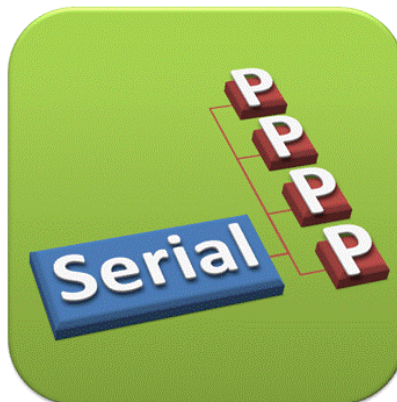
```
Usage: getRegValues_batch.pl <verdi_import_design> \  
      -fsdb <fsdb_file> \  
      -val_time <val_time> \  
      -time_unit <ms|us|ns|ps|fs> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./getRegValues_batch.pl -f run.f -fsdb cpu.fsdb -val_time  
1000 -time_unit ns -o getRegValues.log
```

Limitations:

N/A

[illegible]

Tcl Entry File:

SerialSig2Parallel.tcl

Tcl Procedure Entry:

```
serial_sig_2_parallel { sigName clkName enableName clkCount fsdbFileName outputLog }
```

Arguments:

sigName:

Target serial signal full name.

clkName:

Reference clock signal fullname.

enableName:

Enable signal name.

clkCount:

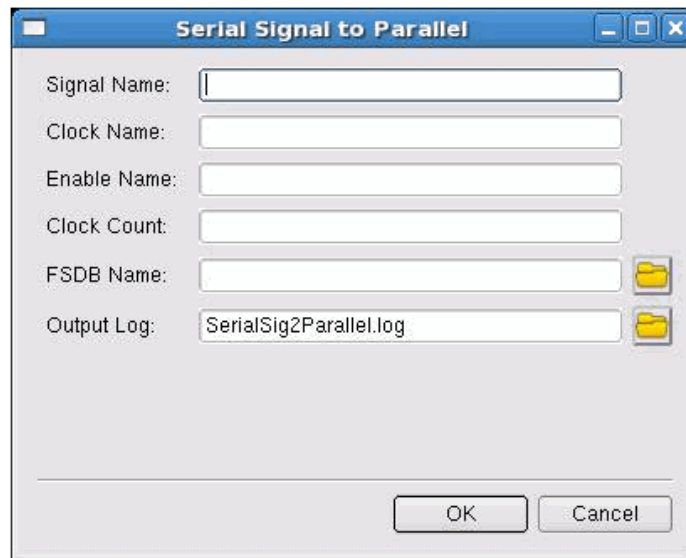
Clock cycles number of each serial data.

fsdbFileName:

Specify the FSDB name.

outputLog:

Specify the output log file name.



Batch Mode Usage:

```
Usage: SerialSig2Parallel_batch.pl -fsdb <fsdb_name> \  
-sig <target_signal> \  
-clk <ref_clock> \  
-en <enable_signal> \  
[-o <output_log>] \  
[-h]
```

For example:

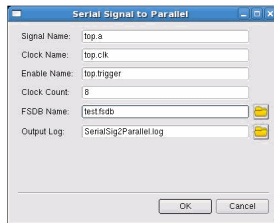
```
./SerialSig2Parallel_batch.pl -fsdb cpu.fsdb -o  
SerialSig2Parallel.log -count 8 -sig top.clk  
-en top.enable
```

Limitations:

N/A

Example:

```
// ----- example FSDB ----- //
```



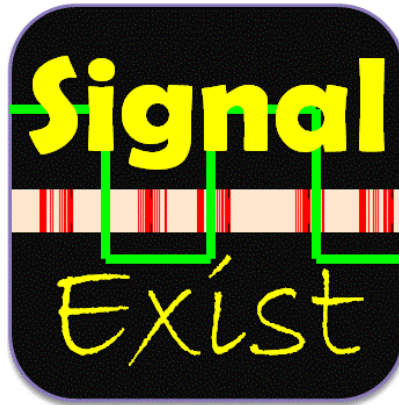
```
// ----- SerialSig2Parallel.log ----- //
```

```
*****
* VIA App: SerialSig2Parallel.tcl
* Report : SerialSig2Parallel.log
* Date   : 2013-11-25 16:32:47
*****
```

```
FSDB file: test.fsdb
FSDB time range: 0 ~ 500
FSDB time unit: 1s
Data signal name: top.a
Clock signal name: top.clk
Enable signal name: top.trigger
Number of clock counts: 8
```

```
-----
(Time period) : Parallel Value
-----
( 25 ~ 95 ) : 0x6A
( 205 ~ 275 ) : 0xA9
( 405 ~ 475 ) : 0x2A
-----
```

Signal Existence Check in FSDB



Description:

Given an FSDB and given a file which lists the signal full hierarchical name per line, the script will check if the signal actually exists in the FSDB.

The signal file looks like the following format:

```
=====
system.i_cpu.i_ALUB.i_alu.U133
system.i_cpu.i_ALUB.i_alu.U140
system.i_cpu.i_ALUB.U243
system.i_cpu.i_CCU.C21_reg
system.i_cpu.i_CCU.add_84.U1_3
=====
```

Tcl Script Location:

```
$VERDI_HOME/share/VIA/Apps/FsdbInvestigation/SignalExistCheckFSDB
```

Tcl Entry File:

```
signalExistCheckFSDB.tcl
```

Tcl Procedure Entry:

```
signal_exist_check_FSDB_main { signal_list_file output_log }
```

Arguments:

signal_list_file:

Specify the file that list the signal's full hierarchical name per line

output_log:

Specify the output log file name.

Batch Mode Usage:

```
Usage: signalExistCheckFSDB_batch.pl \  
      -fsdb <fsdb_file> \  
      -file <signal_file> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./signalExistCheckFSDB_batch.pl -fsdb fulldump.fsdb -file sig.txt -o  
signalExistCheckFSDB.log
```

Limitations:

N/A

Example:



FSDB Signal Query



Description:

Query signals in an FSDB.

Tcl Script Location:

`$VERDI_HOME/share/VIA/Apps/FsdbInvestigation/FsdbSigQ`

Tcl Entry File:

`fsdbSigQ.tcl`

Tcl Procedure Entry:

```
fsdb_sig_Q_main { fsdb_file output_log sig sig_exclude scope scope_exclude  
scope_only }
```

Arguments:

`fsdb_file:`

Specify the fsdb file name.

`output_log:`

Specify the output log file name.

sig:

Specify the signal patterns for the search (wildcard).

Use the space for multiple patterns.

The signal patterns should be simple names, that is, the delimiter is not allowed.

For example, `"*Q* *_reg"`

sig_exclude:

(Optional) Specify the signal patterns that are excluded for the search.

Use the space for multiple patterns.

Empty string is allowed.

scope:

(Optional) Specify the target scope for the search; default is all top scopes.

The target scope should be a full hierarchical name.

scope_exclude:

(Optional) Specify the scope patterns that are excluded for the search.

Use the space for multiple patterns.

The scope patterns should be simple names, that is, the delimiter is not allowed.

Empty string is allowed.

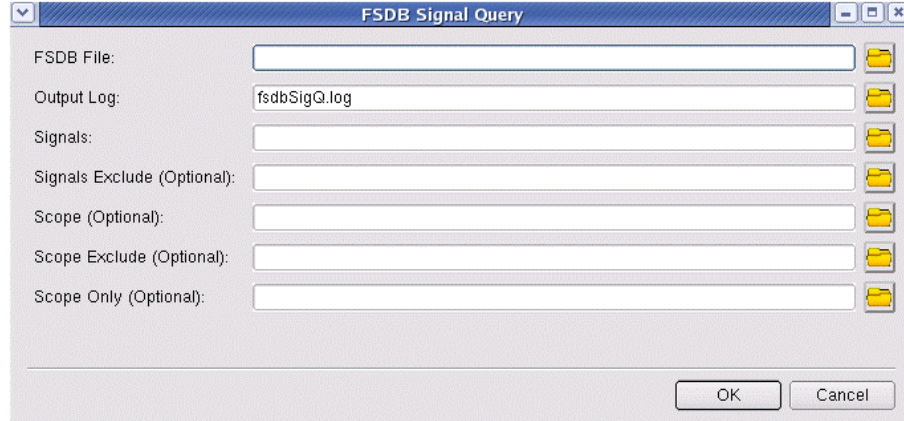
scope_only:

(Optional) Specify the scope patterns that are to be searched.

Use the space for multiple patterns.

The scope patterns should be simple names, that is, the delimiter is not allowed.

Empty string is allowed.



Batch Mode Usage:

```
Usage: fsdbSigQ_batch.pl [verdi_import_design] \  
        [-o <output_log>] \  
        -fsdb <fsdb_file> \  
        -sig <signal_pattern_string> \  
        -sig_exclude <signal_pattern_string> \  
        -scope <target_scope> \  
        -scope_exclude <scope_pattern_string> \  
        -scope_only <scope_pattern_string> \  
        [-h]
```

For example:

```
./fsdbSigQ_batch.pl -fsdb CPU.fsdb -sig "DataIn"  
-o fsdbSigQ.log  
./fsdbSigQ_batch.pl -fsdb CPU.fsdb -sig "*Q"  
./fsdbSigQ_batch.pl -fsdb CPU.fsdb -sig "*Q *RN"  
-scope "system.i_cpu"  
./fsdbSigQ_batch.pl -fsdb CPU.fsdb -sig "*Q" -sig_exclude  
"*_reg*"  
./fsdbSigQ_batch.pl -fsdb CPU.fsdb -sig "*Q" -scope_exclude  
"*N0_blk* *N5_blk*"  
./fsdbSigQ_batch.pl -fsdb CPU.fsdb -scope "system.i_CPU.i_ALUB" -sig  
"*Q" -scope_only "*_REQ"
```

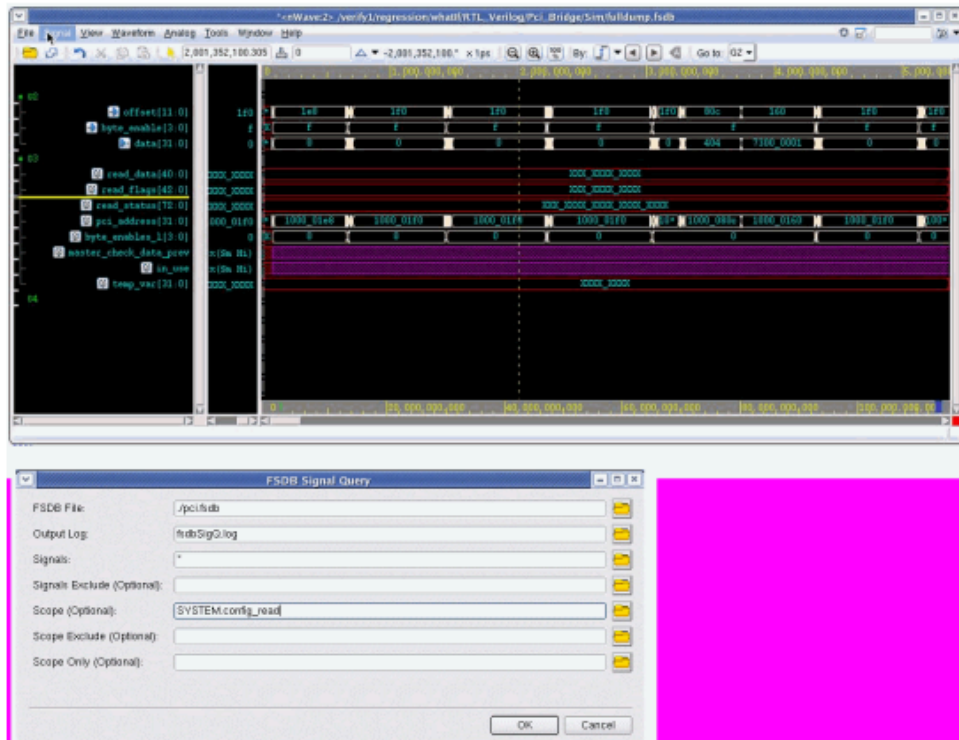
Limitations:

N/A

Example:

```
// ----- FSDB ----- //
```


// ----- FSDB ----- //



// ----- fsdbSigQ.log ----- //

```
*****
* VIA App: fsdbSigQ.tcl
* Report : fsdbSigQ.log
* Date   : Wed Mar 20 15:18:35 2013
*****
```

```
FSDB           : ./pci.fsdb
Query Signals  : *
Excluded Signals: N/A
Target Scope   : SYSTEM.config_read
Excluded Scope : N/A
Scope Only     : N/A
```

```
-----
SYSTEM.config_read.offset
SYSTEM.config_read.byte_enable
SYSTEM.config_read.data
SYSTEM.config_read.read_data
SYSTEM.config_read.read_flags
```

```
SYSTEM.config_read.read_status  
SYSTEM.config_read.pci_address  
SYSTEM.config_read.byte_enables_1  
SYSTEM.config_read.master_check_data_prev  
SYSTEM.config_read.in_use  
SYSTEM.config_read.temp_var
```

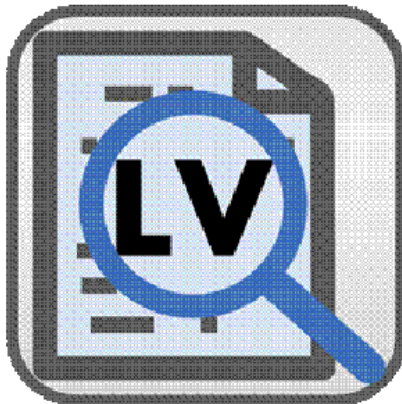
6

Utility

This section describes the following topics:

- [Smart Log](#)
- [Get Instance List](#)
- [Get Signal List](#)
- [Analyze Force Debug Report](#)
- [Macro Utility](#)
- [Analyze Time Text Log](#)
- [View FSDB Frequency](#)
- [Batch Mode Usage:](#)

Smart Log



Description:

Display the log with some bound hyperlinks by specific rule file.

Click the hyperlinks to execute some defined actions to interact with the Verdi platform.

Define the customized rule file to display the specific format files.

Tcl Script Location:

`$VERDI_HOME/share/VIA/Apps/ Utility/LogViewer`

Tcl Entry File:

`logViewer.tcl`

Tcl Procedure Entry:

`log_viewer_main { }`

Arguments:

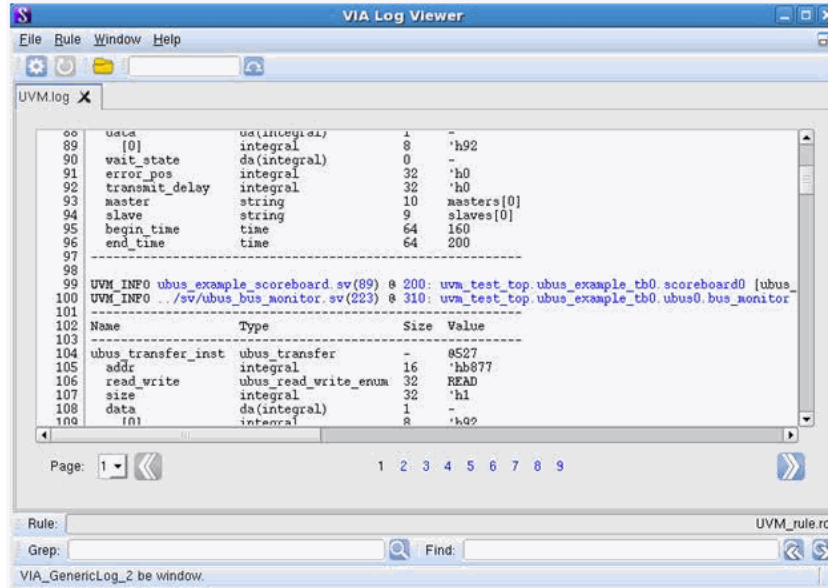
N/A

Limitations:

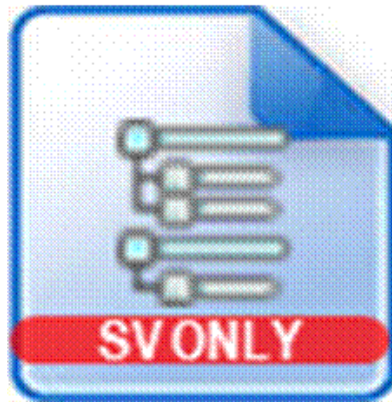
N/A

Example:

Display log with bound hyperlinks.



Get Instance List



Description:

Given the design import, the App will generate the instance list file by GUI.

The format of the instance list file is a full hierarchical name of a design instance per line.
For example:

=====

SYSTEM.bridge32_top.bridge.i_pci_wbs_wbb3_2_wbb2

SYSTEM.bridge32_top.bridge.input_register

```
SYSTEM.bridge32_top.bridge.output_backup
SYSTEM.bridge32_top.bridge.parity_checker.par_gen
SYSTEM.bridge32_top.bridge.parity_checker.serr_en_crit_gen
SYSTEM.bridge32_top.bridge.parity_checker.perr_crit_gen
SYSTEM.bridge32_top.bridge.wishbone_slave_unit.del_sync.comp_sync
SYSTEM.bridge32_top.bridge.wishbone_slave_unit.del_sync.req_sync
=====
```

Tcl Script Location:

```
$VERDI_HOME/share/VIA/Apps/ Utility/InstanceList
```

Tcl Entry File:

```
instanceList.tcl
```

Tcl Procedure Entry:

```
instance_list_main { }
```

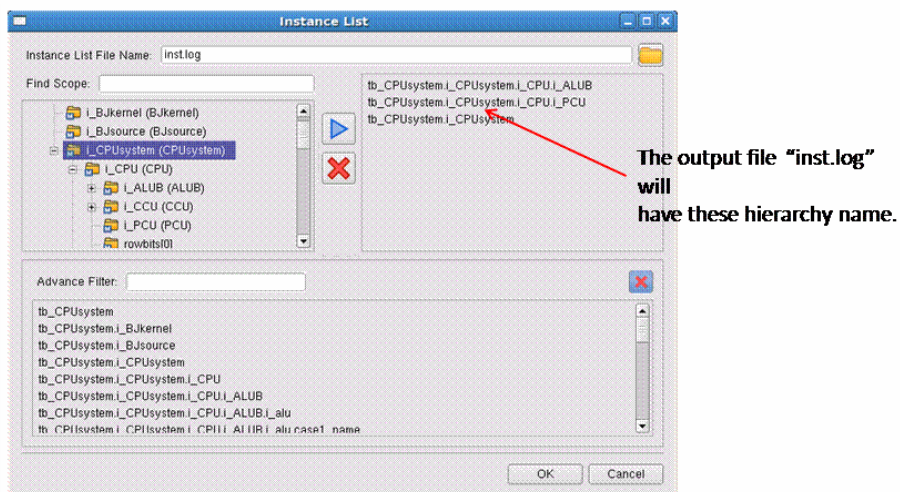
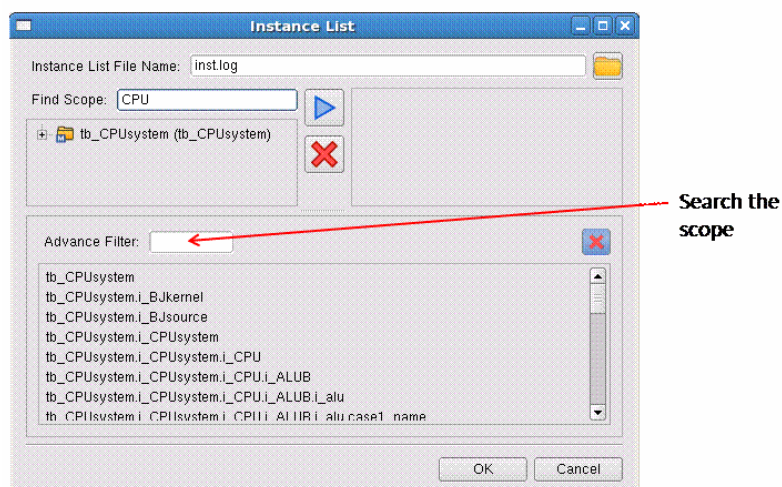
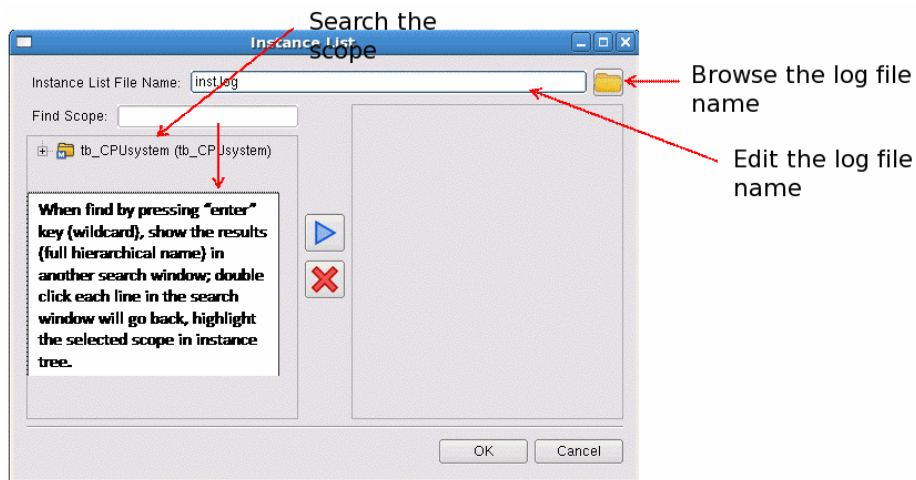
Arguments:

N/A

Limitations:

Support System Verilog Only.

Example:



Get Signal List



Description:

Given the design import, the App will generate the signal list file by GUI.

The format of the signal list file is a full hierarchical name of a design instance's signal per line. For example:

```
=====
tb_CPUsystem.i_BJkernel.BJ_clock
tb_CPUsystem.i_BJkernel.Card
tb_CPUsystem.i_BJkernel.OK
tb_CPUsystem.i_BJkernel.reset
tb_CPUsystem.i_BJkernel.state
=====
```

Tcl Script Location:

```
$VERDI_HOME/share/VIA/Apps/ Utility/SignalList
```

Tcl Entry File:

signalList.tcl

Tcl Procedure Entry:

```
signal_list_main { }
```

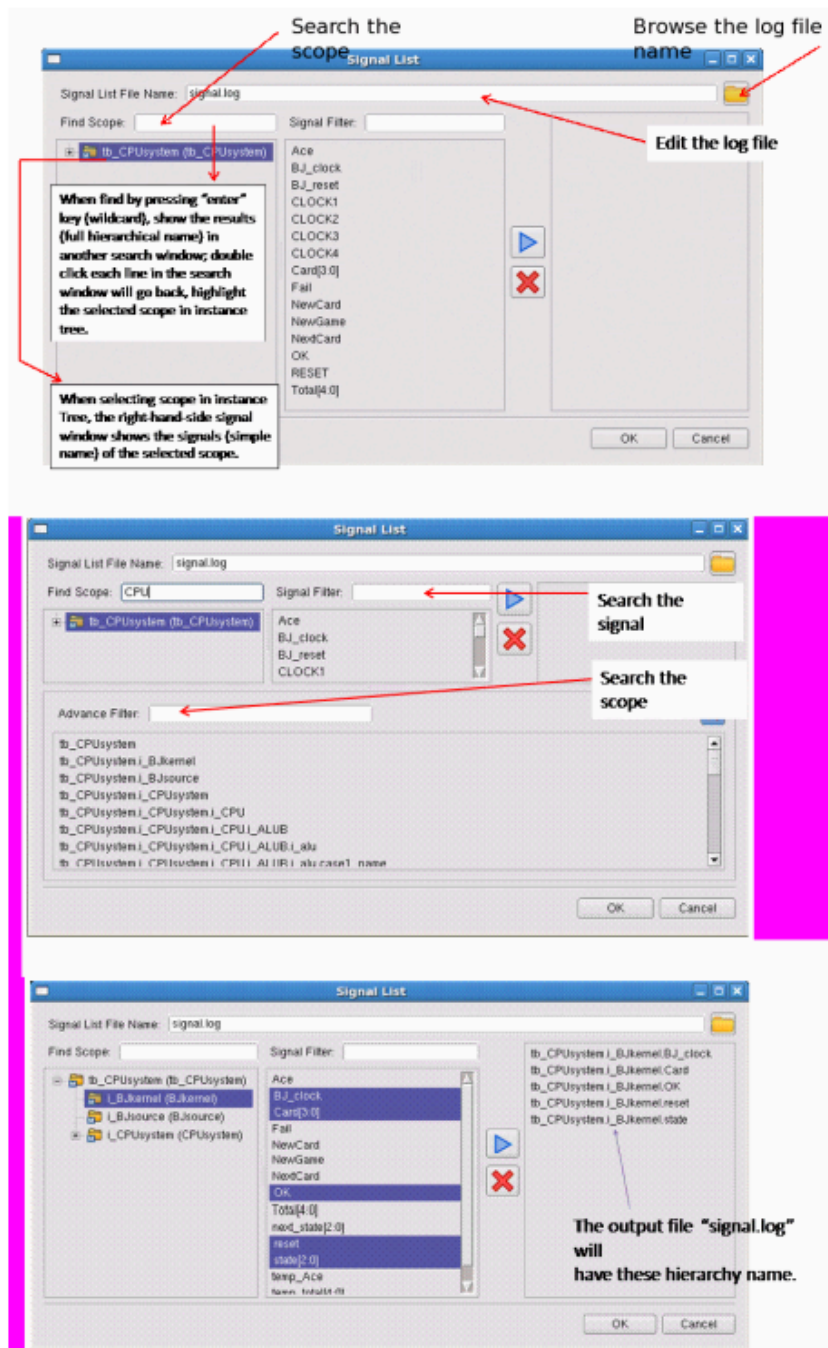
Arguments:

N/A

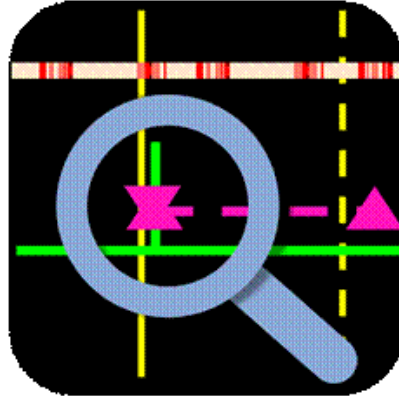
Limitations:

Support System Verilog Only.

Example:



Analyze Force Debug Report



Description:

Open Force Debug Report Log:

Given a log file in the format:

=====

Time(1s),<signal>

<time>, Forced , ^?, design , <file_name>#<line_number>

=====

For Example,

=====

Time(1s),/top/reg_var

2, Forced , ^?, design , reg.v#8

2, Forced , ^?, design , reg.v#9

3, Forced , ^?, design , reg.v#10

=====

This App opens the log file in the Smart Log, the <signal>, <time>, <file_name> and <line_number> will be bonded;

1. Click <signal> in the output file will auto select signal in source.
2. Click <file_name> or <line_number> will auto jump to the file and line in source.
3. Click <time> will change cursor time.

Tcl Script Location:

`$VERDI_HOME/share/VIA/Apps/ Interoperability/AnalyzeForceDebugReport`

Tcl Entry File:

`analyzeForceDebugReport.tcl`

Tcl Procedure Entry:

```
proc analyze_force_debug_report_main { force_debug_report_log }
```

Arguments:

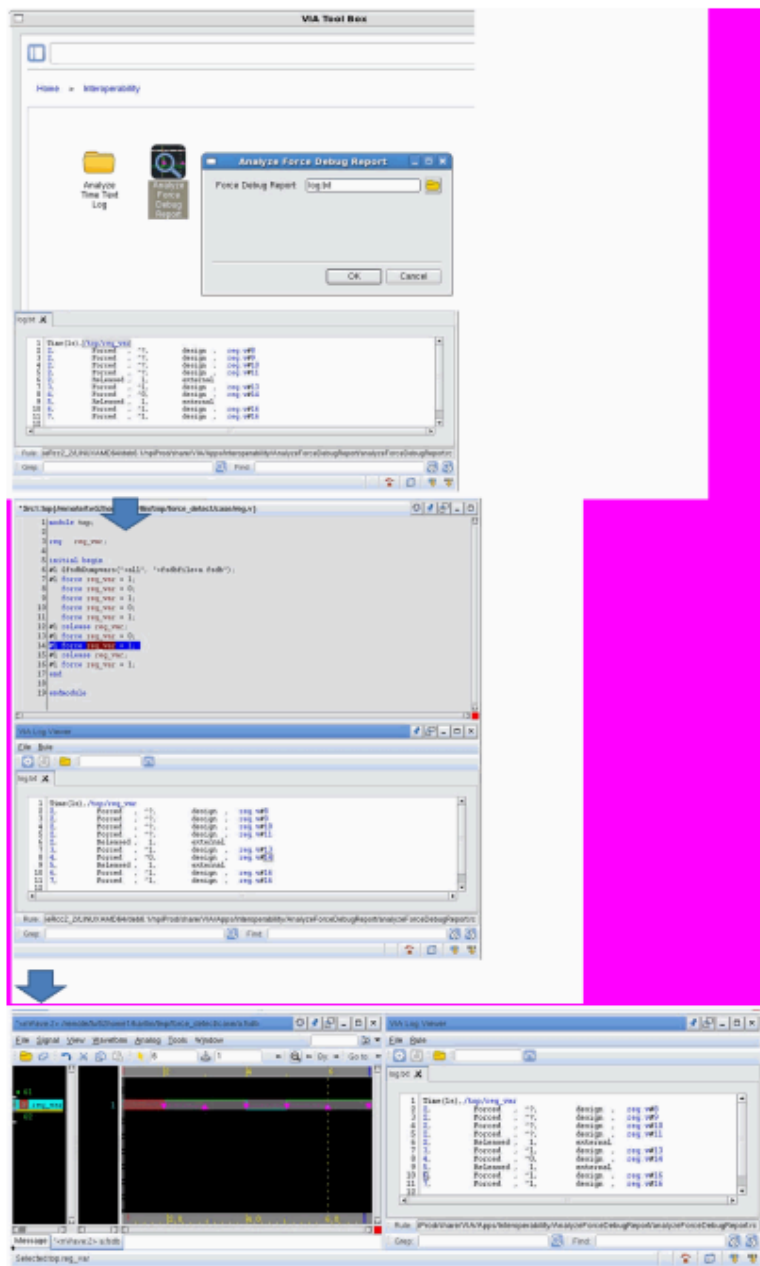
`force_debug_report_log:`

Specify the log file name in Force Debug Report format.

Limitations:

N/A

Example:



Macro Utility

Report Conditional Macro



Description:

Report the information of the macro used in the conditional compilation directives (`ifdef`, `ifndef`, `elsif`, `else`) in the design.

Tcl Script Location:

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/ReportConditionalMacro`

Tcl Entry File:

`reportConditionalMacro.tcl`

Tcl Procedure Entry:

```
report_conditional_macro_main { output_log {verbose 0} }
```

Arguments:

`output_log`:

Specify the output log file name.

`verbose`:

Verbose dump.

Batch Mode Usage:

```
Usage: reportConditionalMacro_batch.pl <verdi_import_design> \  
      [-o <output_log>] \
```

```
[-verbose] \  
[-h]
```

For example:

```
./reportConditionalMacro_batch.pl -f run.f -o  
reportConditionalMacro.log
```

```
./reportConditionalMacro_batch.pl -f run.f -o  
reportConditionalMacro.log -verbose
```

Limitations:

N/A

Example:

```
// ----- example.v ----- //  
  
module test;  
  reg clock, reset, in;  
  wire out1, out2;  
  
  parameter CYCLE = 50;  
  
  dut i_dut(.clock(clock), .reset(reset), .A(in), .Z(out1));  
  fsm i_fsm(.clock(clock), .reset(reset), .A(in), .Z(out2));  
  
  initial  
  begin  
    clock = 0;  
    reset = 1;  
    #(CYCLE/2) reset = 0;  
    #(CYCLE*2) reset = 1;  
  end  
  
  always  
  begin  
    clock = 0;  
    forever #CYCLE clock = ~clock;  
  end  
  
endmodule  
  
module dut (clock, reset, A, Z);  
  input clock, reset, A;  
  output Z;  
  wire tmp;  
  reg Z;  
  
  assign tmp = ~A;  
  
  `ifdef TEST1
```

```

always @(posedge clock or negedge reset)
begin
    if (!reset) Z = 0;
    else Z = tmp;
end
`elsif TEST2
always @(posedge clock or negedge reset)
begin
    if (!reset) Z = 0;
    else Z = ~tmp;
end
`endif

endmodule

module fsm (clock, reset, A, Z);

    parameter S0=2'b00, S1=2'b11, S2=2'b10, S3=2'b01;

    input clock, reset, A;
    output Z;
    reg Z;

    reg [1:0] c_state, n_state;

    always @(c_state or A)
    begin
        Z = 0;
        case (c_state)
            S0 : begin
                Z = 0;
                if (A)
                    n_state = S1;
                else
                    n_state = S0;
            end
            S1 : begin
                Z = 0;
                if (A)
                    n_state = S2;
                else
                    n_state = S1;
            end
            S2 : begin
                Z = 0;
                if (A)
                    n_state = S3;
                else
                    n_state = S2;
            end
            S3 : begin
                Z = 1;

```



```

        if (A)
            n_state = S0;
        else
            n_state = S3;
        end
        default : n_state = S0;
    endcase
end
`ifdef TEST3
always @(posedge clock or negedge reset)
begin : FSM
    if (!reset)
        c_state = S0;
    else
        c_state = n_state;
    end
`elsif TEST4
always @(negedge clock or negedge reset)
begin : FSM
    if (!reset)
        c_state = S0;
    else
        c_state = n_state;
    end
`endif

endmodule

// ----- run.f ----- //
+define+TEST2+TEST3
example.v

```



```
// ----- reportConditionalMacro.log ----- //
```

```
*****
```

```
* VIA App: Report Conditional Macro
```

```
* Report : reportConditionalMacro.log
```

```
* Date      : Tue May  7 18:37:08 2013
*****

=====
Defined Macro in Conditional Compiler Directives
=====

0)          TEST2
1)          TEST3

=====

Active Conditional Compiler Directive Report
=====

TEST2 : Active
-----
    Code :
    /
lx161a/chiachih_yen/DATA/NPI/Text_Model/MacroCheck/Case_Yuji/test/exampl
e.v (63-67)

TEST3 : Active
-----
    Code :
    /
lx161a/chiachih_yen/DATA/NPI/Text_Model/MacroCheck/Case_Yuji/test/exampl
e.v (119-125)

=====
Undefined Macro in Conditional Compiler Directives
=====

0)          TEST1
1)          TEST4

=====

Inactive Conditional Compiler Directive Report
=====

TEST1 : Inactive
-----
    Code :
    /
lx161a/chiachih_yen/DATA/NPI/Text_Model/MacroCheck/Case_Yuji/test/exampl
e.v (57-61)

TEST4 : Inactive
-----
    Code :
    /
```

```
1x161a/chiachih_yen/DATA/NPI/Text_Model/MacroCheck/Case_Yuji/test/example.v (127-133)
```

Detect FSM Macro Define



Description:

Given

- Output log file name.
- Scope to be checked.
 - Optional argument, all the scopes will be checked to confirm whether the scope is given.

Result

- Report macro symbols used in Finite State Machines (FSMs) and show the original definitions.

Note

- Schematic configuration will be changed.
 - schSetPreference -recogFSM on

Tcl Script Location:

```
$VERDI_HOME/share/VIA/Apps/DesignComprehension/DetectFSMMacroDefine
```

Tcl Entry File:

```
detectFSMMacroDefine.tcl
```

Tcl Procedure Entry:

```
detect_FSM_macro_define_main { outFile { scopeHierName "" } }
```

Arguments:

outFile:

Specify the output log file name, default is "FSM_State_Macro.rpt".

scopeHierName:

(Optional) Specify the scope full hierarchical name to be checked. The default is empty.

Batch Mode Usage:

```
Usage: detectFSMMacroDefine_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-scope <scope>] \  
      [-h]
```

For example:

```
detectFSMMacroDefine_batch.pl -f run.f  
detectFSMMacroDefine_batch.pl -f run.f -o FSM_State_Macro.rpt  
detectFSMMacroDefine_batch.pl -f run.f -o FSM_State_Macro.rpt -scope  
top
```

Limitations:

N/A

Example:

example.sv

```

1 module top;
2   macro_state_example instA ();
3   macro_state_example instB ();
4 endmodule
5
6 module macro_state_example(
7   input wire clock, reset, direct
8 );
9
10  `define M0 2'h0
11  `define M1 2'h1
12  `define M2 2'h2
13  `define M3 2'h3
14
15  reg [1:0] macro_state, non_macro_state;
16
17  always @(posedge clock or posedge reset) begin
18    if (reset) begin
19      macro_state <= M0;
20      non_macro_state <= 2'h0;
21    end
22    else begin
23      case (macro_state)
24        M0: macro_state <= M1;
25        M1: macro_state <= M2;
26        M2: macro_state <= (!direct) ? M0 : M2;
27        M3: macro_state <= M1;
28      endcase
29
30      case (non_macro_state)
31        2'h0: non_macro_state <= 2'h1;
32        2'h1: non_macro_state <= 2'h2;
33        2'h2: non_macro_state <= direct ? 2'h0 : 2'h2;
34        2'h3: non_macro_state <= 2'h1;
35      endcase
36    end
37  end
38 endmodule

```

Given the input:

Detect FSM Macro Define

Output File:

Scope Hierarchy Name (Optional):

The output log looks like:

```

1 **** FSM_State_Macro.rpt ****
2
3 SCOPE: top
4
5 DISTANCE: top instA (macro_state_example)
6
7 FSM INFO:
8   NAME: top instA macro_state_example FSM:17-27 FSM
9   FILE: [example.v : 18 : 19] [example.v : 18 : 19] [example.v : 24 : 24] [example.v : 23 : 23] [example.v : 17 : 17] [example.v : 25 : 25] [example.v : 26 : 26]
10  IN : clock
11  IN : direct
12  IN : reset
13  OUT: macro_state[1:0]
14
15 MACRO STACK INFO:
16  FILE: [ /Geno/FSM_State_Macro/example.v : 19 ]
17  STATE: M0 == 2'h0
18  FILE: [ /Geno/FSM_State_Macro/top/example.v : 24 ]
19  STATE: M0 == 2'h0
20  STATE: M1 == 2'h1
21  FILE: [ /Geno/FSM_State_Macro/top/example.v : 25 ]
22  STATE: M1 == 2'h1
23  STATE: M2 == 2'h2
24  FILE: [ /Geno/FSM_State_Macro/top/example.v : 26 ]
25  STATE: M2 == 2'h2
26  STATE: M3 == 2'h3
27  FILE: [ /Geno/FSM_State_Macro/top/example.v : 27 ]
28  STATE: M3 == 2'h3
29  STATE: M0 == 2'h0
30
31 DISTANCE: top instB (macro_state_example)
32
33 FSM INFO:
34   NAME: top instB macro_state_example FSM:17-27 FSM
35   FILE: [example.v : 18 : 19] [example.v : 18 : 19] [example.v : 24 : 24] [example.v : 23 : 23] [example.v : 17 : 17] [example.v : 25 : 25] [example.v : 26 : 26]
36  IN : clock
37  IN : direct
38  IN : reset
39  OUT: macro_state[1:0]
40
41 MACRO STACK INFO:
42  FILE: [ /Geno/FSM_State_Macro/top/example.v : 19 ]
43  STATE: M0 == 2'h0
44  FILE: [ /Geno/FSM_State_Macro/top/example.v : 24 ]
45  STATE: M0 == 2'h0
46  STATE: M1 == 2'h1
47  FILE: [ /Geno/FSM_State_Macro/top/example.v : 25 ]
48  STATE: M1 == 2'h1
49  STATE: M2 == 2'h2
50  FILE: [ /Geno/FSM_State_Macro/top/example.v : 26 ]
51  STATE: M2 == 2'h2
52  STATE: M3 == 2'h3
53  FILE: [ /Geno/FSM_State_Macro/top/example.v : 27 ]
54  STATE: M3 == 2'h3
55

```

Report Macro Define



Description:

Report the macro symbol definition (`define`, `undef`) in the design.

Tcl Script Location:

`$VERDI_HOME/share/VIA/Apps/DesignComprehension/ReportMacroDefine`

Tcl Entry File:

`reportMacroDefine.tcl`

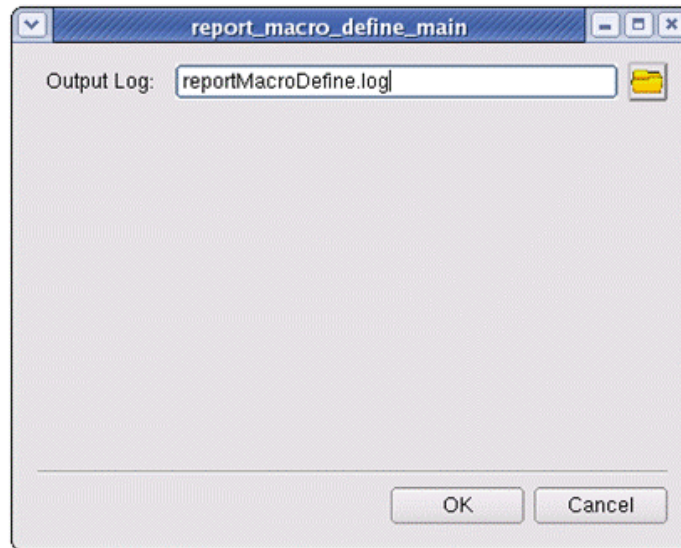
Tcl Procedure Entry:

`report_macro_define_main { output_log }`

Arguments:

`output_log:`

Specify the output log file name.



Batch Mode Usage:

```
Usage: reportMacroDefine_batch.pl <verdi_import_design> \  
      [-o <output_log>] \  
      [-h]
```

For example:

```
./reportMacroDefine_batch.pl -f run.f -o  
reportMacroDefine.log
```

Limitations:

N/A

Example:

```
// ----- example.v ----- //  
  
`define AAA  
`include "example_1.vh"  
`undef AAA  
  
`define BBB  
`include "example_1.vh"  
`undef BBB  
  
`include "example_1.vh"  
  
module TOP;  
  
wire ai, ao;
```



```

wire bi, bo;
wire ci, co;

AAA u_AAA (ai, ao);
BBB u_BBB (bi, bo);
CCC u_CCC (ci, co);

endmodule

// ----- example_1.vh ----- //

`ifdef AAA

module AAA (input ai, output ao);
    assign ao = ai;
endmodule

`elsif BBB

module BBB (input bi, output bo);
    assign bo = bi;
endmodule

`else

module CCC (input ci, output co);
    assign co = ci;
endmodule

`endif

// ----- reportMacroDefine.log ----- //

*****
* VIA App: Report Macro Define
* Report : reportMacroDefine.log
* Date   : Tue Mar 19 17:55:11 2013
*****

    0) AAA
    1) BBB

=====
Detailed information
=====

AAA

-----
`define AAA          (example.v : 2)
`undef AAA           (example.v : 4)

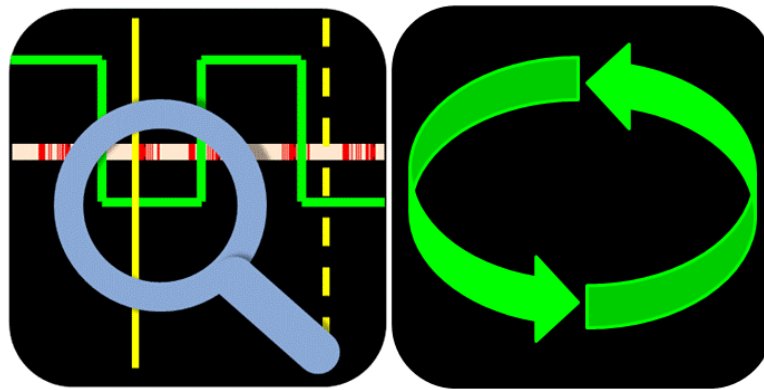
```

BBB

```
-----  
`define BBB          (example.v : 6)  
`undef BBB          (example.v : 8)
```

Analyze Time Text Log

Open Time Text Log / Synchronize Waveform Cursor to Log



Description:

The App contains two parts:

1. Open Time Text Log
2. Synchronize Waveform Cursor to Log
 - Open Time Text Log:

Given a log file in the format (<time> : <text>), the time increases per line. For example,

```
=====
10100 : CPU data rst
10110 : CPUx
111000 : abcdef
=====
```

This script opens the log file in the Smart Log, the <time> will be bonded; that is, when clicking the <time>, *nWave* window will jump to the waveform cursor time to <time>.

- Synchronize Waveform Cursor to Log

Given a waveform loaded in *nWave*, and given the log file imported by Open Time Text Log, the script will go to the line that is equal or the nearest one when changing the cursor in *nWave*.

Tcl Script Location:

`$VERDI_HOME/share/VIA/Apps/ Interoperability/SyncLogTimeWaveformCursor`

Tcl Entry File:

1. `openTimeTextLog.tcl`
2. `syncLogTimeWaveformCursor.tcl`

Tcl Procedure Entry:

1. `proc open_time_text_log_main { time_text_log }`
2. `proc sync_log_time_waveform_cursor_main { sync_type }`

Arguments:

- Open Time Text Log

`time_text_log`:

Specify the log file name in the format of (<time> : <text>) per line.

- Synchronize Waveform Cursor to Log

`sync_type`:

Specify the string "Begin" or "Stop".

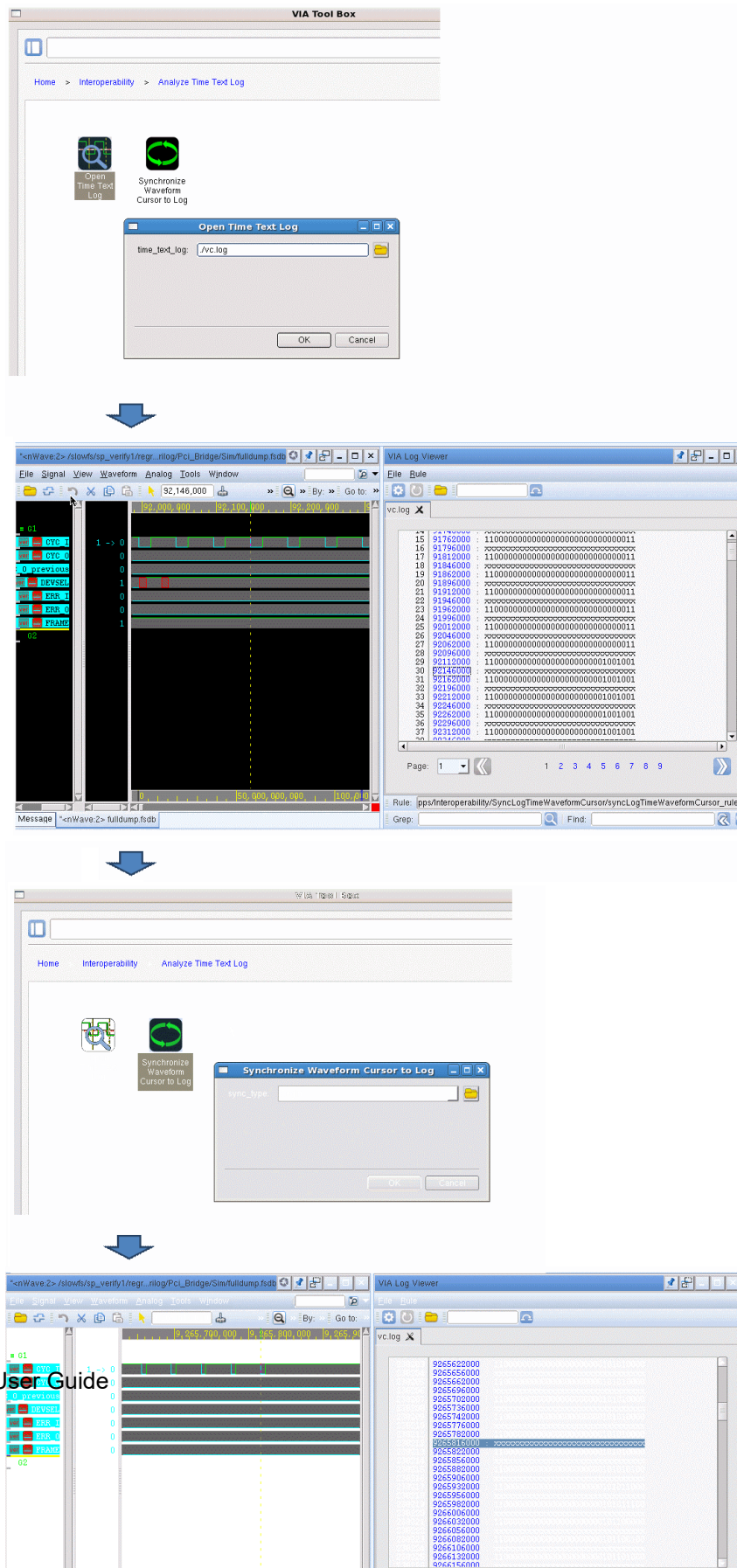
Batch Mode Usage:

N/A

Limitations:

N/A

Example:



View FSDB Frequency

Displays the frequency of a signal in FSDB.

Batch Mode Usage:

```
npiFsdbFreqDuty -waveform <waveform_file_name> -sig  
<signal_file_name>\ -time_window <time_window_file_name>\ [-tag  
<tag_file_name>]\ [-o <output_log>] \ [-h]
```

For example:

```
./npiFsdbFreqDuty -waveform ./novas.fsdb -sig ./sig.txt -  
time_window ./time_window.txt -o "general.log"
```

Arguments:

`-waveform <waveform file>`

The signal file name. One signal full hierarchical name per line is supported. The signal can be a valid logic expression. In the expression the signal name must be quoted by two ", like "<sig name>." For example,

```
top.u1.a  
top.u2.Wn  
~"top.u1.a" & "top.u2.Wn"  
~"top.u3.bus[2]" & "top.u2.Wn"  
top.u3.bus (bus signal)  
tag1 & tag2 | ~tag3 (expression tag)
```

`-time_window <time window file>`

The time window file name. One time window pair per line is supported. The begin/end time of time window is separated by a '-'. For example,

```
4us - 10us  
500ns - 1500ns  
1.2ms - 1.4ms
```

`-tag <tag file>`

The expression tag file. One expression tag per line is supported. The tag name and tag value is separated by '='. For example,

```
tag1 = ~"top.u1.a"  
tag2 = ~"top.u3.bus[2]" & "top.u2.Wn"  
tag3 = top.u1.
```

```
-o <output file>:
```

The output file name. If the output file is not specified, the result is dumped to STDOUT.

7

Advanced VC Apps

This section describes the following topics:

- [Design Manipulation](#)
- [Design Exploration](#)
- [FSDB Investigation](#)

Design Manipulation

ChiptInt



Full Name

Chip Integration

Description

Robust and rapid SoC integration tool with underlying features:

- Integration operations
 - Module Instantiation
 - Port-to-Port Connection
 - Port-to-Hierarchy
 - Instance Insertion
- Creation operations
 - Module Creation
 - Parameter Creation
 - Port Creation
 - Connection Checking
 - IP reuse from coherent setting files

Complete SoC integration flow:

- Build system hierarchy
- IP instantiation
- IP connection
- IP reuse

The Chip Integration App shares the same database with the following Apps:

- Hierarchy Manipulation
- Delete Pass Through
- Disconnect Port

Assume the database has been updated when using one of these Apps. When one of the rest Apps is invoked, the *Question* form will be opened to verify the request of using the updated database. Click the **Yes** button to use the updated database. Click the **No** button to use the original database and abandon all updates to the database.



Question Form

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignManipulation/ChipInt`

Tcl Entry File

`chipInt.tcl`

Tcl Procedure Entry

```
chip_int_main { }
```

Arguments

N/A

Batch Mode Usage

N/A

Limitations

N/A

Usage

For details on the usage of main features of *ChipInt*, refer to the `$VERDI_HOME/share/VIA/Apps/DesignManipulation/ChipInt/ChipInt_Main_Features.pdf` document.

HierMan



Full Name

Hierarchy Manipulation

Description

HierMan App is designed for users to change the design hierarchy and make all the necessary modifications automatically. For example, users can "**group**" several instances in the design and create a new scope for them, or just "**move**" an existing instance from the current position to a specific scope. Instead of making lots of modifications in the design source code manually, HierMan will **handle all connection changes and make the necessary modifications** in the design. With this application, users can put more effort on planning the hierarchy and eliminate the tedious and error prone modifications of all necessary changes in the design source code.

The Hierarchy Manipulation App shares the same database with the following Apps:

- Chip Integration
- Delete Pass Through
- Disconnect Port

Assume the database has been updated when using one of these Apps. When one of the rest Apps is invoked, the *Question* form will be opened to verify the request of using the updated database. Click the **Yes** button to use the updated database. Click the **No** button to use the original database and abandon all updates to the database.

Figure 25 Question Form



Question Form

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignManipulation/HierMan`

Tcl Entry File

`hierman.tcl`

Tcl Procedure Entry

`hierman_main { }`

Arguments

N/A

Batch Mode Usage

N/A

Limitations

N/A

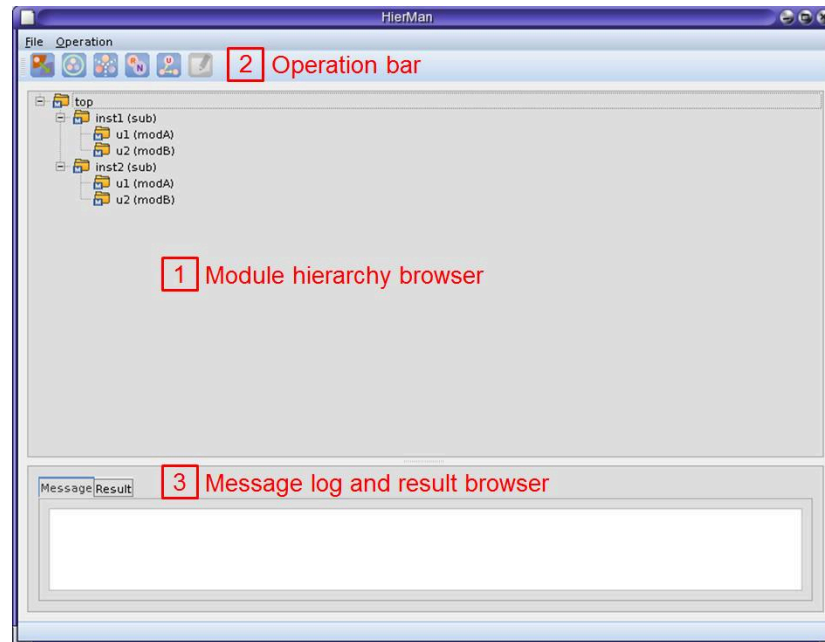
Usage

There are three blocks in the ungroup window as shown in the figure below:

1. **Module hierarchy browser:** You can browse instance hierarchy tree in this block for the operations.
2. **Operation bar:** You can perform “move”, “group”, “ungroup”, “rename”, “uniquify” and “export” operations by clicking the buttons.

3. **Message log and result browser:** This window shows the message log and you can review the modified design files in the “Result” tab.

Figure 26 HierMan Main Window



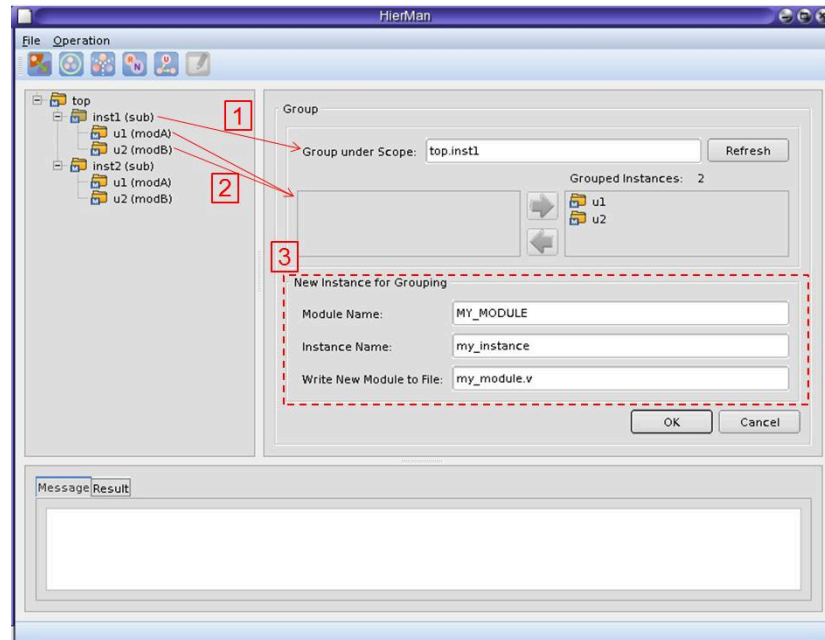
Example

The followings show examples of using HierMan:

- **Group operation:** Press the “Group” button. The figure below shows the following steps:
 1. Select “top.inst1” as the original scope of instances to be grouped.
 2. Select the instances “u1” and “u2” to be grouped from the left instance list to the right hand side.

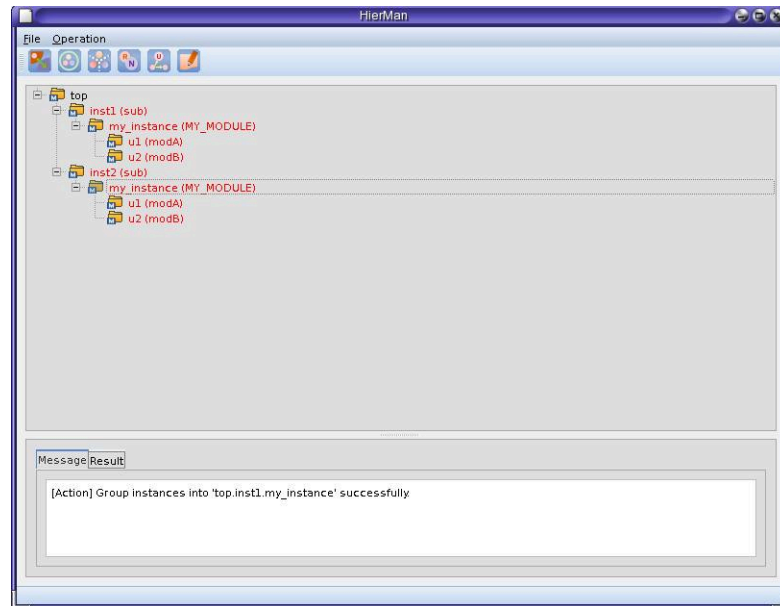
3. Input the new module definition name, file name, and instance name for the new scope to group the instances in the point 2.

Figure 27 Group Operation



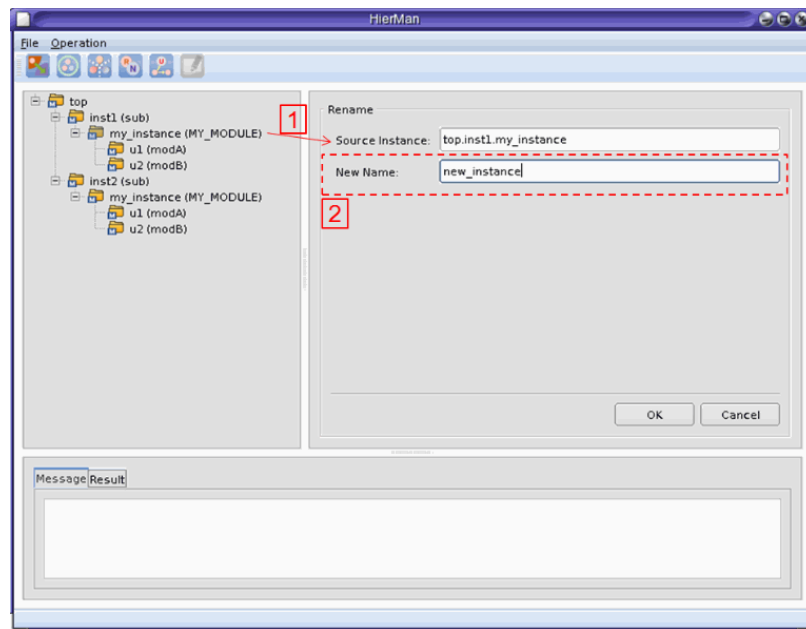
The resultant hierarchy will be shown in the following figure. The red words indicate the modified hierarchical instances. **All connection changes and other necessary modifications** are automatically accomplished.

Figure 28 *Hierarchy Tree After Group Operation*



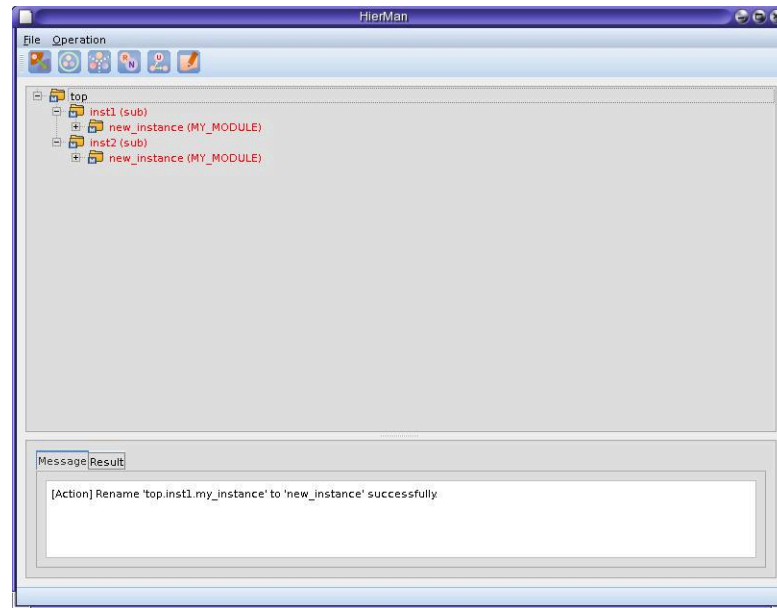
- **Rename operation:** Press the “Rename” button. The figure below shows the following steps.
 1. Select “top.inst1.my_instance” as the source instance to be renamed.
 2. Specify the new name “new_instance” for the source instance.

Figure 29 Rename Operation



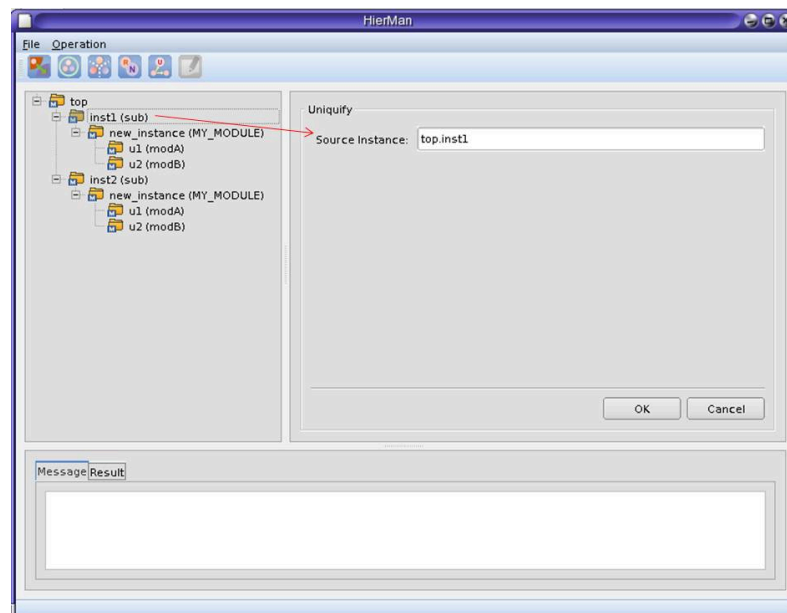
The resultant hierarchy will be shown in the following figure. The red words indicate the modified hierarchical instances.

Figure 30 Hierarchy Tree After Rename Operation



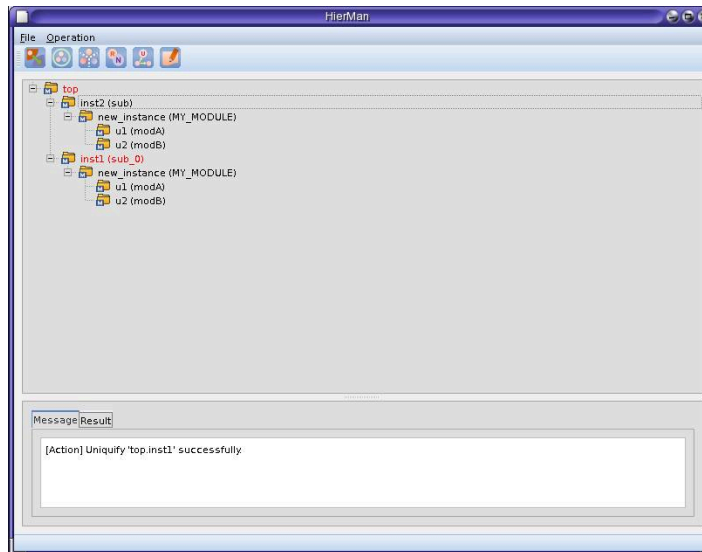
- **Uniquify operation:** Press the “Uniquify” button. The figure below shows the following step. Select “top.inst1” as the source instance to be uniquified.

Figure 31 Uniquify Operation



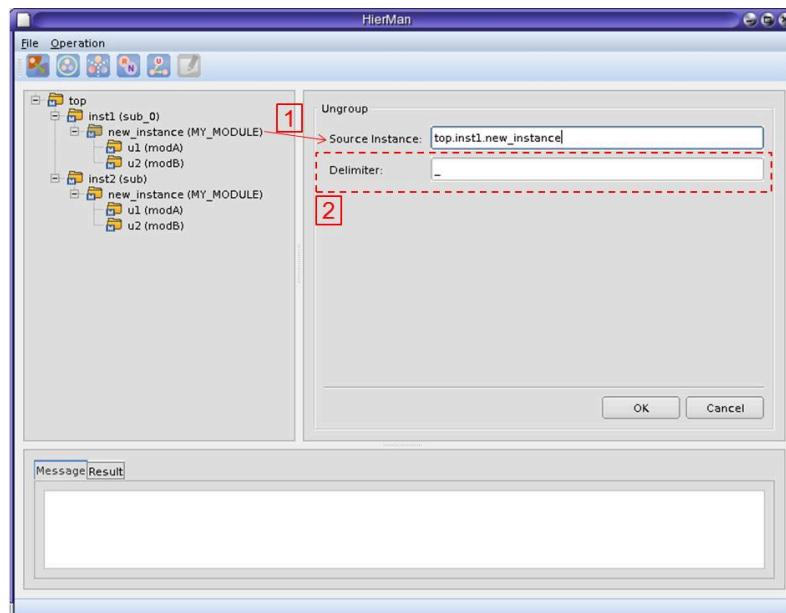
The resultant hierarchy will be shown in the following figures. The red words indicate the modified hierarchical instances.

Figure 32 Hierarchy Tree After Uniquify Operation



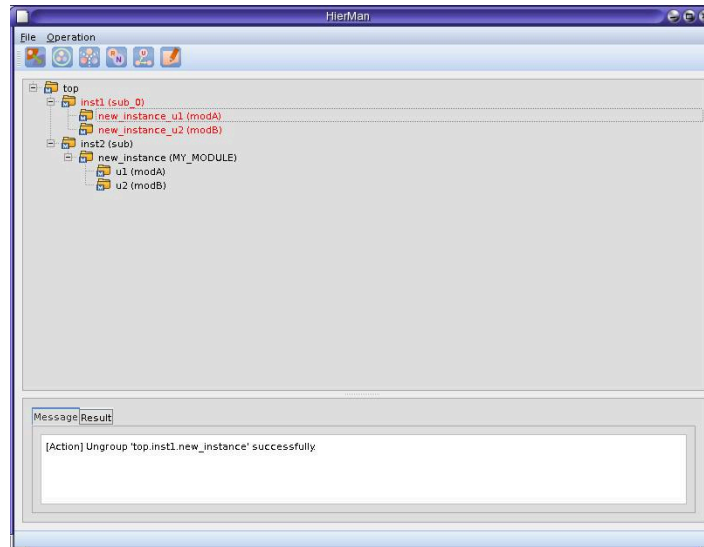
- **Ungroup operation:** Press the “Ungroup” button. The figure below shows the following steps:
 1. Select “top.inst1.new_instance” as the source instance to be ungrouped.
 2. Specify the delimiter “_” used for renaming during ungroup operation.

Figure 33 Ungroup Operation



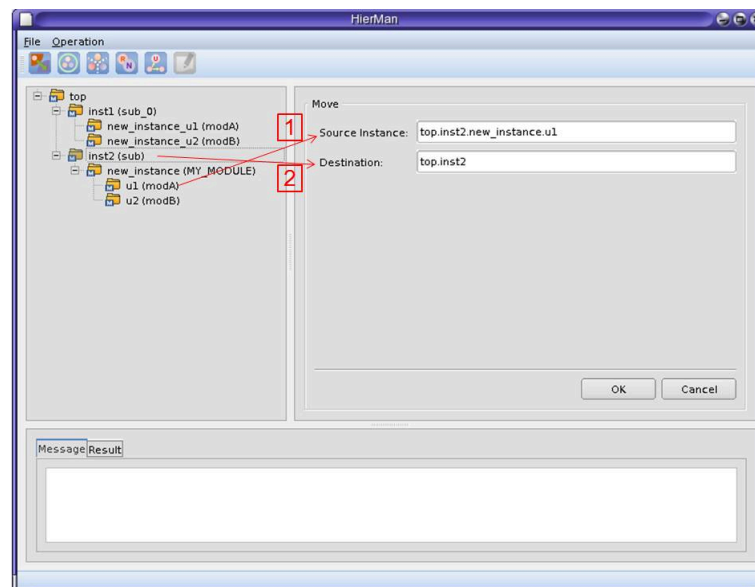
The resultant hierarchy will be shown in the following figure. The red words indicate the modified hierarchical instances. **All connection changes and other necessary modifications are automatically accomplished.**

Figure 34 Hierarchy Tree After Ungroup Operation



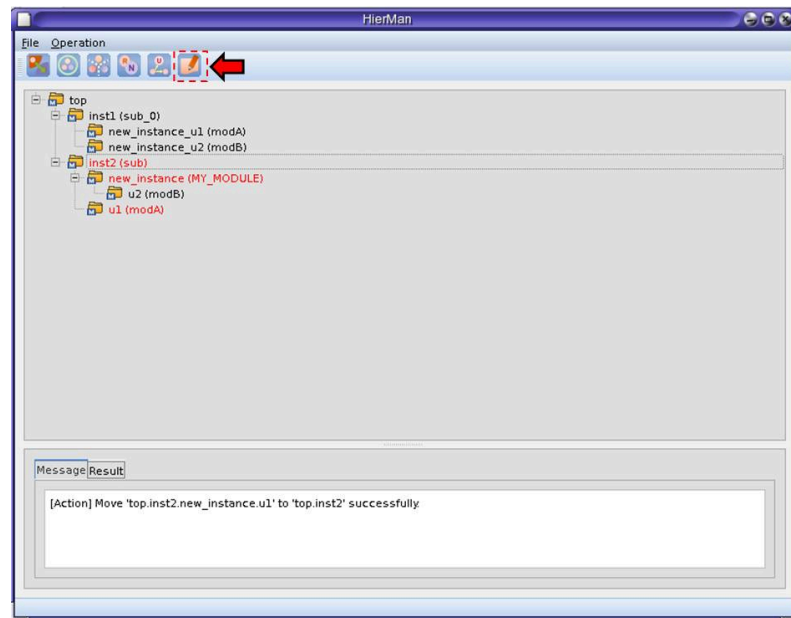
- **Move operation:** Press the “Move” button. The figure below shows the following steps.
 1. Select “top.inst2.new_instance.u1” as the source instance to be moved.
 2. Specify the destination scope “top.inst2” to move to.

Figure 35 Move Operation



The resultant hierarchy will be shown in the following figure. The red words indicate the modified hierarchical instances. **All connection changes and other necessary modifications are automatically accomplished.**

Figure 36 Hierarchy Tree After Move Operation



- **Export operation:**

- Press the “Export” button. The modified design files are shown in the “Result” tab in the message log window.
- Press the magnifier icon. The difference window is shown to for the original source file and the modified one.

Note:

The modified design files are exported to the working directory "HM_DIR".

Figure 37 Modified Design Files After Pressing Export Button

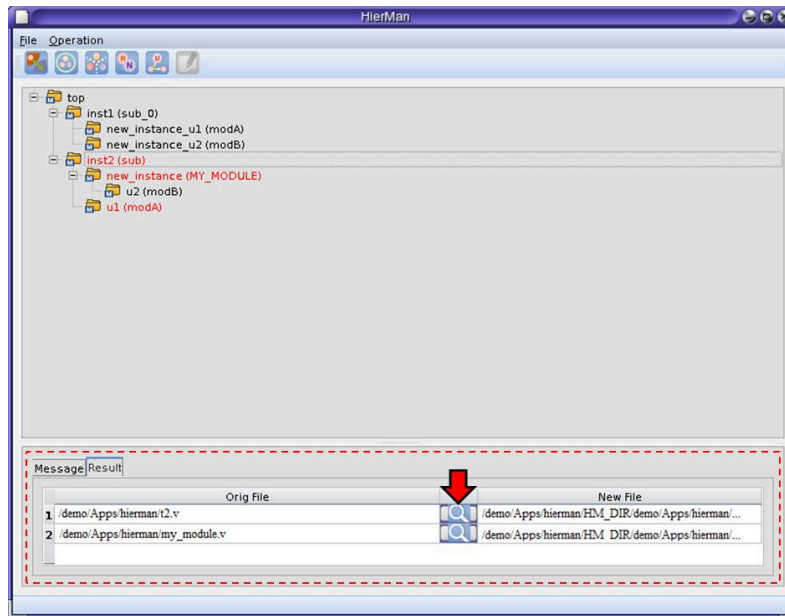
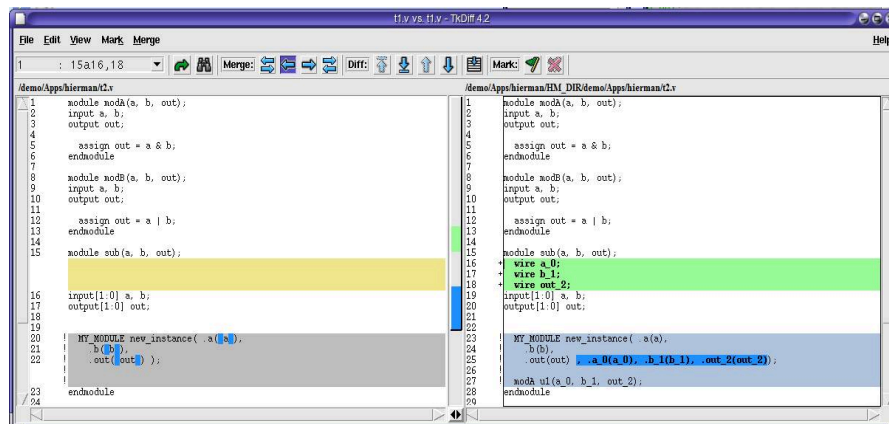


Figure 38 Difference Between Original Source File and Modified file



PortTieConst



Full Name

Tie Port to Constant

Description

Tie constant literal to user specified module input port.

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignManipulation/PortTieConst`

Tcl Entry File

`ptc.tcl`

Tcl Procedure Entry

`ptc_main {}`

Arguments

N/A

Batch Mode Usage

N/A

Limitations

Currently the App only ties constant literal to an entire port. That is, bit-selection or part-selection of a port will be ignored and behaves like an entire port. For example, when a port "top.u1.p[2:1]" is specified to tie a constant 2'b11, the App will ignore the range of the

selection "[2:1]" and then ties the entire range of the port, say "top.u1.p[7:0]" to a constant 2'b11.

Usage

The main window of PortTieConst is shown on Figure 1.

1. PortTieConst table:

- You should put full hierarchy name of input port in first column, and put literal value on second column. The output port or **inout** port is not allowed.
- You can drag a port name from Verdi and then drop to the port name column of PortTieConst window.

2. Add/Delete row:

- You can add/delete data of table.

3. Log dialog:

- Any error and warning messages will be shown in the dialog, and the resultant design file list will be shown on the Result tab.

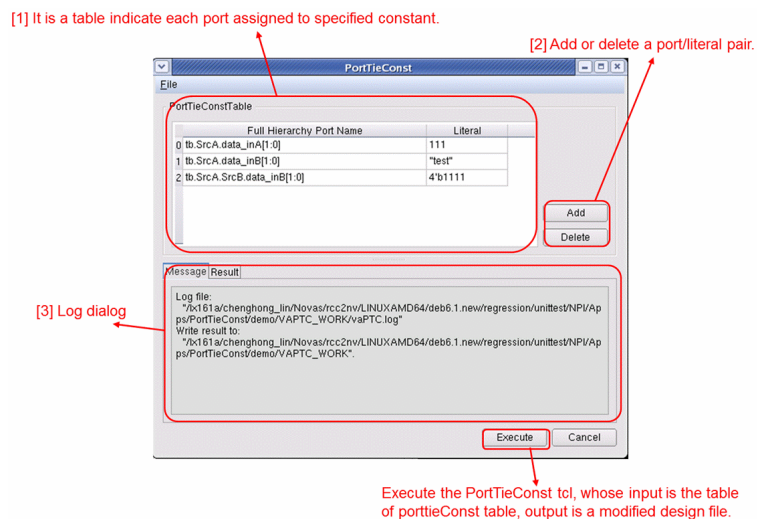


Figure 1: PortTieConst main window example

The table can load/save a CSV file format to do the App. The CSV file example is as follows:

=====

```
tb.SrcA.data_inB[1:0], 2'b10
tb.SrcA.SrcB.Tar.inA[1:0], 2
tb.SrcA.SrcB.Tar.inB[1:0], X
tb.SrcA.SrcB.data_inA[7:0], 8'b00111100
```

```
tb.SrcA.SrcB.data_inB, 0
tb.SrcA.data_inA, 16'hFFFF
```

=====

After pressing the "Execute" button, the modified design files will be written in the directory "VAPTC_WORK". You can review the difference between the modified design files and the original ones.

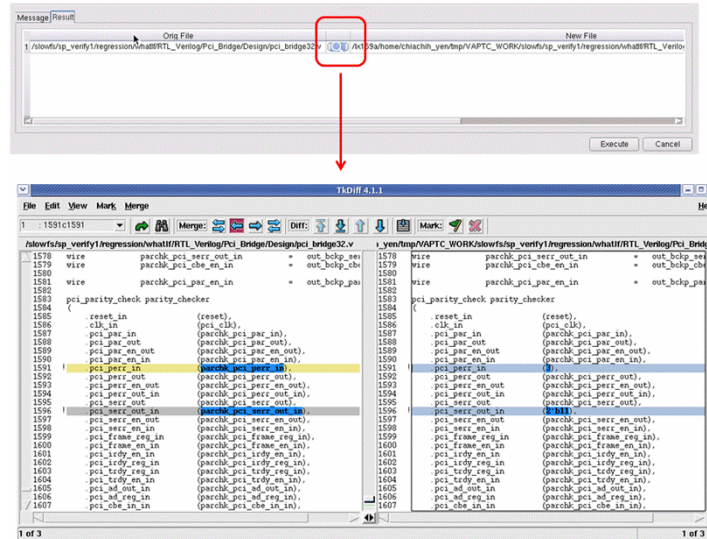


Figure 2: PortTieConst results

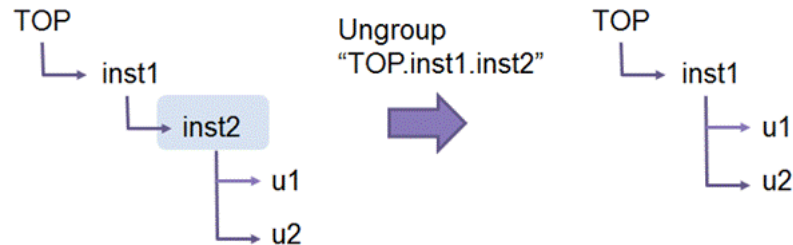
Ungroup



Description

Ungroup a module instance. For example, if originally a design has a module instance "TOP.inst1.inst2", which contains two sub-instances "u1" and "u2". Then after ungrouping the instance "TOP.inst1.inst2", the instances "u1" and "u2" will become the sub-instances

of "TOP.inst1". There will be no "TOP.inst1.inst2" anymore. **All connection changes and other necessary modifications** are automatically accomplished.



Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignManipulation/Ungroup`

Tcl Entry File

`ungroup.tcl`

Tcl Procedure Entry

```
ungroup_main { }
```

Arguments

N/A

Batch Mode Usage

N/A

Limitations

N/A

Usage

There are three blocks in ungroup window (see Figure1):

1. Work directory:
 - Define the working directory for the Ungroup App. The result logs and the modified design files will be generated in the directory.
 - The working directory should be a non-existed directory.
2. Instance hierarchy tree:
 - You can browse instance hierarchy tree in this block and the selected instance hierarchy is shown in (3) automatically.

3. Ungroup instance hierarchy:

- You can fill the line context manually or through (2).

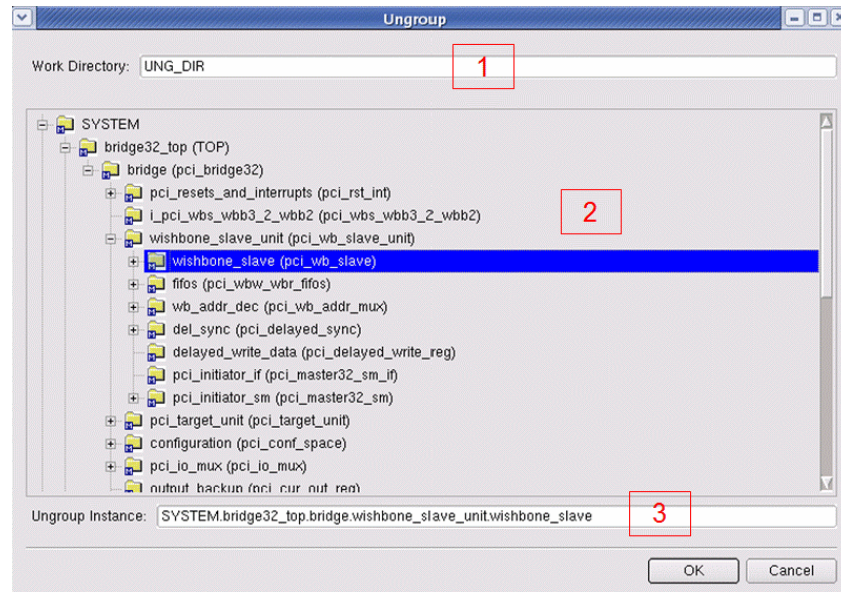


Figure 1: Ungroup main window

After pressing the "OK" button, the Ungroup report window will be shown as follows (Figure 2).

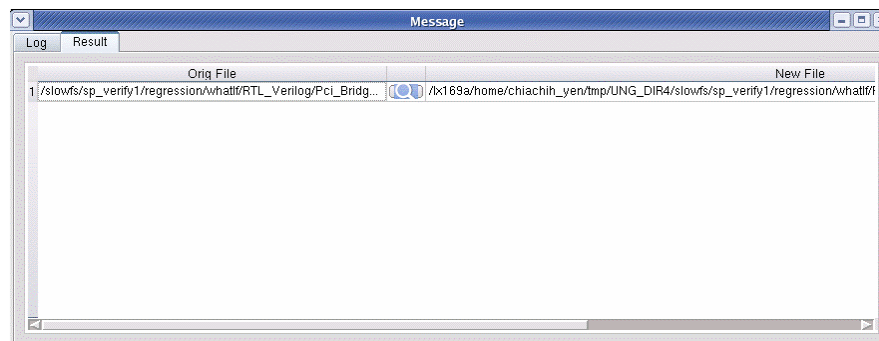


Figure 2: The Ungroup report window

Rename Instance



Description

Given an input instance file list and a postfix string, this App will rename the instance name with the specified prefix and/or postfix.

The instance file is in the format of hierarchical name per line.

Take the following case as an example. In the inst_file "inst.list"

=====

```
system.i_cpu.i_ALUB.i_alu.U133
system.i_cpu.i_ALUB.i_alu.U140
system.i_cpu.i_ALUB.U243
system.i_cpu.i_CCU.C21_reg
system.i_cpu.i_CCU.add_84.U1_3
```

=====

And the prefix is "my_", postfix is "_protect"

The "renameInstance" App will generate the verilog file in the output directory.

When re-import the design with the generated files, the inst will become

=====

```
system.i_cpu.i_ALUB.i_alu.my_U133_protect
system.i_cpu.i_ALUB.i_alu.my_U140_protect
system.i_cpu.i_ALUB.my_U243_protect
```

```
system.i_cpu.i_CCU.my_C21_reg_protect
system.i_cpu.i_CCU.add_84.my_U1_3_protect
=====
```

Tcl Script Location

\$VERDI_HOME/share/VIA/Apps/DesignManipulation/RenameInstance

Tcl Entry File

renameInstance.tcl

Tcl Procedure Entry

```
proc rename_instance_main { inst_file prefix postfix output_dir }
```

Arguments

inst_file:

Specify the instance file name.

prefix:

Specify the prefix string.

postfix:

Specify the postfix string.

output_dir:

Specify the output directory name. The modified source files will be put in the directory.

Batch Mode Usage

```
Usage: renameInstance_batch.pl <verdi_import_design> \
      -inst_file <inst_file_name> \
      -prefix <prefix_string> \
      -postfix <postfix_string> \
      [-out_dir <output_dir>] \
      [-h]
```

For example:

```
./renameInstance_batch.pl -f run.f -inst_file inst.list -
prefix "my_" -postfix "_eco" -out_dir "Rename_Inst"
```

Limitations

This App is currently only supports pure Verilog/SV. The VHDL designs are treated as a limitation.

Example

```
SYSTEM.bridge32_top.bridge.i_pci_wbs.wbb3_2_wbb2
SYSTEM.bridge32_top.bridge.input_register
SYSTEM.bridge32_top.bridge.output_backup
SYSTEM.bridge32_top.bridge.parity_checker.par_gen
SYSTEM.bridge32_top.bridge.parity_checker.serr_en_crit_gen
SYSTEM.bridge32_top.bridge.parity_checker.serr_crit_gen
SYSTEM.bridge32_top.bridge.wishbone_slave_unit.del_sync.comp_sync
SYSTEM.bridge32_top.bridge.wishbone_slave_unit.del_sync.req_sync
```

Instance File: inst.log

Rename Instance

Instance File: /inst.log

Prefix: MV_

Postfix: _POST

Output Directory: Rename_Inst

OK Cancel

rename_inst.log

```
1 *****
2 * VIA App: Rename Instance
3 * Report : rename_inst.log
4 * Date : Tue Feb 11 15:56:05 2014
5 *****
6
7 *****
8 Instance File: /inst.log
9 Prefix: MV_
10 Postfix: _POST
11 Output Dir: Rename_Inst
12 *****
13
14 Line 5: Rename successfully: SYSTEM.bridge32_top.bridge.i_pci_wbs.wbb3_2_wbb2 (MV_i_pci_wbs.wbb3_2_wbb2_POST)
15 Line 3: Rename successfully: SYSTEM.bridge32_top.bridge.input_register (MV_input_register_POST)
16 Line 3: Rename successfully: SYSTEM.bridge32_top.bridge.output_backup (MV_output_backup_POST)
17 Line 4: Rename successfully: SYSTEM.bridge32_top.bridge.parity_checker.par_gen (MV_par_gen_POST)
18 Line 5: Rename successfully: SYSTEM.bridge32_top.bridge.parity_checker.serr_en_crit_gen (MV_serr_en_crit_gen_POST)
19 Line 5: Rename successfully: SYSTEM.bridge32_top.bridge.parity_checker.serr_crit_gen (MV_serr_crit_gen_POST)
20 Line 7: Rename successfully: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.del_sync.comp_sync (MV_comp_sync_POST)
21 Line 8: Rename successfully: SYSTEM.bridge32_top.bridge.wishbone_slave_unit.del_sync.req_sync (MV_req_sync_POST)
```

The log after applying RenameInst

pci_delayed_sync.v vs. pci_delayed_sync.v - TkDiff 4.2

File Edit View Mark Merge

1: 233c233

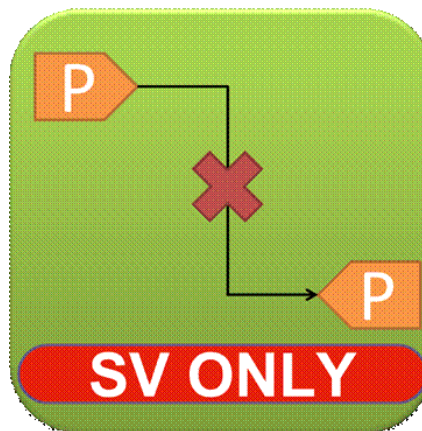
main_inst/slowfs/tp_verify/regression/whatif/RTL_Verilog/Pci_Bridge/Design/pci_delayed_sync

```
220 begin
221 if (reset_in)
222 req_req_pending <= #PF_DELAY 1'b0;
223 else
224 if (req_req_clear)
225 req_req_pending <= #PF_DELAY 1'b0;
226 else
227 if (req_in)
228 req_req_pending <= #PF_DELAY 1'b1;
229 end
230
231 // intermediate stage request synchronization flip - flop - this one is
232 // and should have setup and hold times disabled during simulation
233 pci_synchronizer_flop #(1, 0) MV_req_sync_RESET
234 (
235 .data_in (req_req_pending),
236 .clk_out (comp_clk_in),
237 .sync_data_out (sync_comp_req_pending),
238 .async_reset (reset_in)
239 );
240
241 // wire for clearing completion side request flag - whenever completion
242 wire comp_req_pending_clear = comp_req_pending && (comp_in || retry_req);
243
244 // wire for enabling request flip - flop - it is enabled when completion
245 wire comp_req_pending_ena = ~comp_comp_pending && ~comp_done_req_main;
246
247 // completion side request flip flop - gets a value from intermediate st
248 always@(posedge comp_clk_in or posedge reset_in)
249 begin
```

1 of 2

The source file difference after RenameInst

Delete Pass Through



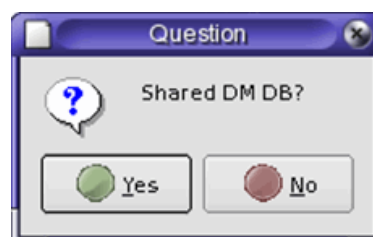
Description

Delete the pass through the path based on the port list file. The Delete Pass Through App shares the same database with the following Apps:

- Chip Integration
- Hierarchy Manipulation
- Disconnect Port

Assume the database has been updated when using one of these Apps. When one of the rest Apps is invoked, the *Question* form will be opened to verify the request of using the updated database. Click the **Yes** button to use the updated database. Click the **No** button to use the original database and abandon all updates to the database.

Figure 39 Question Form



Tcl Script Location

```
$VERDI_HOME/share/VIA/Apps/DesignManipulation/DeletePassThrough
```

Tcl Entry File

```
delete_pass_through.tcl
```

Tcl Procedure Entry

```
delete_pass_through_main { }
```

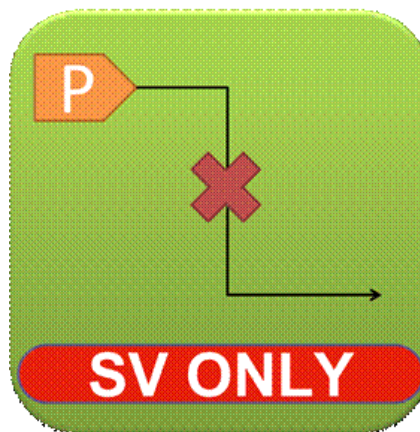
Arguments

N/A

Batch Mode Usage

N/A

Disconnect Port



Description

Disconnect ports within port list file. The Disconnect Port App shares the same database with the following Apps:

- Chip Integration
- Hierarchy Manipulation
- Delete Pass Through

Assume the database has been updated when using one of these Apps. When one of the rest Apps is invoked, the *Question* form will be opened to verify the request of using the updated database. Click the **Yes** button to use the updated database. Click the **No** button to use the original database and abandon all updates to the database.



Question Form

Tcl Script Location

`$VERDI_HOME/share/VIA/Apps/DesignManipulation/Discon`

Tcl Entry File

`discon.tcl`

Tcl Procedure Entry

`discon_main { }`

Arguments

N/A

Batch Mode Usage

N/A

Fast ECO Mode

Description

Supports the use of the VIA engine along with nECO to run ECO script commands without the GUI.

Syntax

```
verdi -fastECO -ecoScript scriptFileName +design
```

Arguments

`scriptFileName`

Specify the absolute file name or relative file name.

`+design`

Indicates a Verdi identifiable import design option such as `-simBin simv`, `-f run.f`, `-lib work -top`, and so on.

Limitations

This feature has the following limitations:

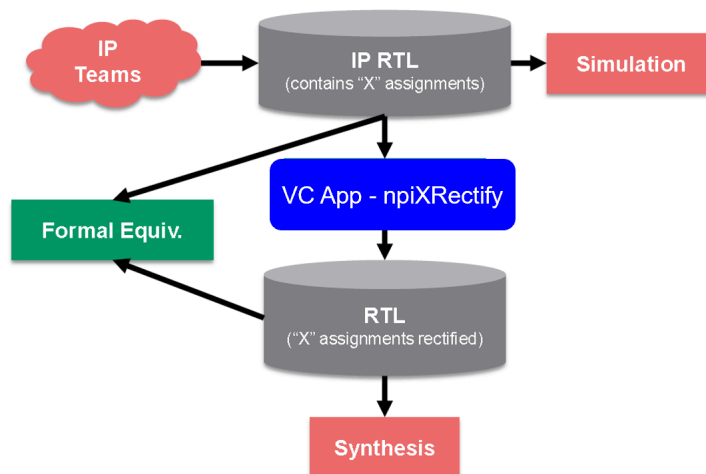
- It generates ICC format script similar to prior releases.
- It works only on batch mode and cannot work with the `ecoLoadScript` command or other GUI commands.
- You cannot preview the result and hence you must save to view the result.
- The `ECOChangeInstScope` command is not supported in the fast ECO mode.

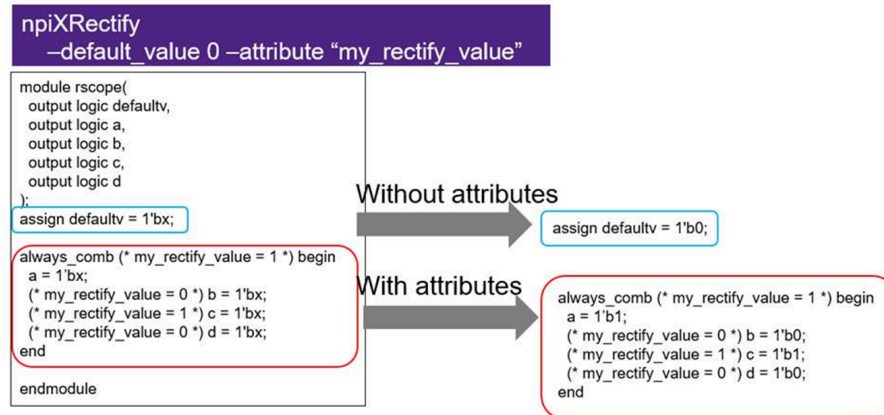
X Rectify

Description

Sometimes RTL IP contain “X” assignments in the development and simulation phase and the “X” assignment may cause non-deterministic synthesized logic (0 or 1). Therefore, using **X rectify**, an additional step is added before synthesis to remove explicit “X” assignments and force them to a specific “0” or “1” value. Removal of “X” assignments enables a deterministic synthesized logic that is compatible over multiple product generations

Following is an RTL X Rectification Flow:





Syntax

```

npixRectify <verdi_import_design_options> \
  -default_value <0|1> \
  -attribute "<attr>*" \
  -skip "<filename>*" \
  -af <attribute list filename> \
  -sf <skip file list filename> \
  -dir <target directory> \
  -macro_restore_off \
  -file_flatten_view \
  [-h]

```

Examples

To rectify x to 0, use the following option:

```
%> npixRectify -dbdir ./simv.daidir -default_value 0
```

To rectify x to 0, while attribute `my_attr` follows the value to rectify x, use the following option:

```
%> npixRectify -dbdir ./simv.daidir -default_value 0 -attribute "my_attr"
```

To rectify multiple attributes, use the following option:

```
%> npixRectify -dbdir ./simv.daidir -default_value 0 -attribute "my_attr1
my_attr2"
```

To rectify x to 0, while skip some files use the following option:

```
%> npixRectify -f run.f -default_value 0 -attribute "my_attr1 my_attr2"
-skip "sub1.sv sub2.sv"
```

To write out the generated rectification files in directory `out_dir` with flatten view, use the following option:

```
%> npixRectify -dbdir ./simv.daidir -default_value 0 -dir out_dir  
-file_flatten_view
```

To use a file `attr_file.txt` to list all attributes (one attribute per line), and to use a file `skip_file.txt` to list all skip files (one file name per line), use the following option:

```
%> npixRectify -dbdir ./simv.daidir -default_value 0 -af attr_file.txt  
-sf skip_file.txt
```

Design Exploration

This section describes the following topics:

- [Trace Instance Connection](#)
- [Trace Pipeline](#)

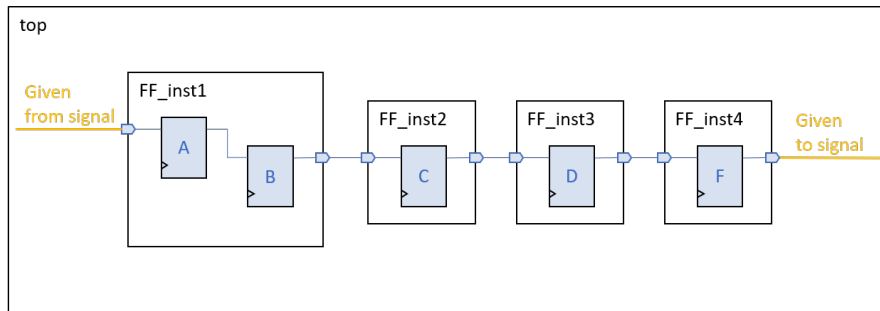
Trace Instance Connection



Reports the connected ports, if the specified instance pairs are connected.

Description

This App traces and displays the connected ports of instances as a pair of from and to instances. The tracing passes through combinational logics and module boundaries. However, it stops at any sequential elements. The report contains the two connected ports, if the specified instance pairs are connected.



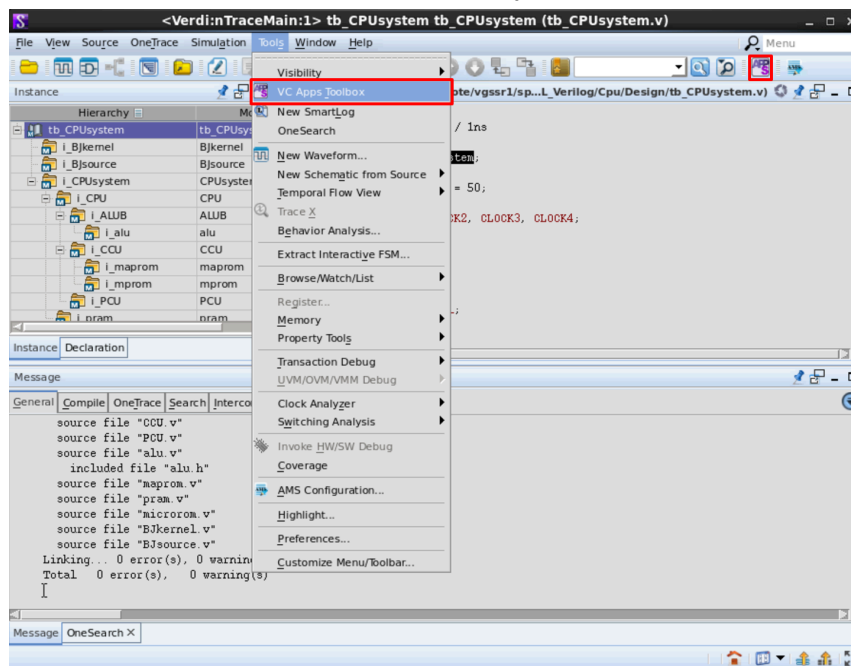
Limitation

None.

How to Open the App in VC Apps Toolbox

To open the App in the VC Apps toolbox, perform the following steps:

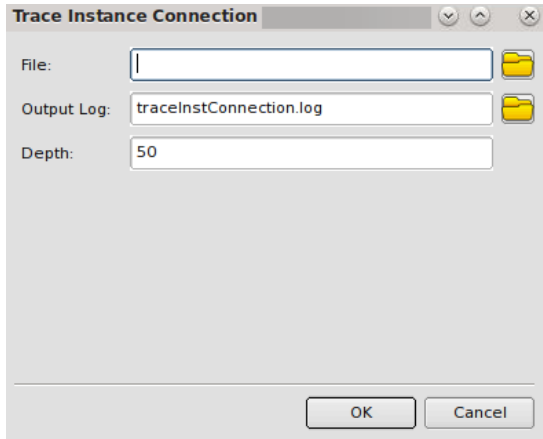
1. Open Verdi with design by selecting **Tools > VC Apps Toolbox**. Otherwise, you can click the **VC Apps Toolbox** icon directly.



2. Go to *Design Exploration* and click the *Trace Instance Connection* icon.



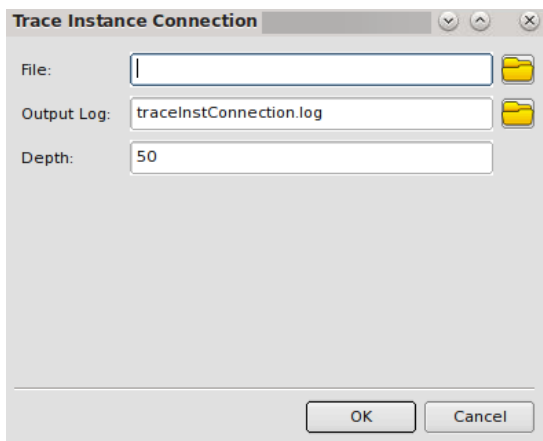
The Trace Instance Connection App opens.



App Usage in Verdi GUI Mode

To use the App in the Verdi GUI mode, perform the following steps:

1. From the **VC Apps toolbox**, open the **Trace Instance Connection** App.

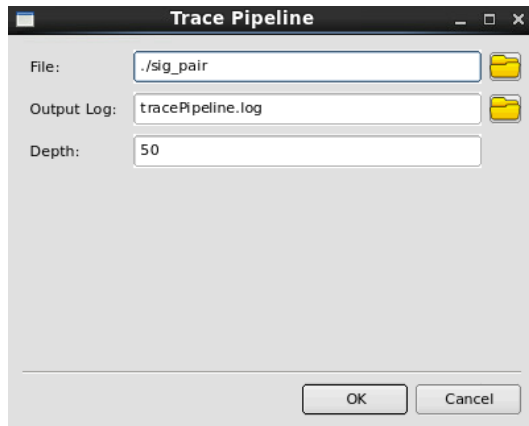


2. Fill in the following fields:

File: Specify the file name of instance pairs with `from_instance => to_instance` in each line.

Output Log: Specify the output log file name.

Depth: Specify the tracing depth based on the netlist model. The default value is 50.



3. Click **OK**. The SmartLog displays the output log, with two connected ports in paths if connected

```

tracePipeline.log X
1 *****
2 * VIA App: tracePipeline.tcl
3 * Report : tracePipeline.log
4 * Date : 2020-10-16 15:20:22
5 *****
6
7 Trace from: pipeline.in
8 Trace to: pipeline.out
9 Path 1:
10     pipeline.in
11     pipeline.flop_name_inst1.in
12     pipeline.flop_name_inst1.w, FF-Q
13     pipeline.flop_name_inst1.out, FF-Q
14     pipeline.in_pipe
15     pipeline.in_pipe_temp
16     pipeline.flop_name_inst2.in
17     pipeline.flop_name_inst2.out, FF-Q
18     pipeline.in_pipe1
19     pipeline.in_pipe1_temp
20     pipeline.flop_name_inst3.in
21     pipeline.flop_name_inst3.out, FF-Q
22     pipeline.in_pipe2
23     pipeline.in_pipe2_temp
24     pipeline.flop_name_inst4.in
25     pipeline.flop_name_inst4.out, FF-Q
26     pipeline.out
27

```

App Usage in Verdi Batch Mode

Script location

\$VERDI_HOME/share/VIA/Apps/Bin/traceInstConnection.pl

Usage

```
traceInstConnection.pl <design_import>  
-file <inst_pair_file>  
[-o <output_log>]  
[-depth <max_tracing_depth>]
```

Options

- **-file:** Specify the file name, which contains source or destination instance full names. The file format is `from_instance => to_instance`.

For example,

```
inst_pair  
=====
```

```
top.inst_b => top.inst_a.inst_a_sub3  
top.inst_c => top.inst_a.inst_a_sub3  
top.inst_a.inst_a_sub3 => top.inst_b  
top.inst_d => top.inst_a.inst_a_sub3  
top.inst_e => top.inst_a.inst_a_sub3  
=====
```

- **-o:** Specify the output log. The default value is `traceInstConnection.log`.
- **-depth:** (Optional) Specify the maximum tracing level. The default value is 50.

Example

```
/> traceInstConnection.pl -sv example.sv -file inst_pair
```

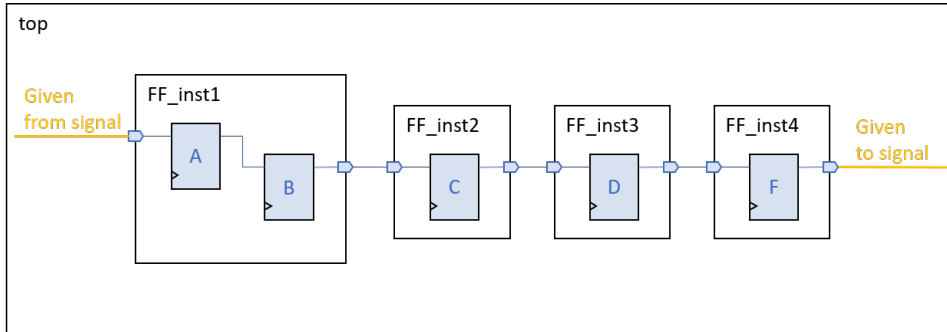
Trace Pipeline



Reports pipeline of registers from the specified signal pairs.

Description

This App traces and displays the pipeline of registers by given signal pairs of from and to signals. The tracing passes through registers from D to Q, module boundaries and assignments. However, it stops at any other combinational logics. The report contains the path. Moreover, FF-Q is also marked in the report.



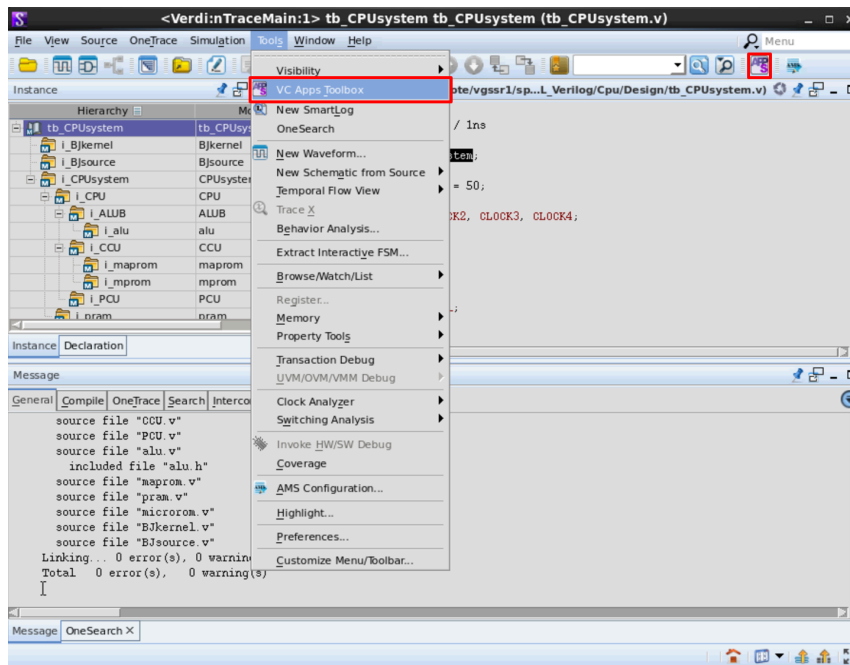
Limitation

None.

How to Open the App in VC Apps Toolbox

To open the App in the VC Apps toolbox, perform the following steps:

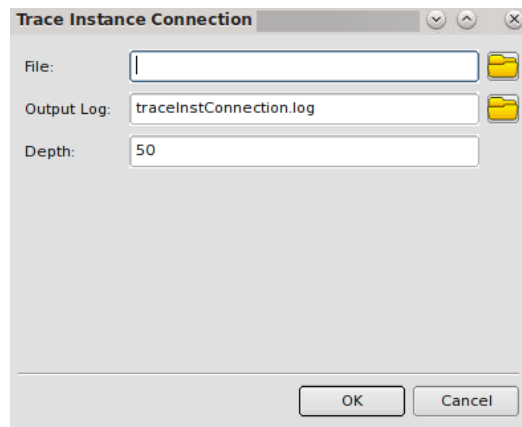
1. Open Verdi with design by selecting **Tools > VC Apps Toolbox**. Otherwise, you can click the **VC Apps Toolbox** icon directly.



2. Go to **Design Exploration** and click the **Trace Pipeline** icon.



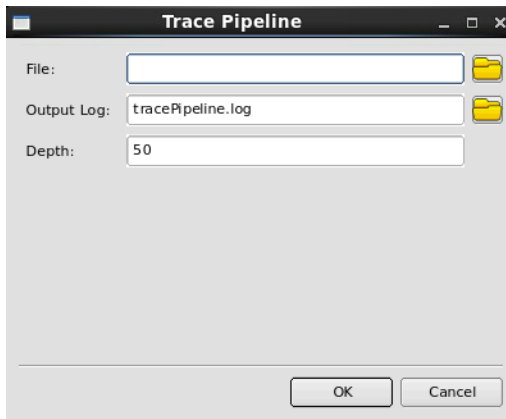
The Trace Pipeline App opens.



App Usage in Verdi GUI Mode

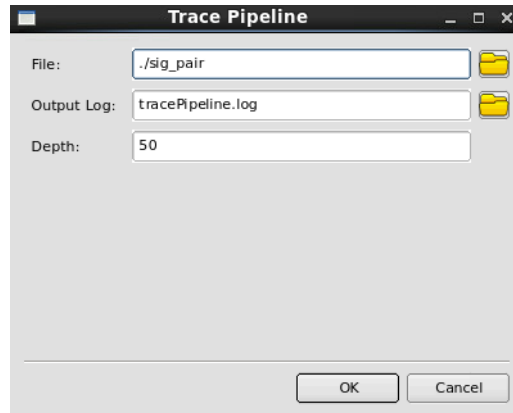
To use the App in the Verdi GUI mode, perform the following steps:

1. From the *VC Apps toolbox*, open the *Trace Pipeline* App.



2. Fill in the following fields:
 - **File:** Specify the file name of signal pairs with *from_signal => to_signal* in each line.
 - **Output Log:** Specify the output log file name.

- **Depth:** Specify the tracing depth based on the netlist model. The default value is 50.



1. Click **OK**. The SmartLog displays the output log, with the pipeline paths and FF-Q marked.

```

1 *****
2 * VIA App: tracePipeline.tcl
3 * Report : tracePipeline.log
4 * Date : 2020-10-16 15:20:22
5 *****
6
7 Trace from: pipeline.in
8 Trace to: pipeline.out
9 Path 1:
10     pipeline.in
11     pipeline.flop_name_inst1.in
12     pipeline.flop_name_inst1.w, FF-Q
13     pipeline.flop_name_inst1.out, FF-Q
14     pipeline.in_pipe
15     pipeline.in_pipe_temp
16     pipeline.flop_name_inst2.in
17     pipeline.flop_name_inst2.out, FF-Q
18     pipeline.in_pipel
19     pipeline.in_pipel_temp
20     pipeline.flop_name_inst3.in
21     pipeline.flop_name_inst3.out, FF-Q
22     pipeline.in_pipe2
23     pipeline.in_pipe2_temp
24     pipeline.flop_name_inst4.in
25     pipeline.flop_name_inst4.out, FF-Q
26     pipeline.out
27

```

App Usage in Verdi Batch Mode

Script location

\$VERDI_HOME/share/VIA/Apps/Bin/tracePipeline.pl

Usage

```
tracePipeline.pl <design_import>  
-file <signal_pair_file>  
[-o <output_log>]  
[-depth <max_tracing_demmarpth>]
```

Options

- **-file:** Specify the file name, which contains source or destination signal full names. The file format is `from_signal => to_signal`.

For example,

```
sig_pair  
=====  
pipeline.in => pipeline.out  
=====
```

- **-o:** Specify the output log. The default value is `tracePipeline.log`.
- **-depth:** (Optional) Specify the maximum tracing level. The default value is `50`.

Example

```
/> tracePipeline.pl -sv pipeline.sv -file sig_pair
```

FSDB Investigation

Report Moving Average

Description

Report Moving Average is an app for power profiling. The input file configuration options are as follows:

- `-fsdb power_fsdb/vf files`
- `-s signal_list`
- `-bt time`

This is optional. It is default as fsdb's begin.

- `-et time`

This is optional. It is default as fsdb's end

- `-ave_size <moving_average_window_size with timeunit>`
- `-ave_offset <time specifies the sliding delta>` This is optional. Default as 10% of the `ave_size`.
- `-ave_dump_fsdb`

This is optional. By default, this option is off.

- `-slope_window <slope_window_size>`

This is optional. Default is same as *ave_offset*.

- `-threshold_slope <threshold of absolute slope value>`

This is optional. It is default to disable max absolute slope searching. Set 0 to show all data

- `-threshold_peak <threshold of peak value>` This is optional. Default is to disable peak searching. Set 0 to show all data
- `-show_top_data <number>`

This is optional, The default value is 10. Set 0 to show all data with ranking.

- `-o filename`

This is optional. Default as `<filename>.peak.log` and `<filename>.slope.log`

The output files are as follows:

- `<moving average output>.fsdb`
- `<filename>.slope.log`
- `<filename>.peak.log`

The function of this app are as follows:

- Dump moving average calculation FSDB

For performing this function, the following input options are required:

- `-fsdb power_fsdb`
- `-s signal_list`
- `-ave_size 10ns`

- -ave_offset 1ns
- -ave_dump_fsdb

The output file is as follows:

<output>.fsdb

- Search peak with moving average

For performing this function, the following input options are required:

- -fsdb power_fsdb
- -s signal_list
- -ave_size 10ns
- -ave_offset 1ns
- -threshold_peak 0

This is optional. Default is to disable peak searching. Set 0 to show all data

- -show_top_data 3

The following output file is generated:

.<filename>.peak.log

The output file format is as follows:

```
#Peak Value#
#Threshold = 800
#Show Top data: 3

<rank>           <signal>           <value>           <timelocation>
#1               TOP.power_1           1000              100ps
#2               TOP.power_2           999               120ps
#3               TOP.power_2           700               90ps
```

- Search Max Slope with moving average

For performing this function, the following input options are required:

- -fsdb power_fsdb
- -s signal_list
- -ave_size 10ns
- -ave_offset 1ns
- -slope_window 30ns #optional, default is same as "ave_offset"

- -threshold_slope 0 #optional, default to disable max absolute slope searching. Set 0 to show all data
- -show_top_data 3

The following output file is generated:

.<filename>.slope.log

The output file format is as follows:

```
#Max Slope range#
      #Slope absolute Threshold = 1.3
      #Show Top data: 3
<rank>          <signal>          <value>
  <time location>
#1              TOP.power_1        1.5
  100ps
#2              TOP.power_2        1.4
  120ps
#2              TOP.power_2        1.3
  90ps
```

Syntax

```
% napi_report_moving_avg -fsdb <FSDB> -s <file> -avg_size <size> [-
ave_dump_fsdb] [-avg_offset <offset>] [-slope_window <win>] [-
threshold_slope <th>] [-threshold_peak <th>] [-show_top_data
<number>] [-bt <beginTime>] [-et <endTime>] [-o <outputFile>]
```

Arguments

-fsdb <FSDB>

A FSDB file.

-s <file>

A file that contains signal list.

-avg_size <size>

Moving average window.

[-ave_dump_fsdb]

Dumps report in FSDB.

[-avg_offset <offset>]

Average window offset (default is 10% of avg_size).

[-slope_window <win>]

Slope window size (default is the same as avg_offset).

```
[-threshold_slope <th>]
```

Threshold of slope value.

```
[-threshold_peak <th>]
```

Threshold of peak value.

```
[-show_top_data <number>]
```

Shows top N data (default is 10, set 0 to show all data).

```
[-bt <time>]
```

Begin time (default is FSDB's min time).

```
[-et <time>]
```

End time (default is FSDB's max time).

```
[-o <output file>]
```

Output file name.

Example

```
mpi_report_moving_avg -ave_dump_fsdb -fsdb power.fsdb -s  
sig.list -avg_size 100ns -avg_offset 20ns -et 20000ns -  
show_top_data 20 -threshold_peak 20.3 -threshold_slope 5.0 -  
slope_window 40ns -o my_report
```